

LAPORAN PROYEK AKHIR

**Optimasi *Task Scheduling* di *Heterogeneous Edge Computing*
Menggunakan *Hybrid Tabu-Diffusion* Berbasis *Multi-Objective*
Energi dan *Latency***



Disusun oleh:

NAMA LENGKAP

Nomor Induk Mahasiswa Lengkap

**PROGRAM STUDI SARJANA TERAPAN TEKNOLOGI
REKAYASA INTERNET
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA YOGYAKARTA
2026**

JUDUL PROYEK AKHIR

**Proyek akhir
Program Studi Teknologi Rekayasa XX**

**Diajukan sebagai syarat kelengkapan studi jenjang Sarjana
Terapan untuk memperoleh gelar Sarjana Terapan
(Teknik/Komputer) pada Program Studi Teknologi Rekayasa XX**

**Oleh:
NAMA LENGKAP
Nomor Induk Mahasiswa Lengkap**

**PROGRAM STUDI SARJANA TERAPAN
TEKNOLOGI REKAYASA XX
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026**

LEMBAR PENGESAHAN

Judul : JUDUL PROYEK AKHIR
Nama : Nama Lengkap
Program Studi : Teknologi Rekayasa XX
Pembimbing : Nama Dosen Pembimbing
Waktu Ujian : Kamis, 01 Januari 2026, Pukul 07:30, Ruang CM102

Telah dipertanggungjawabkan dan diuji oleh Tim Penguji serta disetujui dan disahkan Sebagai syarat kelengkapan studi jenjang Sarjana Terapan Program Studi Teknologi Rekayasa XX Sekolah Vokasi Universitas Gadjah Mada
Yogyakarta, 01 Januari 2026 (Tanggal Ujian)

Tim Penguji

Ketua

Sekretaris

Nama lengkap dan Gelar
NIP.

Nama lengkap dan Gelar
NIP.

Anggota

Nama lengkap dan Gelar
NIP.

Mengetahui,

Ketua Departemen
Teknik Elektro dan Informatika

Ketua Program Studi Teknologi Rekayasa xx

Ir. Yuris Mulya Saputra, S.T., M.Sc., Ph.D., IPM.
NIP. 111198806201605101

Nama lengkap dan Gelar
NIP.

PERNYATAAN KEASLIAN PROYEK AKHIR

Dengan ini, saya NAMA LENGKAP, menyatakan bahwa Proyek Akhir ini adalah asli karya saya sendiri dan belum pernah diajukan sebagai pemenuhan persyaratan untuk memperoleh gelar Sarjana Terapan dari Universitas Gadjah Mada maupun perguruan tinggi lainnya.

Semua informasi yang dimuat dalam Proyek Akhir ini yang berasal dari karya orang lain, baik yang dipublikasikan atau tidak, telah diberikan kutipan nama sumber penulis secara benar dan semua isi dari Proyek Akhir ini sepenuhnya menjadi tanggung jawab penulis.

Yogyakarta, Agustus 2026

NAMA LENGKAP

Nomor Induk Mahasiswa Lengkap

KATA PENGANTAR

Proyek Akhir ini disusun untuk melengkapi salah satu syarat mencapai gelar Sarjana Terapan Program Studi Teknologi Rekayasa XX, Departemen Teknik Elektro dan Informatika, Sekolah Vokasi, Universitas Gadjah Mada.

Dengan selesainya Proyek Akhir ini, Penulis mengucapkan terima kasih kepada :

1. Nama lengkap dan gelar, selaku Ketua Departemen Teknik Elektro dan Informatika, Sekolah Vokasi, Universitas Gadjah Mada
 2. Nama lengkap dan gelar, selaku Ketua Program Studi Teknologi Rekayasa XX, Departemen Teknik Elektro dan Informatika, Sekolah Vokasi, Universitas Gadjah Mada
 3. Nama lengkap dan gelar, selaku Dosen Pembimbing, yang telah
 4. Nama lengkap dan gelar, selaku Dosen Penguji yang telah
 5. Nama lengkap dan gelar, selaku Dosen Pembimbing Akademik yang telah
 6. Dan seterusnya.
-
-

Yogyakarta, tgl/bulan/tahun
Penulis

Nama lengkap
NIM

DAFTAR ISI

HALAMAN JUDUL.....	i
LEMBAR PENGESAHAN	ii
PERNYATAAN KEASLIAN PROYEK AKHIR	iii
KATA PENGANTAR.....	iv
DAFTAR ISI	v
DAFTAR GAMBAR	viii
DAFTAR TABEL.....	ix
DAFTAR SIMBOL	x
INTISARI	xi
<i>ABSTRACT</i>	xii
BAB I PENDAHULUAN.....	1
1.1. Latar belakang.....	1
1.2. Rumusan masalah.....	4
1.3. Tujuan dan manfaat Proyek Akhir	4
1.4. Batasan penelitian	5
1.5. Sistematika penulisan.....	6
BAB II KAJIAN PUSTAKA	8
2.1. Tinjauan Pustaka	8
2.2. Dasar teori	3
2.2.1. <i>Edge Computing</i>	3
2.2.2. <i>Task Schedulling</i>	7
2.2.3. Metode Optimasi	12
2.2.4. <i>Local Optimization</i>	15

2.2.5.	<i>Global Optimization</i>	17
2.2.6.	<i>Latency</i>	23
2.2.7.	<i>Energi</i>	24
2.2.8.	<i>Virtual Private Server (VPS)</i>	25
2.2.1.	<i>Linearisasi</i>	27
2.2.2.	<i>Kalman Filter</i>	27
2.3.	Hipotesis.....	Error! Bookmark not defined.
BAB III	METODE PENELITIAN.....	29
3.1.	Perangkat Penelitian.....	29
3.1.1.	Perangkat Keras	29
3.1.2.	Perangkat Lunak.....	29
3.2.	Tahapan proyek akhir.....	34
3.3.	Pemodelan Sistem.....	38
3.3.1.	Diagram Sistem.....	38
3.3.2.	Pemodelan dengan Program.....	Error! Bookmark not defined.
3.3.3.	Pemodelan dengan VPS	Error! Bookmark not defined.
3.4.	Implementasi Solusi.....	59
3.4.1.	Perancangan objective function.....	59
3.4.2.	Perancangan search space	62
3.4.3.	Penerapan algoritma optimasi.....	73
3.5.	Pengujian	80
3.5.1.	Pengujian menggunakan simulasi program	80
3.5.2.	Pengujian menggunakan simulasi di VPS	81
3.6.	Analisis hasil.....	Error! Bookmark not defined.
4.	Perancangan alat/purwapura	Error! Bookmark not defined.

5.	Tahapan analisa data	Error! Bookmark not defined.
BAB IV	HASIL DAN PEMBAHASAN	82
BAB V	PENUTUP	83
5.1	Kesimpulan	83
5.2	Saran.....	83
DAFTAR PUSTAKA		84
LAMPIRAN		88

DAFTAR GAMBAR

Gambar 2.1. Alur pembentukan dan pemrosesan task dalam sistem EC	11
Gambar 2.2. Ilustrasi permasalahan local optimum dalam ruang solusi optimasi	20
Gambar 2.3 Alur proses pemilihan solusi dalam algoritma Tabu Search [4]	21
Gambar II-4 Model aerodinamik Fixed Wing.....	27
Gambar 3.III-1. Diagram Alir Tahapan Penelitian	Error! Bookmark not defined.
Gambar 3.III-2. Arsitektur sistem edge computing berbasis central node dan edge nodes	Error! Bookmark not defined.
Gambar 3.3. Topologi edge nodes berbentuk grid 3x3.....	43

DAFTAR TABEL

Tabel 2.1 Perbandingan penelitian.....	Error! Bookmark not defined.
Tabel 2.2 Parameter sistem fixed wing.....	27

DAFTAR SIMBOL

[SAMPLE]

ω	=	Kecepatan putar motor DC (<i>rpm</i>)
θ	=	Posisi sudut motor DC (0)
τ	=	Konstanta waktu
t_s	=	<i>Settling time</i>
K_p	=	Konstanta proporsional
K_i	=	Konstanta integral
K_d	=	Konstanta derivative
PID	=	<i>Proportional Integral Derivative</i>

INTISARI

OPTIMASI *TASK SCHEDULING* DI *HETEROGENEOUS EDGE* *COMPUTING* MENGGUNAKAN *HYBRID* TABU-DIFFUSION BERBASIS *MULTI-OBJECTIVE* ENERGI DAN *LATENCY*

<Nama Lengkap>

<Nomor Induk Mahasiswa Lengkap>

[SAMPLE]

Teknologi Java berkembang pesat dalam bidang teknologi informasi. Salah satu teknologi Java yang menarik untuk dipelajari adalah framework web Java Struts 2 yang mendukung design pattern MVC (Model-View-Controller) serta framework ORM (Object/Relational Mapping) Hibernate yang mendukung pemetaan basis data MySQL. Struts 2 sebagai alternatif dalam membangun web Java dapat diimplementasikan bersama dengan Hibernate melakukan fungsi utama sebuah sistem seperti proses menciptakan (create), membaca (read), memperbarui (update), dan menghapus (delete) data.

Sistem informasi Proyek Akhir sebagai studi kasus permasalahan ini dibangun berdasarkan survei pada Program Diploma Teknik Elektro Sekolah Vokasi UGM yang belum memiliki sistem pendataan kerja praktek, magang dan tugas akhir. Implementasi framework web Java Struts 2 dan ORM Hibernate pada sistem ini dirasa tepat sebagai bentuk pembelajaran dan alternatif teknologi yang digunakan selain penggunaan framework web PHP atau Java lainnya.

Kata kunci : Sistem informasi, web Java, Struts2, ORM, Hibernate

ABSTRACT

IMPLEMENTATION OF JAVA STRUTS 2 WEB FRAMEWORK AND OBJECT/RELATIONAL MAPPING (ORM) HIBERNATE IN THE FINAL PROJECT INFORMATION SYSTEM

<Full Name>

<Full Student ID>

[SAMPLE]

Java technology is developing rapidly in the field of information technology. One of the java technologies that are interesting to learn is the Java Struts 2 web framework which supports MVC (Model-View-Controller) design patterns and the Hibernate ORM (Object/Relational Mapping) framework which supports MySQL database mapping. Struts 2 as an alternative in java web building can be implemented together with Hibernate performing the main functions of a system such as the process of creating (create) (create) (read) (read) (update) (update) and deleting (delete) data.

The Final Project information system as a case study of this problem was built based on a survey in the Electrical Engineering Diploma Program at the Vocational School UGM which does not yet have a data collection system for practical work, internships and final projects. The implementation of the Java Struts 2 web framework and Hibernate ORM in this system is considered appropriate as a form of learning and alternative technology used in addition to the use of php or other Java web frameworks.

Keyword: information system, Java web, Struts2, ORM, Hibernate

BAB I

PENDAHULUAN

1.1. Latar belakang

Perkembangan *Internet of Things (IoT)* mendorong peningkatan jumlah perangkat yang saling terhubung dan menghasilkan volume data dalam skala sangat besar. Pada tahun 2024, jumlah perangkat IoT yang terhubung mencapai 18,5 miliar unit dan diproyeksikan meningkat hingga 39,9 miliar unit pada tahun 2030, sehingga kebutuhan pemrosesan data yang cepat dan efisien menjadi semakin tinggi [1]. Peningkatan jumlah perangkat dan volume data menuntut sistem komputasi yang mampu memproses data secara responsif dan mendukung layanan berbasis *real-time*. Pendekatan komputasi berbasis *cloud* yang terpusat sering kali menghadapi kendala berupa peningkatan *latency*, beban jaringan, serta ketergantungan terhadap konektivitas yang stabil. Untuk mengatasi keterbatasan tersebut, *edge computing* berkembang sebagai solusi dengan menempatkan proses komputasi lebih dekat dengan sumber data atau perangkat pengguna sehingga respons sistem dapat ditingkatkan dan beban komunikasi ke pusat data dapat dikurangi [2].

Kebutuhan terhadap *edge computing* semakin meningkat seiring berkembangnya teknologi 5G dan berbagai aplikasi yang membutuhkan responsivitas tinggi. Aplikasi seperti *augmented reality*, *virtual reality*, *autonomous vehicles*, dan sistem kesehatan *real-time* memerlukan *latency* yang sangat rendah serta kemampuan pengambilan keputusan secara cepat [3]. Karakteristik aplikasi tersebut menuntut sistem komputasi yang mampu memproses data secara lokal tanpa bergantung pada pusat data terpusat. *Edge computing* memungkinkan pemrosesan data dilakukan secara terdistribusi pada beberapa *edge nodes* yang berada di dekat pengguna. Pendekatan ini mendukung peningkatan kualitas layanan pada aplikasi yang bersifat *time-sensitive* serta mengurangi keterlambatan dalam pengiriman dan pemrosesan data.

Selain meningkatkan performa sistem, *edge computing* juga berperan dalam meningkatkan efisiensi penggunaan sumber daya dan energi. Pemrosesan data secara lokal pada *edge nodes* dapat mengurangi volume data yang harus dikirim ke pusat data sehingga beban jaringan menjadi lebih ringan [4]. Pengurangan proses transmisi data dalam skala besar berkontribusi terhadap penurunan konsumsi energi pada jaringan komunikasi. Sistem berbasis *edge computing* juga memungkinkan data sensitif diproses secara lokal sehingga meningkatkan aspek keamanan dan privasi [5]. Karakteristik ini menjadikan *edge computing* sebagai pendekatan yang relevan untuk mendukung sistem komputasi yang efisien, responsif, dan aman.

Implementasi *edge computing* menghadapi berbagai tantangan pada pengelolaan sumber daya. Kapasitas CPU, memori, dan daya pada *edge devices* umumnya terbatas sehingga kemampuan pemrosesan setiap node menjadi tidak merata. Lingkungan *edge computing* juga bersifat heterogen karena setiap node memiliki karakteristik sumber daya yang berbeda. Variasi *workload* yang masuk ke sistem semakin meningkatkan kompleksitas dalam proses pengelolaan tugas. Kondisi ini menyebabkan proses *task scheduling* menjadi lebih sulit karena harus mempertimbangkan keterbatasan sumber daya dan heterogenitas node secara bersamaan [6], [7].

Heterogenitas sumber daya dan distribusi beban komputasi yang tidak seimbang dapat menyebabkan peningkatan *latency* dan penurunan performa sistem. Node dengan beban tinggi dapat mengalami antrean tugas yang panjang sehingga waktu pemrosesan menjadi lebih lama. Kondisi ini berdampak langsung pada kualitas layanan yang diberikan kepada pengguna. Sistem yang menangani aplikasi *time-sensitive* membutuhkan respons cepat dan stabil untuk menjaga keandalan layanan. Ketidakseimbangan distribusi beban dapat menghambat pencapaian performa sistem yang optimal dan menurunkan efisiensi pemanfaatan sumber daya.

Distribusi beban komputasi yang tidak optimal juga berdampak pada peningkatan konsumsi energi. Node yang bekerja di atas kapasitas optimal akan

menggunakan daya yang lebih besar dibandingkan kondisi normal. Pemanfaatan sumber daya yang tidak efisien dapat meningkatkan biaya operasional sistem secara keseluruhan. Sistem komputasi modern dituntut untuk tidak hanya memiliki performa tinggi, tetapi juga efisien dalam penggunaan energi. Oleh karena itu, diperlukan pendekatan yang mampu menjaga keseimbangan antara performa sistem dan efisiensi energi [8].

Permasalahan pengelolaan sumber daya pada *edge computing* dapat diatasi melalui mekanisme *task scheduling* yang efisien dan adaptif. Proses *task scheduling* bertujuan untuk menentukan alokasi tugas ke node yang paling sesuai berdasarkan kondisi sistem. Pendekatan berbasis metode konvensional maupun *single optimization method* masih memiliki keterbatasan dalam menangani kompleksitas lingkungan yang dinamis dan heterogen. Sistem *edge computing* membutuhkan pendekatan yang mampu mengakomodasi lebih dari satu tujuan secara simultan. Kebutuhan ini mendorong penggunaan metode optimasi yang lebih fleksibel dan adaptif [9].

Perkembangan penelitian menunjukkan bahwa pendekatan *hybrid optimization* dan *multi-objective optimization* mampu memberikan solusi yang lebih efektif dalam mengatasi permasalahan pada *edge computing* [10]. Pendekatan ini memungkinkan optimasi beberapa parameter secara bersamaan, seperti *latency* dan konsumsi energi. Integrasi antara mekanisme optimasi global dan penyeimbangan beban menjadi aspek penting dalam meningkatkan kualitas solusi. Penelitian ini mengusulkan pendekatan *hybrid optimization* untuk *task scheduling* pada lingkungan *heterogeneous edge computing*, dengan mengintegrasikan mekanisme optimasi global untuk mengeksplorasi ruang solusi secara menyeluruh dan mekanisme *local balancing* untuk mendistribusi beban antar *node* secara adaptif berdasarkan topologi sistem. Pendekatan *hybrid* ini dirancang untuk mengoptimalkan dua tujuan secara simultan, yaitu minimisasi konsumsi energi dan minimisasi *latency*, sehingga diharapkan mampu menghasilkan solusi *scheduling* yang lebih efisien dan seimbang pada lingkungan *edge computing* yang heterogen.

1.2. Rumusan masalah

Peningkatan beban komputasi pada *edge nodes* dengan sumber daya yang heterogen dapat menyebabkan kenaikan *latency* dan konsumsi energi. Kondisi ini menunjukkan perlunya mekanisme optimasi yang mampu mengelola distribusi tugas secara efisien dalam sistem komputasi *edge*. Oleh karena itu, permasalahan yang diangkat dalam Proyek Akhir ini meliputi:

1. Bagaimana menurunkan *latency* dan konsumsi energi pada sistem komputasi *edge* dengan mempertimbangkan heterogenitas sumber daya antar node?
2. Bagaimana mengimplementasikan metode optimasi *latency* dan konsumsi energi pada sistem komputasi *edge* menggunakan lingkungan *Virtual Private Server (VPS)*?

1.3. Tujuan dan manfaat Proyek Akhir

Penelitian ini bertujuan untuk mengembangkan dan mengimplementasikan metode optimasi guna menurunkan konsumsi energi dan *latency* pada sistem komputasi *edge*. Adapun tujuan penelitian ini adalah sebagai berikut:

1. Menyusun metode optimasi dan merumuskan fungsi objektif untuk meminimalkan konsumsi energi dan *latency* pada sistem komputasi *edge* dengan beberapa *edge nodes* heterogen.
2. Menyusun model komputasi *edge* di VPS dan menerapkan metode optimasi yang telah dibuat serta mengevaluasi kinerjanya dalam menurunkan *latency* dan konsumsi energi.

1.4. Batasan penelitian

Agar penelitian ini memiliki ruang lingkup yang terarah dan pembahasan tetap fokus sesuai tujuan yang telah ditetapkan, diperlukan batasan penelitian untuk mendefinisikan aspek-aspek yang menjadi cakupan penelitian maupun yang tidak dibahas dalam Proyek Akhir ini. Batasan penelitian ini disusun untuk menghindari perluasan pembahasan yang berada di luar fokus penelitian, serta memperjelas ruang lingkup penerapan metode yang diusulkan. Adapun batasan penelitian dalam Proyek Akhir ini adalah sebagai berikut:

1. Penelitian difokuskan pada optimasi *task scheduling* pada lingkungan *heterogeneous edge computing*, dengan mempertimbangkan perbedaan kapasitas sumber daya antar node, khususnya CPU dan memori, tanpa membahas heterogenitas perangkat berbasis akselerator khusus seperti GPU, TPU, atau *specialized hardware* lainnya.
2. Penelitian menggunakan pendekatan *hybrid optimization* berbasis *Tabu Search* sebagai mekanisme optimasi global dan *diffusion-inspired local balancing* sebagai mekanisme redistribusi beban lokal. Beberapa metode optimasi lain digunakan sebagai pembandingan untuk mengevaluasi performa, dan berdasarkan hasil evaluasi dipilih metode *Tabu-Diff* sebagai pendekatan utama yang menjadi fokus pengembangan dan pembahasan penelitian.
3. Evaluasi penelitian dilakukan dalam lingkungan simulasi (*simulation-based evaluation*) dan belum diimplementasikan pada sistem *real-time edge deployment* atau lingkungan produksi nyata.
4. Model sistem yang digunakan dibatasi pada skenario *task scheduling* dan distribusi beban antar node *edge*.
5. Parameter yang dioptimasi pada penelitian dibatasi pada minimisasi *latency* dan *energy consumption*. Adapun evaluasi performa metode yang diusulkan dibatasi menggunakan metrik *latency*, *energy consumption*, *resource utilization*, dan *load balancing*.

6. Mekanisme *diffusion-inspired local balancing* pada penelitian dibatasi pada redistribusi beban berbasis *node adjacency* dan kondisi utilisasi sumber daya, sehingga tidak membahas model *diffusion* berbasis fisika lanjutan, *stochastic diffusion models*, maupun pendekatan *graph neural network-based balancing*.
7. Dataset Google Cluster Trace digunakan sebagai sumber *benchmark workload* dan *resource trace* untuk membangun karakteristik node dan *task* pada fase simulasi. Dataset ini tidak digunakan sebagai representasi langsung dari sistem *edge computing*, tetapi sebagai dasar pembentukan beban kerja dan variasi sumber daya yang relevan untuk mengevaluasi metode *task scheduling* pada lingkungan *heterogeneous edge computing*.
8. Penelitian menggunakan *benchmark workload* berbasis dataset untuk simulasi model serta *task generator* berbasis skenario terkontrol pada lingkungan VM untuk merepresentasikan variasi beban kerja dan tekanan CPU. Namun, penelitian ini belum membahas *real-time task arrivals* yang bersifat kontinu dan tidak terprediksi penuh pada sistem *real-time edge deployment*.

1.5. Sistematika penulisan

Sistematika penulisan membahas tentang penjelasan setiap BAB yang dimasukkan pada penulisan Proyek Akhir. Jumlah BAB pada setiap proyek akhir dapat berbeda sesuai dengan topik Proyek Akhir yang diambil.

BAB I : PENDAHULUAN

Bab ini meliputi latar belakang, permasalahan, tujuan dan sistematika penulisan Proyek Akhir.

BAB II : KAJIAN PUSTAKA

Bagian ini menjelaskan tentang berbagai referensi pendukung dari Proyek Akhir yang dilakukan dari konsep dasar *Quadcopter*, kendali *state feedback*, metode kendali optimal *Linear Quadratic*

Tracking (LQT).

BAB III : METODE PENELITIAN

Metode penelitian menjelaskan berbagai metode yang digunakan untuk dapat mencapai hasil penelitian. Terdapat metode pengujian setiap bagian sistem dan metode pengujian sistem secara keseluruhan.

BAB IV : HASIL DAN PEMBAHASAN

Hasil dan pembahasan menampilkan hasil dan analisa yang didapatkan dari pengujian sistem.

BAB V : PENUTUP

Bab ini berisi kesimpulan dan saran yang dapat menjadi ide pada penelitian berikutnya

BAB II

KAJIAN PUSTAKA

2.1. Tinjauan Pustaka

Edge computing berkembang sebagai paradigma komputasi terdistribusi yang memungkinkan pemrosesan data dilakukan lebih dekat ke sumber data, sehingga dapat mengurangi *latency*, menghemat penggunaan bandwidth, dan meningkatkan efisiensi layanan dibanding pendekatan *cloud computing* terpusat. Namun, karakteristik lingkungan *edge computing* yang bersifat dinamis, heterogen, terdistribusi, dan memiliki keterbatasan sumber daya menjadikan pengelolaan sumber daya sebagai permasalahan yang kompleks [2], [11]. Kompleksitas tersebut semakin meningkat ketika sistem harus menangani *dynamic workloads*, variasi kapasitas komputasi antar node, serta kebutuhan optimasi yang melibatkan lebih dari satu objektif, seperti *latency* dan *energy consumption* secara simultan [12], [13].

Penelitian [2] menunjukkan bahwa kompleksitas *edge computing* tidak hanya dipengaruhi oleh keterbatasan sumber daya, tetapi juga oleh kebutuhan mekanisme adaptasi untuk menangani perubahan kondisi sistem secara dinamis. Studi tersebut menunjukkan bahwa pendekatan yang ada masih didominasi oleh *reactive adaptation* dan *centralized adaptation control*, sementara *distributed adaptation* dan *resource-level adaptation* masih menjadi tantangan terbuka [2]. Hal ini menunjukkan bahwa pengelolaan sumber daya pada lingkungan *edge computing* belum sepenuhnya mampu mengakomodasi kebutuhan adaptasi yang efisien pada lingkungan heterogen dan terdistribusi.

Selain itu, penelitian [12] menjelaskan bahwa *resource management* pada *mobile edge computing* merupakan permasalahan *joint optimization* yang cenderung bersifat *NP-hard*, khususnya ketika melibatkan keputusan *task scheduling*, *computation offloading*, dan *resource allocation* secara simultan. Permasalahan ini semakin kompleks ketika optimasi harus mempertimbangkan *tradeoff* antara *latency*, *energy consumption*, dan utilisasi sumber daya, sehingga

pendekatan optimasi konvensional sering kali menghasilkan solusi suboptimal [12]. Temuan ini diperkuat oleh penelitian [13], yang menunjukkan bahwa hingga saat ini belum terdapat algoritma *task scheduling* universal yang mampu bekerja optimal untuk seluruh skenario *edge cloud environments*.

Di sisi lain, aspek keberlanjutan (*sustainability*) juga menjadi dimensi penting dalam pengelolaan sumber daya pada *edge computing*. Penelitian [11] menunjukkan bahwa efisiensi energi, *load balancing*, dan *adaptive scheduling* masih menjadi tantangan utama, khususnya pada lingkungan *heterogeneous edge environments*. Studi tersebut juga menekankan bahwa *collaborative resource management* dan pendekatan *energy-aware scheduling* masih memerlukan pengembangan lebih lanjut untuk mendukung sistem *edge computing* yang lebih adaptif dan efisien [11].

Selain tantangan pada sisi komputasi dan pengelolaan sumber daya, integrasi *edge computing* dengan ekosistem *Internet of Everything (IoE)* juga meningkatkan kompleksitas sistem secara keseluruhan. Penelitian [14] dan [15] menunjukkan bahwa meningkatnya skala perangkat, keragaman layanan, dan kompleksitas distribusi sumber daya memperkuat kebutuhan mekanisme *adaptive resource management* yang mampu menangani heterogenitas sistem secara lebih efektif [14], [15].

Berdasarkan berbagai penelitian terdahulu tersebut, dapat disimpulkan bahwa kompleksitas *edge computing* dipengaruhi oleh karakteristik lingkungan yang dinamis, heterogen, dan *resource-constrained*, serta oleh belum optimalnya pendekatan *resource management* existing dalam menangani *latency-energy tradeoff*, *load balancing*, dan kebutuhan adaptasi pada sistem terdistribusi. Oleh karena itu, dibutuhkan pendekatan optimasi yang mampu mendukung pengelolaan sumber daya secara lebih adaptif dan efisien pada lingkungan *heterogeneous edge computing*, yang selanjutnya menjadi dasar pembahasan tantangan *task scheduling*, *task allocation*, dan *load balancing* pada subbab berikutnya.

Selain kompleksitas pada pengelolaan sumber daya, salah satu tantangan utama dalam *edge computing* terletak pada mekanisme *task scheduling*, *task allocation*, dan *load balancing*. Ketiga aspek tersebut berperan penting dalam menentukan efisiensi pemanfaatan sumber daya, kualitas layanan, serta performa sistem secara keseluruhan, khususnya pada lingkungan *edge computing* yang bersifat heterogen dan dinamis [10], [13], [16], [17], [18], [19]. Namun, berbagai penelitian menunjukkan bahwa hingga saat ini permasalahan *task scheduling* dan distribusi beban masih belum terselesaikan secara optimal.

Penelitian [13] menunjukkan bahwa *task scheduling* pada *edge cloud environments* merupakan permasalahan yang kompleks akibat adanya *resource heterogeneity*, *inter-task dependency*, serta perubahan kondisi sistem yang dinamis. Studi tersebut juga menunjukkan bahwa belum terdapat algoritma *task scheduling* universal yang mampu memberikan solusi optimal untuk seluruh skenario, sehingga pemilihan metode optimasi masih sangat bergantung pada karakteristik sistem yang dihadapi [13]. Hal ini menunjukkan bahwa *task scheduling* pada *edge computing* masih merupakan *open challenge*.

Selanjutnya, penelitian [16] menunjukkan bahwa meskipun berbagai pendekatan *heuristic-based workload scheduling* telah dikembangkan, banyak pendekatan existing masih memiliki keterbatasan dalam menghasilkan keseimbangan antara *global optimization* dan *local balancing*. Keterbatasan ini berkontribusi terhadap munculnya distribusi beban yang tidak merata dan berpotensi menghasilkan performa sistem yang suboptimal [16]. Temuan ini diperkuat oleh penelitian [17], yang menunjukkan bahwa strategi *task offloading* existing juga masih menghadapi tantangan dalam menangani *dynamic workloads* dan heterogenitas sumber daya secara efektif [16].

Pada aspek *task allocation*, penelitian [18] menunjukkan bahwa pendekatan berbasis kolaborasi, seperti *Collaborative Adaptive Computation Task Allocation (CACTA)*, mampu meningkatkan efisiensi distribusi *task* antar node melalui koordinasi kolaboratif. Namun demikian, studi tersebut juga menunjukkan bahwa *collaborative task allocation* tetap menghadapi kompleksitas tinggi, terutama

dalam menjaga keseimbangan distribusi beban dan efisiensi pengambilan keputusan pada lingkungan *edge collaboration* [18]. Hal ini menunjukkan bahwa permasalahan *load balancing* dan distribusi beban masih menjadi tantangan terbuka, bahkan ketika pendekatan kolaboratif telah diterapkan.

Lebih lanjut, penelitian [19] menunjukkan bahwa pendekatan *task allocation* existing umumnya masih menghadapi *tradeoff* antara efisiensi global dan adaptabilitas lokal. Pendekatan *centralized* cenderung memiliki keunggulan dalam pengambilan keputusan global, namun berpotensi menimbulkan *bottleneck*, sedangkan pendekatan *decentralized* memiliki fleksibilitas lokal yang lebih baik tetapi berisiko menghasilkan keputusan suboptimal akibat keterbatasan informasi global [19]. Oleh karena itu, studi tersebut menekankan kebutuhan pendekatan *hybrid* untuk menyeimbangkan *global efficiency* dan *local adaptability* [19].

Temuan serupa juga diperkuat oleh penelitian [10], yang menunjukkan bahwa *adaptive resource scheduling* memerlukan mekanisme optimasi yang tidak hanya mampu menangani keputusan global, tetapi juga mampu mempertimbangkan mekanisme *exploration-exploitation balancing* untuk meningkatkan kualitas keputusan alokasi sumber daya [10]. Hal ini menunjukkan bahwa tantangan *task scheduling* pada *edge computing* tidak hanya terletak pada pemilihan node tujuan, tetapi juga pada bagaimana menjaga keseimbangan distribusi beban setelah keputusan alokasi dilakukan.

Berdasarkan berbagai penelitian terdahulu tersebut, dapat disimpulkan bahwa tantangan utama pada *task scheduling*, *task allocation*, dan *load balancing* tidak hanya berkaitan dengan kompleksitas optimasi, tetapi juga dengan belum optimalnya integrasi antara pengambilan keputusan global dan mekanisme penyeimbangan beban lokal. Oleh karena itu, dibutuhkan pendekatan optimasi yang tidak hanya mendukung *global scheduling decisions*, tetapi juga mampu menangani *local load balancing* secara adaptif pada lingkungan *heterogeneous edge computing*. Kebutuhan tersebut menjadi dasar pengembangan berbagai pendekatan *hybrid* dan *metaheuristic optimization* yang dibahas pada subbab berikutnya.

Untuk mengatasi keterbatasan pendekatan konvensional pada *task scheduling* dan *resource allocation*, berbagai penelitian mulai mengembangkan pendekatan *hybrid optimization* dan *metaheuristic optimization* sebagai solusi alternatif. Pendekatan ini berkembang karena berbagai penelitian menunjukkan bahwa penggunaan satu mekanisme optimasi tunggal sering kali belum mampu menangani kompleksitas *edge computing* secara optimal, terutama ketika sistem harus mengakomodasi *latency-energy tradeoff*, heterogenitas sumber daya, serta kebutuhan keseimbangan antara keputusan global dan adaptasi lokal [10], [14], [15], [20], [21], [22].

Penelitian [10] menunjukkan bahwa pendekatan *hybrid optimization* berbasis integrasi *Bayesian Optimization* dan *Boltzmann-driven scheduling* mampu meningkatkan efisiensi *adaptive resource scheduling* melalui mekanisme *exploration-exploitation balancing*. Studi tersebut menunjukkan bahwa integrasi dua mekanisme optimasi dapat meningkatkan performa *energy-latency scheduling* dibanding pendekatan *single optimization method* [10]. Temuan ini menunjukkan bahwa pendekatan *hybrid* berpotensi menjadi solusi untuk menangani kompleksitas optimasi pada *edge computing*.

Pendekatan *hybrid* juga dikembangkan pada penelitian [11], yang mengintegrasikan *Cellular Potts Model (CPM)* sebagai mekanisme optimasi lokal dan *Particle Swarm Optimization (PSO)* sebagai optimasi global. Hasil penelitian menunjukkan bahwa integrasi *local-global optimization* mampu meningkatkan efisiensi energi, menurunkan *latency*, serta meningkatkan *convergence* dan *scalability* sistem [20]. Temuan ini menunjukkan bahwa kombinasi mekanisme optimasi global dan lokal dapat memberikan peningkatan performa dibanding penggunaan optimasi tunggal.

Selain itu, penelitian [20] mengembangkan pendekatan *two-stage hybrid evolutionary optimization* berbasis *Improved Two-Stage Multiobjective Evolutionary Algorithm (IT-MOEA)* untuk menyelesaikan *joint optimization problem* pada *task offloading* di lingkungan *heterogeneous cloud-edge systems*. Hasil penelitian menunjukkan bahwa pendekatan *two-stage optimization* mampu

meningkatkan performa dibanding algoritma *single-objective* maupun *multiobjective optimization* existing [20]. Hal ini semakin memperkuat bahwa pendekatan *hybrid optimization* menjadi salah satu arah pengembangan penting dalam *resource scheduling*.

Di sisi lain, pendekatan berbasis *reinforcement learning* juga berkembang sebagai alternatif optimasi, sebagaimana ditunjukkan pada penelitian [22], yang menggunakan *Multi-Objective Reinforcement Learning (MORL)* dan *Proximal Policy Optimization (PPO)* untuk mengoptimalkan *resource scheduling*. Studi tersebut menunjukkan bahwa pendekatan *multi-objective reinforcement learning* efektif dalam meningkatkan *latency-energy tradeoff* melalui mekanisme pengambilan keputusan adaptif berbasis *Pareto optimization* [22]. Namun demikian, pendekatan berbasis *reinforcement learning* juga memiliki kompleksitas komputasi dan kebutuhan pelatihan model yang relatif tinggi, sehingga tidak selalu menjadi solusi yang paling efisien untuk seluruh skenario *edge computing*.

Lebih lanjut, penelitian [23] menunjukkan bahwa pendekatan *hybrid metaheuristic* berbasis integrasi *Flower Pollination Algorithm (FPA)* dan *Tabu Search* juga mampu meningkatkan performa *resource allocation* melalui kombinasi *global exploration* dan *local refinement*. Namun demikian, pada pendekatan tersebut *Tabu Search* digunakan sebagai mekanisme *local refinement*, bukan sebagai mekanisme optimasi global [23]. Hal ini menunjukkan bahwa meskipun *Tabu Search* telah digunakan pada penelitian existing, peran dan integrasinya dalam arsitektur *hybrid optimization* masih terbuka untuk dikembangkan lebih lanjut.

Selain itu, penelitian [15] juga menunjukkan bahwa perkembangan *adaptive resource management* pada *Multi-Access Edge Computing* semakin menegaskan kebutuhan pendekatan optimasi yang lebih adaptif untuk menangani heterogenitas sistem dan dinamika distribusi sumber daya [15]. Secara keseluruhan, berbagai penelitian tersebut menunjukkan bahwa pendekatan *hybrid* dan *metaheuristic*

optimization telah berkembang sebagai solusi yang menjanjikan, khususnya melalui integrasi optimasi global dan mekanisme adaptasi lokal.

Namun demikian, berdasarkan berbagai penelitian terdahulu tersebut, dapat diidentifikasi bahwa pendekatan *hybrid optimization* existing umumnya masih menggunakan kombinasi seperti *CPM-PSO*, *IT-MOEA*, *MORL-PPO*, maupun *FPA-Tabu*, sementara belum terdapat pendekatan yang secara eksplisit menempatkan *Tabu Search* sebagai mekanisme optimasi global yang dikombinasikan dengan *diffusion-inspired local balancing* untuk mendukung distribusi beban adaptif pada *heterogeneous edge computing*. Kesenjangan ini menjadi dasar untuk merumuskan *research gap* dan posisi penelitian yang diusulkan pada subbab berikutnya.

Berdasarkan hasil kajian pustaka pada penelitian terdahulu, dapat diidentifikasi bahwa berbagai pendekatan telah dikembangkan untuk menangani permasalahan *resource management*, *task scheduling*, *task allocation*, dan *load balancing* pada *edge computing*. Berbagai penelitian juga menunjukkan bahwa pendekatan *hybrid optimization* mampu meningkatkan performa sistem dibanding penggunaan metode optimasi tunggal, khususnya dalam menangani *latency-energy tradeoff*, heterogenitas sumber daya, serta kompleksitas pengambilan keputusan pada lingkungan *edge computing* [10], [14], [15], [20], [21], [22], [23]. Namun demikian, masih terdapat beberapa kesenjangan penelitian yang belum sepenuhnya teratasi.

Pertama, meskipun berbagai pendekatan optimasi telah dikembangkan, hingga saat ini belum terdapat konsensus mengenai metode optimasi yang mampu memberikan performa optimal untuk seluruh skenario *task scheduling* pada *heterogeneous edge computing environments* [13], [16], [19]. Kompleksitas karakteristik sistem, variasi *workload*, dan perbedaan kapasitas antar node menyebabkan efektivitas suatu metode sangat bergantung pada kondisi sistem yang dihadapi.

Kedua, berbagai penelitian menunjukkan bahwa *load balancing* dan distribusi beban adaptif masih menjadi tantangan terbuka. Meskipun pendekatan *task allocation* dan *hybrid scheduling* telah berkembang, sebagian besar penelitian existing masih berfokus pada keputusan alokasi awal, sementara mekanisme penyeimbangan beban setelah keputusan alokasi dilakukan masih belum banyak dieksplorasi secara spesifik [16], [18], [19]. Kondisi ini menunjukkan bahwa aspek *local balancing* masih memiliki ruang pengembangan, khususnya pada lingkungan *heterogeneous edge computing*.

Ketiga, berbagai pendekatan *hybrid optimization* existing umumnya menggunakan kombinasi mekanisme seperti *CPM-PSO* [20], *IT-MOEA* [20], *MORL-PPO* [22], dan *FPA-Tabu* [23]. Meskipun pendekatan-pendekatan tersebut menunjukkan peningkatan performa, integrasi mekanisme optimasi global dan adaptasi lokal masih dikembangkan dalam bentuk yang beragam, serta belum menunjukkan satu pendekatan yang dominan sebagai solusi universal. Selain itu, pada penelitian existing yang melibatkan *Tabu Search*, algoritma tersebut umumnya digunakan sebagai mekanisme *local refinement* [23], sementara pemanfaatan *Tabu Search* sebagai mekanisme optimasi global masih relatif terbatas dalam konteks integrasi dengan mekanisme *local balancing*.

Keempat, pendekatan *collaborative resource distribution* dan mekanisme adaptasi lokal pada lingkungan heterogen juga masih relatif minim dieksplorasi [2], [11], [19]. Padahal, berbagai penelitian menunjukkan bahwa efisiensi keputusan global tidak selalu menjamin keseimbangan distribusi beban pada level lokal, sehingga diperlukan mekanisme tambahan untuk mendukung redistribusi beban yang lebih adaptif setelah proses alokasi dilakukan.

Berdasarkan kesenjangan penelitian tersebut, penelitian ini mengusulkan pendekatan *hybrid optimization* berbasis *Tabu Search* sebagai mekanisme optimasi global dan *diffusion-inspired local balancing* sebagai mekanisme redistribusi beban lokal untuk mengoptimalkan *latency* dan *energy consumption* pada lingkungan *heterogeneous edge computing*. Berbeda dengan penelitian existing yang menempatkan *Tabu Search* sebagai mekanisme *local refinement*,

penelitian ini memposisikan *Tabu Search* sebagai *global optimization mechanism* untuk menghasilkan keputusan alokasi yang efisien, yang selanjutnya disempurnakan melalui mekanisme *diffusion-inspired local balancing* untuk mendukung keseimbangan distribusi beban antar node.

Dengan demikian, posisi penelitian ini terletak pada pengembangan *alternative hybrid optimization architecture* yang mengintegrasikan keputusan optimasi global dan penyeimbangan beban lokal secara adaptif, sebagai upaya untuk mengatasi keterbatasan pendekatan existing pada *task scheduling* dan *load balancing* di lingkungan *heterogeneous edge computing*.

Tabel 2.1. Perbandingan Penelitian

No	Penelitian	Tahun	Objective	Domain Permasalahan	Global Opt	Local Opt
1.	<i>Task Allocation Algorithm and Optimization Model on Edge Collaboration</i> (X. Deng, dkk.) [18]	2020	Fokus pada <i>task scheduling</i> dalam skenario <i>edge collaboration</i> , di mana beberapa node bekerja secara terdistribusi sehingga diperlukan mekanisme alokasi tugas yang adaptif untuk menghindari overload pada node tertentu.	Fokus pada <i>task scheduling</i> dalam skenario <i>edge collaboration</i> , di mana beberapa node bekerja secara terdistribusi sehingga diperlukan mekanisme alokasi tugas yang adaptif untuk menghindari overload pada node tertentu.	Menggunakan pendekatan <i>heuristic</i> atau <i>hybrid optimization</i> untuk menentukan alokasi tugas secara global dengan mempertimbangkan kondisi sistem secara keseluruhan.	Tidak terdapat mekanisme <i>local optimization</i> yang eksplisit, sehingga penyesuaian distribusi beban setelah alokasi awal belum dilakukan secara adaptif.
2.	<i>Optimizing Low-Power Task Scheduling for Multiple Users</i>	2024	Penelitian ini berfokus pada minimisasi <i>energy consumption</i> secara	Fokus pada <i>task scheduling</i> dalam environment multi-user multi-server	Menggunakan <i>LI_PSO</i> (enhanced PSO dengan dynamic inertia	Tidak terdapat <i>local optimization</i> eksplisit. Penyeimbangan

	<i>and Servers in Mobile Edge Computing by The MUMS Framework</i> (G. Li dkk) [24]		global pada sistem MEC multi-user multi-server. Delay dijadikan constraint (batas toleransi) bukan objective utama yang dioptimasi secara langsung.	(MUMS) di Mobile Edge Computing. Mencakup communication model, offloading model, energy model, dan delay model secara terintegrasi.	weight dan probabilistic server selection) untuk menyelesaikan 0–1 integer programming problem task-server assignment.	beban dilakukan secara implisit melalui probabilistic server selection dalam PSO position update, bukan mekanisme balancing yang terpisah.
3.	<i>Task Offloading for Multi-Server Edge Computing in Industrial Internet With Joint Load Balance and Fuzzy Security</i> (W. Jin dkk.) [25]	2024	Penelitian ini mengoptimalkan <i>latency</i> dan <i>load variance</i> antar edge server pada konteks industrial Internet. Keamanan data industri juga menjadi objective tambahan melalui representasi <i>fuzzy security risk</i> . Energi tidak secara eksplisit dijadikan objective utama.	Fokus pada <i>task offloading</i> ke multiple edge servers dalam industrial Internet, dengan penekanan pada <i>load balancing</i> antar edge server dan perlindungan keamanan data sensitif industri.	Menggunakan <i>Adaptive Genetic Algorithm (AGA)</i> pada upper-layer untuk menentukan lokasi offloading task (server mana yang menerima task).	Menggunakan <i>Simulated Annealing PSO (SA-PSO)</i> pada lower-layer untuk mengoptimalkan <i>offloading ratio</i> (berapa proporsi task yang di-offload). Dua-lapis solusi: GA menentukan lokasi, SA-PSO menentukan rasio.
4.	<i>Optimizing Energy and Latency in Edge</i>	2025	Penelitian ini memfokuskan optimasi pada <i>energy</i>	Permasalahan difokuskan pada <i>resource allocation</i>	Menggunakan kombinasi <i>Bayesian</i>	Tidak terdapat <i>local optimizer</i> , sehingga perbaikan solusi

	<i>Computing through a Boltzmann-Driven Bayesian Framework for Adaptive Resource Scheduling</i> (D. Sahu, dkk.) [10]		dan <i>latency</i> karena kedua metrik tersebut merupakan indikator utama efisiensi sistem dalam <i>resource scheduling</i> , khususnya pada sistem adaptif berbasis probabilistik.	dan <i>task scheduling</i> , dengan pendekatan adaptif yang mempertimbangkan kondisi sistem secara dinamis melalui model probabilistik.	<i>optimization</i> dan <i>Boltzmann mechanism</i> untuk eksplorasi solusi global yang adaptif terhadap perubahan kondisi sistem.	hanya bergantung pada proses optimasi global tanpa <i>fine-tuning</i> pada level lokal.
5.	<i>Beyond Boundaries: A Hybrid Cellular Potts and Particle Swarm Optimization Model for Energy</i>	2025	Penelitian ini mengoptimalkan <i>energy</i> dan <i>latency</i> karena kedua metrik tersebut digunakan untuk mengevaluasi efisiensi distribusi	Penelitian ini mengoptimalkan <i>energy</i> dan <i>latency</i> karena kedua metrik tersebut digunakan untuk mengevaluasi efisiensi distribusi	Menggunakan <i>Particle Swarm Optimization (PSO)</i> sebagai mekanisme optimasi global untuk mencari solusi terbaik	Memanfaatkan interaksi lokal berbasis model <i>Cellular Potts</i> , namun tidak dirancang sebagai <i>local optimizer</i>

	<i>and Latency Optimization in Edge Computing</i> (D. Sahu, dkk.) [20]		beban dan performa sistem dalam lingkungan komputasi edge yang dinamis.	beban dan performa sistem dalam lingkungan komputasi edge yang dinamis.	dalam ruang pencarian yang kompleks.	eksplisit sehingga masih terbatas sebagai mekanisme pendukung, bukan <i>refinement strategy</i> yang terpisah.
6.	<i>Energy-Efficient and Latency-Aware Task Offloading for Industrial Cloud-Edge Systems With Heterogeneous CPUs and GPUs</i> (J. Zhai dkk.) [21]	2025	Penelitian ini mengoptimalkan <i>energy</i> dan <i>latency</i> karena sistem industri membutuhkan efisiensi energi tinggi serta respons cepat dalam menangani <i>compute-intensive tasks</i> pada lingkungan heterogen.	Permasalahan difokuskan pada <i>task offloading</i> dari perangkat industri ke <i>edge</i> atau <i>cloud</i> , dengan mempertimbangkan heterogenitas CPU dan GPU dalam sistem.	Menggunakan <i>multi-objective evolutionary algorithm</i> (IT-MOEA) untuk mengoptimalkan beberapa tujuan secara simultan dalam skala sistem yang kompleks.	Menggunakan <i>multi-objective evolutionary algorithm</i> (IT-MOEA) untuk mengoptimalkan beberapa tujuan secara simultan dalam skala sistem yang kompleks.

7.	<i>Resource Scheduling Algorithm for Edge Computing Networks Based on Multi-Objective Optimization</i> (W. Liu dkk.) [22]	2025	Penelitian ini memfokuskan optimasi pada <i>energy</i> dan <i>latency</i> sebagai metrik utama dalam mengevaluasi performa algoritma <i>resource scheduling</i> pada jaringan edge.	Permasalahan difokuskan pada <i>resource scheduling</i> , yaitu bagaimana mengalokasikan sumber daya secara optimal untuk meningkatkan efisiensi sistem secara keseluruhan.	Menggunakan pendekatan <i>multi-objective optimization</i> untuk menghasilkan solusi optimal berdasarkan beberapa kriteria performa secara simultan.	Tidak terdapat mekanisme <i>local optimization</i> , sehingga distribusi beban setelah proses optimasi global tidak disesuaikan kembali secara lokal.
8.	<i>Optimizing Resource Allocation for IoT Applications in the Edge Cloud Continuum Using Hybrid Metaheuristic Algorithms</i> (N. M.	2025	Penelitian ini mengoptimalkan <i>energy</i> , <i>latency</i> , dan <i>resource utilization</i> karena sistem IoT memerlukan efisiensi tinggi dalam penggunaan sumber daya serta respons	Penelitian ini mengoptimalkan <i>energy</i> , <i>latency</i> , dan <i>resource utilization</i> karena sistem IoT memerlukan efisiensi tinggi dalam penggunaan sumber daya serta	Penelitian ini mengoptimalkan <i>energy</i> , <i>latency</i> , dan <i>resource utilization</i> karena sistem IoT memerlukan efisiensi tinggi dalam penggunaan	Terdapat mekanisme <i>local improvement</i> berbasis Tabu Search sebagai <i>local refinement</i> , namun masih terbatas pada <i>refinement</i> sederhana dan belum mengakomodasi <i>load</i>

	Dankolo dkk.) [23]		yang cepat terhadap permintaan komputasi.	respons yang cepat terhadap permintaan komputasi.	sumber daya serta respons yang cepat terhadap permintaan komputasi.	<i>balancing</i> adaptif secara eksplisit.
9.	<i>A Hybrid GA-PSO Strategy for Computing task Offloading Towards MES Scenarios</i> (W. Li dkk.) [26]	2023	Penelitian ini mengoptimalkan <i>load balancing</i> (workload standard deviation antar <i>edge servers</i>) dan <i>offloading delay</i> secara bersamaan. Energi tidak dipertimbangkan sebagai objective.	Penelitian ini mengoptimalkan <i>load balancing</i> (workload standard deviation antar <i>edge servers</i>) dan <i>offloading delay</i> secara bersamaan. Energi tidak dipertimbangkan sebagai objective.	Menggunakan <i>Hybrid GA-PSO</i> dengan representasi partikel dua dimensi (2D-particle) untuk menentukan server tujuan setiap task dari setiap Access Point (AP).	Menggunakan <i>Hybrid GA-PSO</i> dengan representasi partikel dua dimensi (2D-particle) untuk menentukan server tujuan setiap task dari setiap Access Point (AP).
10.	Optimasi <i>Task Scheduling</i> di Heterogeneous	2026	Penelitian ini mengoptimalkan <i>energy consumption</i>	Fokus pada <i>task scheduling</i> di <i>heterogeneous edge</i>	Menggunakan Heterogeneity-aware Tabu Search	Menggunakan Diffusion Local Balancing

	Edge Computing Menggunakan <i>Hybrid Tabu-Diffusion</i> Berbasis <i>Multi-Objective</i> Energi dan <i>Latency</i>		dan <i>latency</i> secara simultan melalui satu weighted objective function. Pendekatan dual-objective ini mempertimbangkan trade-off antara konsumsi daya node dan waktu respons sistem secara bersamaan dalam satu formulasi.	<i>computing</i> dengan topologi grid 3×3 (9 nodes). Node memiliki kapasitas CPU dan memory yang bervariasi (heterogeneous), menciptakan variasi <i>load imbalance</i> yang perlu diatasi secara adaptif.	sebagai <i>Global Optimizer</i> untuk mengeksplorasi ruang solusi scheduling secara global. Tabu Search bersifat memory-based, menghindari revisit solusi buruk, dan sensitif terhadap kapasitas heterogen tiap node melalui utilization-weighted move generation.	Mechanism untuk menyeimbangkan beban antar node bertetangga secara adaptif. Diffusion bersifat <i>topology-aware</i> (grid adjacency) dan <i>power-aware</i> (mempertimbangkan konsumsi daya tiap node), berbeda dari local search konvensional.
--	---	--	---	---	--	--

Berdasarkan Tabel 2.1, dapat disimpulkan bahwa penelitian terdahulu telah banyak membahas optimasi pada *edge computing* melalui berbagai domain permasalahan, seperti *task scheduling*, *task offloading*, *resource allocation*, dan *load balancing*. Beberapa penelitian berfokus pada pengalokasian tugas dalam lingkungan kolaboratif antar node untuk menghindari *overload* pada node tertentu [18], sedangkan penelitian lain menitikberatkan pada optimasi *energy consumption* pada skenario *multi-user multi-server* dalam *mobile edge computing* [24]. Selain itu, terdapat pula penelitian yang menggabungkan aspek *latency*, *load balance*, dan keamanan pada lingkungan industri [25]. Pola ini menunjukkan bahwa permasalahan penjadwalan dan alokasi tugas pada *edge computing* tidak hanya berkaitan dengan pemilihan node eksekusi, tetapi juga berhubungan erat dengan efisiensi energi, distribusi beban, dan kualitas layanan sistem.

Dari sisi *objective*, sebagian besar penelitian menjadikan *latency* dan *energy consumption* sebagai metrik utama yang dioptimasi. Penelitian [10] secara eksplisit mengoptimalkan *energy* dan *latency* melalui pendekatan *Boltzmann-driven Bayesian framework* untuk *adaptive resource scheduling*. Penelitian [20] juga mengoptimalkan *energy* dan *latency* melalui integrasi *Cellular Potts Model* dan *Particle Swarm Optimization*, sedangkan penelitian [21] menangani optimasi *energy* dan *latency* pada sistem *industrial cloud-edge* dengan sumber daya CPU dan GPU yang heterogen. Temuan ini memperlihatkan bahwa *latency-energy trade-off* merupakan salah satu isu utama dalam pengembangan metode optimasi pada lingkungan *edge computing*.

Dari sisi domain permasalahan, penelitian terdahulu menunjukkan cakupan yang cukup beragam. Beberapa penelitian berfokus pada *task scheduling* dan *task allocation* [18], [24], sementara penelitian lain lebih menekankan *task offloading* pada sistem *edge* maupun *cloud-edge* [21], [25]. Selain itu, terdapat penelitian yang membahas *resource scheduling* dan *resource allocation* untuk mendukung aplikasi IoT dan *edge cloud continuum* [22], [23]. Perbedaan domain tersebut menunjukkan bahwa optimasi pada *edge computing* bersifat kompleks karena

melibatkan hubungan antara karakteristik *task*, kondisi sumber daya node, serta strategi distribusi beban yang digunakan.

Dari sisi metode optimasi global, berbagai penelitian menggunakan pendekatan *heuristic*, *metaheuristic*, dan *hybrid optimization*. Penelitian [24] menggunakan pengembangan *Particle Swarm Optimization* untuk menyelesaikan permasalahan *task-server assignment*, sedangkan penelitian [25] menggunakan kombinasi *Adaptive Genetic Algorithm* dan *SA-PSO* untuk menentukan lokasi *offloading* dan rasio *offloading*. Penelitian [21] menggunakan *Improved Two-Stage Multi-Objective Evolutionary Algorithm* untuk menangani optimasi pada sistem industri yang heterogen. Sementara itu, penelitian [23] menggunakan pendekatan *hybrid metaheuristic* yang melibatkan *Tabu Search*, tetapi posisi *Tabu Search* pada penelitian tersebut lebih diarahkan sebagai mekanisme *local refinement*, bukan sebagai mekanisme optimasi global.

Dari sisi *local optimization*, Tabel 2.1 menunjukkan bahwa tidak semua penelitian menyediakan mekanisme optimasi lokal yang eksplisit. Beberapa penelitian hanya mengandalkan proses optimasi global untuk menentukan alokasi tugas, sehingga penyesuaian distribusi beban setelah alokasi awal belum dilakukan secara adaptif [10], [18], [22]. Penelitian lain memang memiliki mekanisme lokal atau *local improvement*, tetapi belum secara spesifik diarahkan sebagai mekanisme *load balancing* berbasis keterhubungan antar node dan kondisi utilisasi sumber daya [20], [23]. Kondisi ini menunjukkan bahwa integrasi antara optimasi global dan penyeimbangan beban lokal masih menjadi ruang pengembangan yang relevan.

Berdasarkan perbandingan tersebut, dapat diidentifikasi bahwa pendekatan *hybrid optimization* telah banyak digunakan untuk meningkatkan kualitas solusi pada sistem *edge computing*. Namun, sebagian besar pendekatan yang ada masih berfokus pada pencarian solusi global atau menggabungkan metode optimasi tanpa pemisahan peran yang tegas antara *global optimizer* dan *local optimizer*. Selain itu, mekanisme redistribusi beban lokal setelah proses alokasi awal masih belum banyak dibahas secara eksplisit, terutama pada lingkungan *heterogeneous*

edge computing yang memiliki variasi kapasitas CPU, memori, dan daya antar node. Hal ini menjadi celah penelitian yang dapat dikembangkan lebih lanjut.

Penelitian ini memposisikan pendekatan *Tabu-Diff* sebagai alternatif *hybrid optimization* untuk mengatasi celah tersebut. Perbedaan utama penelitian ini terletak pada penggunaan *Tabu Search* sebagai mekanisme *global optimization* untuk mengeksplorasi ruang solusi *task scheduling* secara menyeluruh, serta *diffusion-inspired local balancing* sebagai mekanisme redistribusi beban lokal berbasis keterhubungan antar node. Dengan pendekatan ini, proses optimasi tidak hanya berhenti pada penentuan alokasi awal, tetapi juga dilanjutkan dengan penyesuaian beban lokal agar distribusi *task* menjadi lebih seimbang. Oleh karena itu, pendekatan yang diusulkan diharapkan mampu menurunkan *latency* dan *energy consumption* secara simultan, sekaligus meningkatkan *load balancing* pada lingkungan *heterogeneous edge computing*.

2.2. Dasar teori

2.2.1. Edge Computing

Edge computing merupakan pendekatan komputasi yang mengalihkan proses pemrosesan data dari pusat data terpusat menuju lokasi yang lebih dekat dengan sumber data atau pengguna. Pendekatan ini dikembangkan untuk mengurangi ketergantungan terhadap *cloud computing* yang sering mengalami keterbatasan dalam hal *latency* dan beban jaringan. Dengan memproses data di sisi jaringan yang lebih dekat dengan pengguna, waktu respons sistem dapat ditingkatkan sehingga lebih sesuai untuk aplikasi yang membutuhkan pemrosesan secara cepat dan berkelanjutan [27], [28].

Pertumbuhan perangkat *Internet of Things (IoT)* menjadi salah satu faktor utama yang mendorong adopsi *edge computing*. Perangkat IoT menghasilkan data dalam jumlah besar dan secara terus-menerus, sehingga pengiriman seluruh data ke pusat data tidak selalu efisien. Proses transmisi data dalam skala besar dapat meningkatkan waktu tunda serta membebani jaringan komunikasi. Oleh karena

itu, pemrosesan sebagian data dilakukan langsung pada node yang berada di dekat sumber data untuk mengurangi beban sistem secara keseluruhan [28], [29].

Dalam implementasinya, *edge computing* terdiri dari sejumlah node yang tersebar dan saling terhubung dalam suatu sistem terdistribusi. Setiap node memiliki kemampuan pemrosesan tertentu dan bertanggung jawab terhadap sebagian beban kerja. Pendekatan ini memungkinkan sistem untuk meningkatkan skalabilitas serta fleksibilitas dalam menangani berbagai jenis *task*. Namun, distribusi komputasi ke banyak node juga menimbulkan tantangan dalam hal koordinasi dan pengelolaan sumber daya [27], [29].

Keterbatasan kapasitas pada setiap node menjadi salah satu karakteristik utama dalam lingkungan *edge computing*. Sumber daya seperti CPU, memori, dan energi tidak selalu tersedia dalam jumlah yang besar, sehingga pemanfaatannya harus dilakukan secara efisien. Ketidakseimbangan dalam distribusi beban kerja dapat menyebabkan sebagian node bekerja melebihi kapasitasnya, sementara node lain tidak dimanfaatkan secara optimal. Kondisi ini berpotensi meningkatkan *latency* serta konsumsi energi dalam sistem [29].

a. *Resource Allocation dan Load Balancing*

Resource allocation dalam *edge computing* merupakan proses pengelolaan dan pendistribusian sumber daya komputasi seperti CPU, memori, dan bandwidth kepada *task* yang diproses oleh sistem. Dalam lingkungan *edge computing*, sumber daya tidak terpusat seperti pada *cloud*, melainkan tersebar pada berbagai *edge nodes* yang memiliki kapasitas terbatas. Kondisi ini menyebabkan proses alokasi sumber daya menjadi lebih kompleks karena harus mempertimbangkan keterbatasan dan distribusi sumber daya pada setiap node [28].

Selain itu, *edge computing* dirancang untuk mendukung aplikasi yang bersifat *delay-sensitive*, sehingga pengelolaan sumber daya harus mampu memastikan bahwa *task* dapat diproses dengan waktu respons yang rendah. Kedekatan lokasi *edge node* dengan pengguna memang dapat mengurangi

latency, namun tanpa mekanisme alokasi sumber daya yang tepat, keunggulan ini tidak dapat dimanfaatkan secara optimal [28].

Load balancing merupakan konsep yang berkaitan erat dengan *resource allocation*, yaitu proses mendistribusikan beban kerja secara merata ke berbagai *edge nodes*. Dalam sistem yang terdistribusi, ketidakseimbangan beban dapat menyebabkan sebagian node mengalami *overload*, sementara node lainnya tidak dimanfaatkan secara maksimal. Kondisi ini tidak hanya meningkatkan *latency*, tetapi juga menurunkan efisiensi penggunaan sumber daya secara keseluruhan.

Dalam arsitektur *edge computing*, interaksi antara *edge layer*, *thing layer*, dan *cloud layer* menunjukkan bahwa *task* dapat diproses di berbagai lokasi tergantung pada kondisi sistem. Oleh karena itu, diperlukan mekanisme penjadwalan dan alokasi sumber daya yang mampu menentukan lokasi pemrosesan *task* secara adaptif untuk mencapai performa yang optimal [29].

Tantangan utama dalam *resource allocation* dan *load balancing* terletak pada sifat sistem yang dinamis dan heterogen. Variasi jumlah *task*, perbedaan kapasitas node, serta perubahan kondisi jaringan menyebabkan kebutuhan sumber daya menjadi tidak menentu. Oleh karena itu, diperlukan pendekatan optimasi yang mampu mengelola distribusi beban kerja secara efisien guna meminimalkan *latency* dan konsumsi energi dalam sistem *edge computing*.

b. *Heterogeneous Edge Computing*

Heterogeneous edge computing mengacu pada kondisi sistem *edge computing* yang terdiri dari berbagai komponen dengan karakteristik yang berbeda, baik dari sisi perangkat keras, perangkat lunak, maupun jaringan komunikasi. Heterogenitas ini dapat ditemukan pada berbagai elemen sistem, seperti perangkat pengguna (*end devices*), *edge nodes*, hingga *edge servers*, yang masing-masing memiliki arsitektur, kapasitas komputasi, dan teknologi yang tidak seragam [28]. Perbedaan ini merupakan salah satu karakteristik utama dalam sistem *edge computing* modern yang berkembang dari integrasi berbagai perangkat *Internet of Things (IoT)*.

Keberadaan heterogenitas sumber daya menyebabkan setiap *edge node* memiliki kemampuan pemrosesan yang berbeda, baik dari sisi CPU, memori, penyimpanan, maupun konsumsi energi. Variasi ini berdampak langsung pada kemampuan node dalam menangani *task* yang diberikan. Node dengan kapasitas rendah cenderung lebih cepat mengalami kelebihan beban (*overload*), sedangkan node dengan kapasitas lebih tinggi dapat menangani beban komputasi yang lebih besar. Oleh karena itu, distribusi beban kerja yang tidak mempertimbangkan heterogenitas dapat menyebabkan ketidakseimbangan dalam sistem dan menurunkan efisiensi pemanfaatan sumber daya [29].

Heterogenitas juga memengaruhi proses *task scheduling* dan *resource allocation* dalam sistem *edge computing*. Pemilihan node untuk mengeksekusi suatu *task* harus mempertimbangkan kesesuaian antara kebutuhan komputasi *task* dan kapasitas sumber daya yang dimiliki oleh node. Apabila proses penjadwalan tidak mempertimbangkan perbedaan ini, maka sistem dapat mengalami peningkatan *latency* serta konsumsi energi yang tidak optimal. Hal ini menunjukkan bahwa heterogenitas merupakan faktor penting yang harus diperhitungkan dalam perancangan mekanisme optimasi pada sistem *edge computing* [29], [30].

Selain perbedaan kapasitas komputasi, heterogenitas juga mencakup perbedaan platform, arsitektur sistem, serta teknologi komunikasi yang digunakan antar node. Perbedaan ini dapat menimbulkan tantangan dalam interoperabilitas dan koordinasi antar komponen sistem. Dalam lingkungan yang terdistribusi, kemampuan sistem untuk mengelola komunikasi dan kolaborasi antar node menjadi sangat penting agar proses distribusi beban kerja dapat berjalan secara efektif [28]. Oleh karena itu, sistem *edge computing* memerlukan pendekatan yang adaptif untuk mengatasi kompleksitas yang diakibatkan oleh heterogenitas ini.

2.2.2. Task Schedulling

Task scheduling merupakan proses penentuan urutan (*ordering*) dan pemetaan (*mapping*) tugas ke sumber daya komputasi yang tersedia dalam suatu sistem. Secara umum, tujuan dari *task scheduling* adalah untuk mengoptimalkan kinerja sistem dengan meminimalkan waktu penyelesaian, meningkatkan utilisasi sumber daya, serta memenuhi batasan tertentu seperti waktu dan kapasitas sistem. Dalam konteks sistem komputasi heterogen, proses penjadwalan menjadi lebih kompleks karena perbedaan kemampuan antar sumber daya yang digunakan untuk mengeksekusi tugas [31].

Dalam sistem komputasi terdistribusi, *task scheduling* tidak hanya menentukan urutan eksekusi, tetapi juga lokasi pemrosesan tugas. Setiap tugas harus dipetakan ke unit komputasi yang sesuai agar dapat diproses secara efisien. Keputusan ini menjadi krusial karena kesalahan dalam penjadwalan dapat menyebabkan ketidakseimbangan beban kerja, peningkatan waktu tunggu, serta penurunan performa sistem secara keseluruhan [31].

Dalam konteks *edge computing*, konsep *task scheduling* berkembang menjadi lebih kompleks karena melibatkan berbagai lokasi pemrosesan, seperti *end devices*, *edge nodes*, dan *cloud servers*. Sistem harus menentukan apakah suatu *task* akan diproses secara lokal atau dialihkan (*offloaded*) ke node lain dengan mempertimbangkan kondisi sumber daya, *latency* jaringan, serta kebutuhan komputasi dari tugas tersebut. Hal ini menjadikan *task scheduling* sebagai bagian penting dalam pengelolaan sistem *edge computing* yang bersifat terdistribusi dan dinamis [28], [29].

Selain itu, *task scheduling* pada *edge computing* juga berkaitan erat dengan tujuan optimasi sistem, seperti meminimalkan *latency*, mengurangi konsumsi energi, dan meningkatkan efisiensi penggunaan sumber daya. Ketidaktepatan dalam proses penjadwalan dapat menyebabkan penumpukan beban pada node tertentu, sehingga meningkatkan waktu respons dan konsumsi energi. Oleh karena itu, diperlukan mekanisme penjadwalan yang adaptif dan mampu menyesuaikan dengan kondisi sistem yang berubah-ubah [28], [29].

a. Definisi Task dalam Edge Computing

Dalam sistem komputasi, *task* didefinisikan sebagai unit pekerjaan yang memerlukan sumber daya komputasi untuk diproses. Pada lingkungan *edge computing*, *task* umumnya dihasilkan oleh perangkat pengguna atau perangkat *Internet of Things (IoT)* yang secara kontinu menghasilkan data dan membutuhkan pemrosesan dengan waktu respons yang cepat. Oleh karena itu, *task* dalam *edge computing* memiliki karakteristik yang erat kaitannya dengan kebutuhan *real-time* serta keterbatasan sumber daya pada *edge nodes* [27], [28].

Berbeda dengan sistem komputasi terpusat, *task* pada *edge computing* dapat diproses pada berbagai lokasi, seperti *edge node* maupun *cloud server*, melalui mekanisme *task offloading*. Hal ini menyebabkan setiap *task* harus direpresentasikan dengan atribut tertentu agar dapat diproses dan dijadwalkan secara optimal dalam sistem yang bersifat terdistribusi [28], [29].

Dalam berbagai model *task scheduling* dan *task offloading*, suatu *task* umumnya direpresentasikan menggunakan karakteristik seperti:

- **Kebutuhan komputasi (*computational demand*)**

Menunjukkan jumlah sumber daya komputasi yang diperlukan untuk menyelesaikan *task*, biasanya terkait dengan penggunaan CPU

- **Ukuran data (*data size*)**

Menentukan besarnya data yang perlu ditransmisikan saat *task* dikirim ke node lain

- **Waktu kedatangan (*arrival time*)**

Menunjukkan waktu masuknya *task* ke dalam sistem

- **Batas waktu (*deadline*)**

Digunakan pada *task* yang bersifat *time-sensitive* atau *real-time*

- **Prioritas *task***

Menentukan tingkat kepentingan suatu *task* dalam proses penjadwalan

Karakteristik tersebut menjadi parameter penting dalam menentukan bagaimana suatu *task* diproses, baik dari sisi penentuan lokasi eksekusi maupun alokasi sumber daya yang digunakan. Dengan mempertimbangkan

atribut-atribut ini, sistem *edge computing* dapat mengoptimalkan distribusi beban kerja [32], [31].

b. Bentuk Task dalam *Edge Computing*

Dalam sistem *edge computing*, *task* dapat memiliki berbagai bentuk tergantung pada jenis aplikasi dan sumber data yang dihasilkan. Variasi bentuk *task* ini dipengaruhi oleh karakteristik aplikasi, kebutuhan komputasi, serta jenis data yang diproses. Secara umum, *task* dalam *edge computing* berkaitan dengan pemrosesan data yang bersifat *real-time* dan membutuhkan respons cepat, sehingga sering ditemukan pada aplikasi berbasis *Internet of Things (IoT)* dan sistem cerdas [1], [2].

Berikut adalah beberapa bentuk *task* yang umum ditemukan dalam lingkungan *edge computing*:

1. Pemrosesan Data Sensor

Task berupa pemrosesan data sensor merupakan bentuk paling umum dalam *edge computing*. Data dihasilkan secara kontinu oleh perangkat seperti sensor suhu, kelembapan, atau perangkat monitoring lingkungan. Pemrosesan dilakukan untuk:

- filtering data
- agregasi data
- deteksi anomali

Jenis *task* ini biasanya memiliki ukuran data kecil namun bersifat kontinu dan membutuhkan respons cepat.

2. Analisis Citra dan Video

Task ini melibatkan pemrosesan data visual seperti gambar atau video, yang umumnya memiliki ukuran data besar dan membutuhkan komputasi tinggi. Contohnya:

- pengenalan wajah (*face recognition*)
- deteksi objek (*object detection*)
- sistem pengawasan (*smart surveillance*)

Jenis *task* ini termasuk dalam kategori *compute-intensive task* dan sering diproses di *edge node* untuk mengurangi *latency*.

3. Pemrosesan Data Real-Time

Task yang bersifat *real-time* membutuhkan waktu respons yang sangat cepat, biasanya dalam skala milidetik. Contohnya:

- kendaraan otonom (*autonomous vehicles*)
- sistem kesehatan berbasis sensor
- aplikasi *augmented reality*

Keterlambatan dalam pemrosesan dapat menyebabkan penurunan kualitas layanan atau bahkan risiko kegagalan sistem.

4. Machine Learning Inference

Pada *edge computing*, *task* juga dapat berupa proses inferensi dari model *machine learning*, di mana model yang telah dilatih sebelumnya digunakan untuk melakukan prediksi atau klasifikasi. Contohnya:

- klasifikasi gambar
- deteksi aktivitas pengguna
- prediksi kondisi sistem

Task ini membutuhkan keseimbangan antara kebutuhan komputasi dan efisiensi energi, karena sering dijalankan pada perangkat dengan sumber daya terbatas.

c. Pembentukan Task

Pembentukan *task* dalam sistem *edge computing* berkaitan dengan proses bagaimana data yang dihasilkan oleh perangkat atau aplikasi diubah menjadi unit pekerjaan yang siap diproses oleh sistem komputasi. Berbeda dengan sistem komputasi tradisional, *task* dalam *edge computing* umumnya terbentuk secara dinamis dan kontinu, mengikuti aliran data yang dihasilkan oleh perangkat *Internet of Things (IoT)* dan aplikasi pengguna [1], [2].

Task dalam *edge computing* dapat berasal dari berbagai sumber, antara lain:

- **Perangkat IoT**

Perangkat seperti sensor, kamera, dan perangkat wearable menghasilkan data secara terus-menerus. Data ini kemudian dikonversi menjadi *task* untuk diproses, seperti deteksi kondisi lingkungan atau analisis aktivitas.

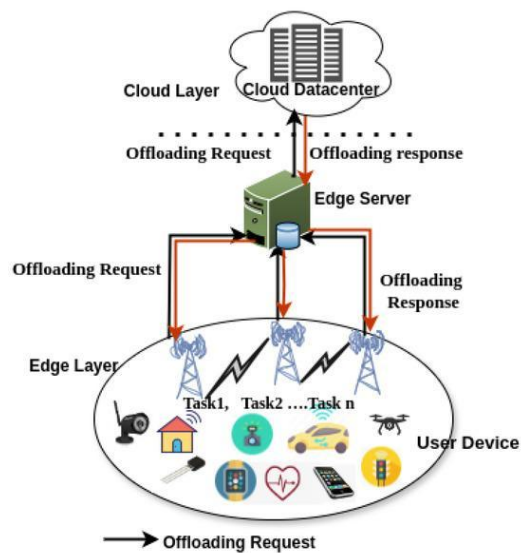
- **Aplikasi pengguna**

Aplikasi berbasis *mobile* atau *web* menghasilkan *task* berdasarkan interaksi pengguna, seperti permintaan layanan berbasis lokasi, pemrosesan multimedia, atau analisis data pengguna.

- **Sistem monitoring dan control**

Sistem seperti *smart city*, *industrial IoT*, dan sistem keamanan menghasilkan *task* untuk keperluan pengawasan, pengendalian, dan pengambilan keputusan secara otomatis.

Secara umum, proses pembentukan *task* dalam *edge computing* dapat dijelaskan sebagaimana ditunjukkan pada Gambar 2.1. Proses ini dimulai dari tahap *data generation*, di mana perangkat atau aplikasi menghasilkan data mentah (*raw data*) secara kontinu. Data yang dihasilkan tersebut kemudian masuk ke tahap *task creation*, yaitu proses pengemasan data menjadi suatu *task* yang dilengkapi dengan parameter tertentu, seperti ukuran data dan kebutuhan komputasi.



Gambar 2.II.1. Alur pembentukan dan pemrosesan task dalam sistem *edge computing* [33]

Selanjutnya, *task* yang telah terbentuk dikirim ke sistem *edge computing* melalui tahap *task transmission*, baik menuju *edge node* terdekat maupun ke pusat kontrol. Setelah itu, sistem melakukan proses *task decision*, yaitu menentukan apakah *task* akan diproses secara lokal atau dialihkan (*offloaded*) ke node lain berdasarkan kondisi sumber daya dan kebutuhan sistem.

Pada tahap berikutnya, *task* dieksekusi pada node yang telah dipilih dalam proses *task execution*. Hasil dari proses ini kemudian dikirim kembali ke pengguna atau sistem melalui tahap *result return*. Alur ini menunjukkan bahwa pembentukan dan pemrosesan *task* dalam *edge computing* melibatkan beberapa tahapan yang saling terintegrasi, mulai dari pembangkitan data hingga pengembalian hasil pemrosesan.

2.2.3. Metode Optimasi

Metode optimasi merupakan pendekatan yang digunakan untuk menentukan solusi terbaik dari suatu permasalahan dengan mempertimbangkan tujuan tertentu serta batasan (*constraints*) yang ada. Secara umum, optimasi dapat diartikan sebagai upaya untuk memperoleh penggunaan sumber daya yang paling efektif guna mencapai tujuan yang diinginkan [34]. Dalam sistem komputasi, optimasi banyak digunakan untuk meningkatkan kinerja sistem dengan cara meminimalkan parameter seperti waktu eksekusi, konsumsi energi, dan biaya penggunaan sumber daya.

Dalam konteks *edge computing*, permasalahan optimasi menjadi lebih kompleks karena sistem bersifat terdistribusi, memiliki keterbatasan sumber daya, serta menghadapi beban kerja yang dinamis. Penelitian terbaru menunjukkan bahwa optimasi dalam *edge computing* umumnya melibatkan beberapa parameter secara simultan, seperti *latency*, konsumsi energi, dan biaya komputasi, sehingga diperlukan pendekatan optimasi yang mampu menangani banyak tujuan sekaligus. Pendekatan ini menjadi penting karena metode optimasi dengan satu tujuan (*single-objective*) tidak lagi mampu memenuhi kebutuhan sistem secara menyeluruh dalam lingkungan komputasi modern [35].

Secara umum, optimasi merupakan proses pencarian solusi terbaik dari sekumpulan solusi yang mungkin dalam suatu ruang solusi (*search space*). Setiap solusi dievaluasi menggunakan fungsi tertentu yang disebut *objective function*, yang merepresentasikan tujuan sistem. Dalam banyak kasus, optimasi diformulasikan sebagai proses minimisasi atau maksimisasi suatu fungsi matematis terhadap sejumlah variabel keputusan [34].

Dalam permasalahan *task scheduling*, optimasi bertujuan untuk menentukan pemetaan *task* ke sumber daya komputasi yang tersedia sehingga diperoleh kinerja sistem yang optimal. Proses ini melibatkan eksplorasi berbagai kemungkinan solusi untuk menemukan konfigurasi terbaik, terutama pada sistem dengan sumber daya yang heterogen.

a. *Multi-Objective*

Multi-objective optimization merupakan pendekatan optimasi yang melibatkan lebih dari satu fungsi tujuan yang harus dipenuhi secara bersamaan. Berbeda dengan optimasi satu tujuan (*single-objective optimization*), pendekatan ini mempertimbangkan beberapa kriteria yang sering kali saling bertentangan, sehingga solusi yang diperoleh tidak hanya berfokus pada satu aspek saja [34], [36].

Dalam permasalahan *multi-objective optimization*, setiap fungsi tujuan direpresentasikan dalam bentuk matematis dan dioptimalkan secara simultan. Secara umum, permasalahan ini dapat dinyatakan sebagai berikut:

$$\min f_k(x), k = 1, 2, \dots, n \quad (2.II.1)$$

di mana:

- $f_k(x)$ merupakan fungsi tujuan ke- k
- x merupakan variabel keputusan dalam ruang solusi

Karena adanya konflik antar tujuan, tidak selalu terdapat satu solusi yang secara optimal memenuhi semua fungsi tujuan sekaligus. Oleh karena itu, solusi yang dihasilkan dalam *multi-objective optimization* biasanya berupa sekumpulan solusi yang dikenal sebagai *Pareto optimal solutions*. Suatu

solusi dikatakan *Pareto optimal* apabila tidak terdapat solusi lain yang dapat meningkatkan satu tujuan tanpa menurunkan tujuan lainnya [36].

Konsep *Pareto optimality* menunjukkan adanya hubungan *trade-off* antar tujuan. Dalam praktiknya, pengguna atau sistem akan memilih solusi yang paling sesuai dari himpunan solusi *Pareto* berdasarkan kebutuhan tertentu. Pendekatan ini banyak digunakan dalam berbagai bidang, termasuk sistem komputasi terdistribusi dan *edge computing*, yang memiliki banyak parameter kinerja yang harus dioptimalkan secara bersamaan.

Dalam konteks *edge computing*, *multi-objective optimization* sering digunakan untuk mengoptimalkan beberapa parameter utama, seperti *latency*, konsumsi energi, dan biaya komputasi. Penelitian terbaru menunjukkan bahwa pendekatan optimasi dengan satu tujuan tidak cukup untuk memenuhi kebutuhan sistem modern, sehingga diperlukan metode yang mampu menangani beberapa tujuan sekaligus untuk mencapai performa yang lebih optimal [35]. Dengan menggunakan *multi-objective optimization*, sistem dapat menemukan keseimbangan antara berbagai parameter yang saling memengaruhi, sehingga kualitas layanan dapat ditingkatkan secara keseluruhan.

b. *Objective Function*

Objective function merupakan fungsi matematis yang digunakan untuk mengevaluasi kualitas suatu solusi dalam proses optimasi. Fungsi ini merepresentasikan tujuan yang ingin dicapai oleh sistem, sehingga setiap solusi yang dihasilkan akan dinilai berdasarkan nilai dari fungsi tersebut. Dalam proses optimasi, solusi terbaik adalah solusi yang memberikan nilai optimal, baik dalam bentuk minimisasi maupun maksimisasi [37].

Dalam permasalahan *multi-objective optimization*, *objective function* tidak hanya terdiri dari satu fungsi, tetapi beberapa fungsi tujuan yang dioptimalkan secara bersamaan. Secara umum, fungsi objektif dapat dinyatakan seperti pada persamaan 2.1.

Setiap fungsi tujuan merepresentasikan parameter kinerja yang berbeda, sehingga solusi yang diperoleh harus mempertimbangkan keseimbangan

antara berbagai tujuan tersebut. Dalam praktiknya, untuk mempermudah proses optimasi, beberapa fungsi tujuan sering dikombinasikan menjadi satu fungsi tunggal menggunakan pendekatan penjumlahan berbobot (*weighted sum*) [36].

$$f(x) = w_1f_1(x) + w_2f_2(x) + \dots + w_nf_n(x) \quad (2.II.2)$$

di mana:

- w_i merupakan bobot yang menunjukkan tingkat kepentingan masing-masing tujuan
- $\sum w_i = 1$

Pendekatan ini memungkinkan sistem untuk mengontrol prioritas antara berbagai tujuan yang dioptimalkan.

Dalam konteks *edge computing*, *objective function* umumnya dirancang untuk mengoptimalkan beberapa parameter utama, seperti *latency*, konsumsi energi, dan biaya komputasi. Penelitian menunjukkan bahwa kombinasi parameter tersebut digunakan untuk menentukan strategi *task scheduling* atau *task offloading* yang optimal dalam sistem yang memiliki keterbatasan sumber daya [35].

2.2.4. Local Optimization

Local optimization merupakan pendekatan optimasi yang berfokus pada perbaikan solusi dalam ruang solusi yang terbatas di sekitar solusi awal (*current solution*). Berbeda dengan *global optimization* yang mengeksplorasi seluruh ruang solusi, *local optimization* bekerja dengan melakukan penyesuaian kecil secara bertahap untuk meningkatkan kualitas solusi yang telah ada [36].

Dalam konteks sistem komputasi terdistribusi, *local optimization* sering digunakan untuk meningkatkan efisiensi sistem melalui penyesuaian beban kerja antar node. Pendekatan ini memungkinkan sistem untuk merespons perubahan kondisi secara cepat tanpa perlu melakukan perhitungan ulang secara global, sehingga lebih efisien dalam lingkungan yang dinamis [28].

Pada sistem *edge computing*, *local optimization* berperan dalam melakukan penyesuaian distribusi *task* antar *edge nodes* untuk menghindari

ketidakseimbangan beban. Ketika suatu node mengalami kelebihan beban (*overload*), sebagian *task* dapat dipindahkan ke node lain yang memiliki kapasitas lebih tersedia. Proses ini dilakukan secara lokal dengan mempertimbangkan kondisi node di sekitarnya.

a. Diffusion-Inspired Local Balancing

Difusi merupakan proses penyebaran suatu kuantitas dari area dengan konsentrasi tinggi menuju area dengan konsentrasi rendah hingga mencapai kondisi yang lebih seimbang. Dalam teori difusi, perpindahan kuantitas tersebut dapat dijelaskan melalui konsep gradien, yaitu perbedaan nilai antar lokasi. Secara matematis, proses difusi banyak dijelaskan melalui persamaan difusi yang menunjukkan perubahan suatu konsentrasi terhadap waktu akibat penyebaran pada ruang tertentu .

Secara umum, prinsip difusi dapat dinyatakan melalui hukum Fick sebagai berikut:

$$J = -D\nabla u \quad (2.II.3)$$

dengan J sebagai *flux* atau laju aliran, D sebagai koefisien difusi, u sebagai konsentrasi, dan ∇u sebagai gradien konsentrasi. Tanda negatif menunjukkan bahwa aliran bergerak dari nilai konsentrasi tinggi menuju nilai konsentrasi rendah. Prinsip ini menjadi dasar bahwa perbedaan nilai antar lokasi dapat mendorong proses penyebaran hingga sistem mendekati kondisi seimbang [38].

Dalam sistem komputasi terdistribusi, konsep difusi dapat diadaptasi untuk permasalahan *load balancing*. Beban komputasi pada setiap node dipandang sebagai kuantitas yang dapat “menyebar” dari node dengan beban tinggi menuju node dengan beban lebih rendah. Cybenko menjelaskan bahwa skema difusi dapat digunakan untuk *dynamic load balancing* pada jaringan multiprosesor terdistribusi, di mana proses penyeimbangan dilakukan melalui pertukaran beban antar node yang saling terhubung [39]. Pendekatan ini juga diperkuat oleh kajian *task allocation* dan *load balancing* yang menjelaskan

bahwa negosiasi beban antar node bertetangga dalam sistem terdistribusi dapat direpresentasikan sebagai proses difusi.

Pada bentuk diskrit, sistem dapat dimodelkan sebagai graf $G = (V, E)$, dengan V sebagai himpunan node dan E sebagai hubungan antar node. Jika $u_i^{(t)}$ menyatakan beban atau utilisasi node i pada iterasi ke- t , maka proses difusi sederhana dapat dinyatakan sebagai:

$$u_i^{(t+1)} = u_i^{(t)} + \gamma \sum_{j \in N(i)} (u_j^{(t)} - u_i^{(t)}). \quad (2.11.4)$$

dengan $N(i)$ sebagai himpunan node tetangga dari node i , dan γ sebagai koefisien difusi yang mengatur besar perubahan beban pada setiap iterasi. Persamaan ini menunjukkan bahwa nilai beban pada suatu node akan berubah berdasarkan selisih beban antara node tersebut dan node tetangganya. Jika node memiliki beban lebih tinggi dibandingkan tetangganya, maka sebagian beban cenderung dialihkan ke node dengan beban lebih rendah.

Dalam penelitian ini, konsep difusi tidak digunakan sebagai penyelesaian persamaan diferensial penuh, tetapi diadaptasi sebagai mekanisme *diffusion-inspired local balancing*. Beban node direpresentasikan melalui utilisasi CPU, sedangkan hubungan antar node direpresentasikan melalui *adjacency* atau keterhubungan antar *edge nodes*. Setelah *Tabu Search* menghasilkan solusi alokasi awal secara global, mekanisme *diffusion-inspired local balancing* digunakan untuk melakukan penyesuaian lokal dengan mempertimbangkan perbedaan utilisasi antar node yang saling terhubung.

2.2.5. Global Optimization

Global optimization merupakan pendekatan optimasi yang bertujuan untuk menemukan solusi terbaik (*global optimum*) dari seluruh ruang solusi (*search space*) yang tersedia. Berbeda dengan *local optimization* yang hanya berfokus pada perbaikan solusi di sekitar titik tertentu, *global optimization* melakukan eksplorasi secara lebih luas untuk menghindari terjebak pada solusi lokal (*local optimum*) [37].

Dalam banyak permasalahan optimasi, terutama yang bersifat kompleks dan non-linear, ruang solusi dapat memiliki banyak titik optimum lokal. Kondisi ini menyebabkan metode optimasi sederhana berisiko berhenti pada solusi yang tidak optimal secara global. Oleh karena itu, *global optimization* dirancang untuk mengeksplorasi berbagai kemungkinan solusi secara lebih menyeluruh sehingga peluang menemukan solusi terbaik menjadi lebih besar. Secara umum, *global optimization* memiliki beberapa karakteristik utama:

- **Eksplorasi ruang solusi secara luas**
Proses pencarian tidak hanya terbatas pada area tertentu, tetapi mencakup berbagai kemungkinan solusi dalam ruang solusi
- **Kemampuan menghindari *local optimum***
Menggunakan strategi tertentu untuk keluar dari solusi lokal dan melanjutkan pencarian
- **Iteratif dan berbasis pencarian (*search-based*)**
Solusi diperbaiki secara bertahap melalui proses iterasi
- **Cocok untuk permasalahan kompleks dan non-linear**
Terutama pada sistem yang memiliki banyak variabel dan keterbatasan

Dalam konteks *task scheduling*, *global optimization* digunakan untuk menentukan alokasi *task* ke seluruh *edge nodes* dengan mempertimbangkan kondisi sistem secara menyeluruh. Proses ini mencakup pencarian berbagai kemungkinan kombinasi alokasi *task* untuk menemukan konfigurasi yang memberikan kinerja terbaik, seperti meminimalkan *latency* dan konsumsi energi secara simultan.

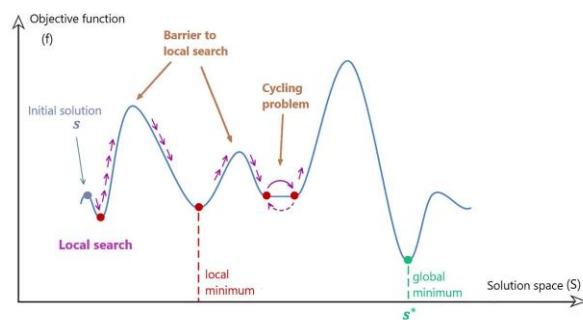
Karena jumlah kemungkinan kombinasi alokasi *task* dapat sangat besar, permasalahan ini termasuk dalam kategori kompleks (*combinatorial optimization*). Oleh karena itu, metode *global optimization* yang digunakan biasanya berbasis heuristik atau metaheuristik, seperti *Genetic Algorithm*, *Particle Swarm Optimization*, dan *Tabu Search*, yang mampu menemukan solusi mendekati optimal dalam waktu yang wajar [32].

Dalam banyak sistem optimasi modern, *global optimization* dan *local optimization* sering digunakan secara bersamaan. *Global optimization* digunakan untuk mencari solusi awal yang baik pada skala sistem secara keseluruhan, sedangkan *local optimization* digunakan untuk memperbaiki solusi tersebut secara lebih detail pada tingkat lokal.

a. Tabu Search

Tabu Search merupakan metode *metaheuristic* yang diperkenalkan oleh Fred Glover untuk menyelesaikan permasalahan optimasi kombinatorial yang memiliki ruang solusi besar dan kompleks. Metode ini bekerja dengan mengeksplorasi solusi-solusi tetangga (*neighborhood solutions*) dari solusi saat ini, kemudian memilih solusi terbaik berdasarkan nilai fungsi objektif. Berbeda dengan metode pencarian lokal konvensional, *Tabu Search* dilengkapi dengan mekanisme memori untuk menghindari pengulangan solusi, sehingga dapat mengurangi risiko terjebak pada *local optimum* dan meningkatkan kemampuan eksplorasi ruang solusi [40].

Dalam permasalahan optimasi, ruang solusi umumnya memiliki beberapa titik *local minimum* dan satu *global minimum*. Metode pencarian lokal (*local search*) cenderung hanya melakukan eksplorasi di sekitar solusi saat ini, sehingga berpotensi terjebak pada solusi lokal yang belum tentu optimal secara global. Selain itu, proses pencarian juga dapat mengalami *cycling problem*, yaitu kondisi di mana algoritma berulang pada solusi yang sama tanpa menemukan perbaikan yang signifikan. Ilustrasi kondisi tersebut ditunjukkan pada Gambar 2.2.

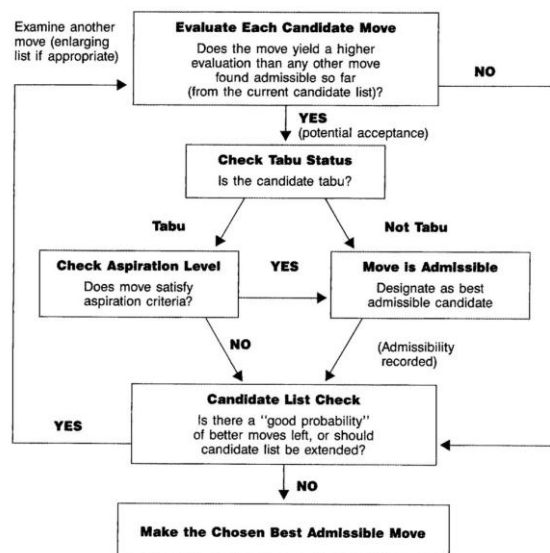


Gambar 2.II.2. Ilustrasi permasalahan local optimum dalam ruang solusi optimasi [41]

Berdasarkan ilustrasi pada Gambar 2.2, diperlukan metode optimasi yang mampu keluar dari jebakan *local optimum* dan melakukan eksplorasi ruang solusi secara lebih luas. *Tabu Search* menawarkan solusi melalui penggunaan struktur memori (*tabu list*) dan mekanisme *aspiration criterion* yang memungkinkan algoritma untuk menghindari pengulangan solusi sekaligus tetap menerima solusi tabu yang memberikan peningkatan kualitas signifikan.

Pada *Tabu Search*, proses pencarian dilakukan dengan membangkitkan solusi-solusi tetangga dari solusi saat ini. Setiap solusi kandidat dievaluasi menggunakan fungsi objektif, kemudian dipilih solusi terbaik sebagai solusi berikutnya dengan mempertimbangkan aturan *tabu list*. *Tabu list* menyimpan informasi mengenai solusi atau perpindahan (*move*) yang tidak diperbolehkan untuk sementara waktu, sehingga algoritma tidak kembali ke solusi yang telah dikunjungi sebelumnya. Selain itu, *aspiration criterion* memungkinkan solusi tabu tetap dipilih apabila solusi tersebut lebih baik dibandingkan solusi terbaik yang telah ditemukan [40], [42].

Alur kerja *Tabu Search* secara keseluruhan dapat dilihat pada Gambar 2.3.



Gambar 2.II.3 Alur proses pemilihan solusi dalam algoritma *Tabu Search* [43]

Gambar 2.3 menunjukkan bahwa proses *Tabu Search* dimulai dengan evaluasi solusi kandidat (*candidate move*), dilanjutkan dengan pengecekan apakah solusi tersebut termasuk dalam *tabu list*. Jika solusi tidak tabu, maka solusi dapat diterima sebagai kandidat terbaik. Jika solusi termasuk dalam *tabu list*, maka dilakukan pengecekan terhadap *aspiration criterion*. Apabila memenuhi kriteria tersebut, solusi tetap dapat diterima. Setelah solusi terbaik dipilih, *tabu list* diperbarui dan proses pencarian dilanjutkan secara iteratif hingga kondisi berhenti terpenuhi.

Untuk mempermudah pemahaman terhadap mekanisme *Tabu Search*, komponen utama yang digunakan dalam algoritma ini dirangkum pada Tabel 2.1.

Tabel 2.II.1 Komponen utama pada *Tabu Search*

Komponen	Penjelasan
Solusi awal	Konfigurasi awal alokasi <i>task</i>
Neighborhood	Himpunan solusi hasil perubahan kecil dari solusi saat ini
Move	Operasi perubahan solusi, misalnya pemindahan <i>task</i> antar node

Tabu List	Struktur memori untuk menyimpan solusi atau <i>move</i> yang dilarang sementara
Aspiration Criterion	Aturan yang memungkinkan solusi tabu tetap diterima jika lebih baik
Objective Function	Fungsi untuk mengevaluasi kualitas solusi
Stopping Condition	Kriteria penghentian proses optimasi

Dalam konteks *task scheduling*, solusi dapat direpresentasikan sebagai pemetaan *task* ke node komputasi. Sebagai contoh, apabila terdapat sejumlah *task* dan beberapa *edge nodes*, maka satu solusi menunjukkan node mana yang digunakan untuk mengeksekusi setiap *task*. Perubahan solusi atau *move* dapat berupa pemindahan satu *task* dari satu node ke node lain. Dengan pendekatan ini, *Tabu Search* mampu mengeksplorasi berbagai kemungkinan alokasi *task* untuk mencari solusi yang memiliki nilai *latency* dan konsumsi energi yang lebih optimal.

Secara matematis, proses optimasi dalam *Tabu Search* tetap berlandaskan pada fungsi objektif, yaitu pencarian solusi x^* yang meminimalkan nilai $f(x)$ dalam ruang solusi S . Proses pencarian dilakukan dengan memilih solusi terbaik dari himpunan solusi tetangga yang tidak termasuk dalam *tabu list*, sehingga eksplorasi ruang solusi dapat dilakukan secara efektif tanpa terjebak pada solusi lokal.

Pseudocode sederhana dari *Tabu Search* dapat dituliskan sebagai berikut:

```

Initialize solution x
Set x_best = x
Initialize tabu list T

While stopping condition not met:
    Generate neighborhood N(x)
    Select best  $x' \in N(x)$  not in T
    If  $x'$  better than x_best:
        x_best =  $x'$ 
    Update tabu list T
     $x = x'$ 

Return x_best

```

2.2.6. Latency

Latency merupakan salah satu parameter utama dalam sistem komputasi yang menunjukkan waktu yang dibutuhkan sejak suatu *task* dikirimkan hingga hasil pemrosesan diterima kembali oleh pengguna. Dalam sistem komputasi modern, khususnya pada *edge computing*, *latency* menjadi faktor yang sangat penting karena banyak aplikasi yang bersifat *time-sensitive* dan membutuhkan respons sistem dalam waktu yang cepat. Oleh karena itu, optimasi *latency* menjadi salah satu tujuan utama dalam perancangan sistem komputasi terdistribusi [32].

Secara umum, *latency* terdiri dari dua komponen utama, yaitu *latency* jaringan dan *latency* pemrosesan. *Latency* jaringan (*network latency*) merepresentasikan waktu yang dibutuhkan untuk mentransmisikan data dari perangkat pengguna ke node komputasi dan sebaliknya. Sementara itu, *latency* pemrosesan (*processing latency*) merupakan waktu yang dibutuhkan oleh sistem untuk memproses *task* yang diberikan. Kedua komponen ini secara bersama-sama menentukan total waktu respons sistem terhadap suatu permintaan.

Dalam konteks *edge computing*, keberadaan node komputasi yang lebih dekat dengan pengguna bertujuan untuk mengurangi *latency* jaringan dibandingkan dengan pendekatan komputasi berbasis *cloud*. Namun demikian, *latency* pemrosesan tetap menjadi faktor penting yang dipengaruhi oleh kapasitas sumber daya dan distribusi beban kerja pada setiap node. Oleh karena itu, pengelolaan *task scheduling* dan *resource allocation* yang tepat diperlukan untuk menjaga *latency* tetap rendah.

Pada penelitian ini, fokus analisis *latency* dibatasi pada sisi sistem komputasi, khususnya yang berkaitan dengan proses pemrosesan *task* dan kondisi beban pada *edge nodes*. Pendekatan ini dilakukan untuk menyesuaikan model sistem yang digunakan, sehingga perhitungan *latency* lebih merepresentasikan pengaruh alokasi *task* terhadap performa sistem. Dengan demikian, optimasi yang dilakukan bertujuan untuk meminimalkan *latency* melalui distribusi *task* yang efisien serta penyeimbangan beban antar node.

2.2.7. Energi

Energi merupakan salah satu parameter penting dalam sistem komputasi yang menunjukkan jumlah daya yang digunakan selama proses pemrosesan *task*. Dalam sistem *edge computing*, efisiensi energi menjadi aspek yang sangat krusial karena perangkat *edge* umumnya memiliki keterbatasan sumber daya serta kebutuhan untuk menjaga konsumsi daya agar tetap optimal. Selain itu, konsumsi energi yang tinggi juga berdampak pada peningkatan biaya operasional serta isu keberlanjutan sistem komputasi [32].

Secara umum, energi dalam sistem komputasi dinyatakan dalam satuan Joule (J) atau kilowatt-hour (kWh), yang merepresentasikan jumlah energi yang digunakan dalam suatu periode waktu tertentu. Konsumsi energi dalam sistem komputasi dipengaruhi oleh berbagai faktor, di antaranya adalah tingkat utilisasi CPU, penggunaan memori, durasi pemrosesan, serta karakteristik perangkat keras yang digunakan. Penelitian menunjukkan bahwa konsumsi energi memiliki hubungan yang erat dengan utilisasi CPU, di mana peningkatan utilisasi akan menyebabkan peningkatan konsumsi daya secara signifikan [44].

Dalam konteks *edge computing*, konsumsi energi tidak hanya dipengaruhi oleh aktivitas pemrosesan, tetapi juga oleh distribusi *task* antar node. Node dengan beban kerja yang tinggi akan mengalami peningkatan konsumsi energi akibat intensitas penggunaan sumber daya yang lebih besar. Sebaliknya, distribusi beban yang tidak merata dapat menyebabkan inefisiensi energi, karena beberapa node bekerja secara berlebihan sementara node lain tidak dimanfaatkan secara optimal. Oleh karena itu, strategi *task scheduling* dan *load balancing* memiliki peran penting dalam mengoptimalkan penggunaan energi pada sistem terdistribusi [45].

Dalam penelitian ini, energi digunakan sebagai salah satu parameter dalam fungsi objektif yang dioptimasi bersama dengan *latency*. Pendekatan ini bertujuan untuk mencapai keseimbangan antara performa sistem dan efisiensi energi, mengingat kedua parameter tersebut sering kali memiliki hubungan *trade-off*. Dengan demikian, optimasi yang dilakukan tidak hanya berfokus pada

peningkatan kecepatan pemrosesan, tetapi juga pada pengurangan konsumsi energi secara keseluruhan melalui alokasi *task* yang lebih efisien.

2.2.8. Virtual Private Server (VPS)

Virtual Private Server (VPS) merupakan layanan komputasi berbasis virtualisasi yang memungkinkan satu server fisik dibagi menjadi beberapa server virtual yang berjalan secara independen. Setiap VPS memiliki sistem operasi, sumber daya komputasi, serta konfigurasi jaringan sendiri, sehingga dapat berfungsi layaknya server fisik yang terpisah. Teknologi ini dibangun melalui mekanisme virtualisasi, yang memungkinkan pembagian sumber daya seperti CPU, memori, dan penyimpanan secara fleksibel pada satu infrastruktur fisik [46].

Virtualisasi menjadi komponen utama dalam perkembangan *cloud computing*, di mana sumber daya komputasi dapat disediakan secara *on-demand* dan elastis sesuai kebutuhan pengguna. Dengan adanya virtualisasi, sistem komputasi mampu meningkatkan efisiensi penggunaan sumber daya serta memberikan fleksibilitas dalam pengelolaan infrastruktur. Selain itu, virtualisasi juga memungkinkan isolasi antar lingkungan komputasi sehingga setiap VPS dapat berjalan secara independen tanpa saling memengaruhi [47].

Dalam konteks sistem komputasi terdistribusi, VPS banyak digunakan sebagai platform implementasi karena mampu merepresentasikan node komputasi tanpa memerlukan perangkat keras fisik dalam jumlah besar. Setiap VPS dapat dikonfigurasi sebagai *edge node* maupun sebagai *central node*, sehingga memungkinkan simulasi arsitektur sistem yang mendekati kondisi nyata. Hal ini memberikan fleksibilitas dalam pengujian metode optimasi, khususnya pada lingkungan *edge computing* yang memiliki karakteristik terdistribusi.

Selain itu, penggunaan VPS memungkinkan penerapan heterogenitas sumber daya, di mana setiap node dapat memiliki kapasitas CPU, memori, dan penyimpanan yang berbeda. Kondisi ini sesuai dengan karakteristik sistem *edge computing* di dunia nyata, yang terdiri dari perangkat dengan kemampuan komputasi yang tidak seragam. Dengan demikian, VPS menjadi platform yang

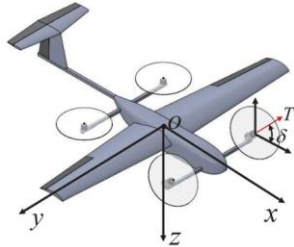
tepat untuk mengevaluasi metode *task scheduling* dan optimasi sumber daya dalam sistem yang heterogen.

Dalam penelitian ini, VPS digunakan sebagai lingkungan implementasi sistem yang terdiri dari satu *central node* dan beberapa *edge nodes*. *Central node* berfungsi sebagai pengendali sistem yang melakukan proses pengambilan keputusan, sedangkan *edge nodes* bertugas mengeksekusi *task* berdasarkan hasil optimasi. Penggunaan VPS memungkinkan pengujian metode yang diusulkan dalam kondisi yang mendekati sistem nyata, sekaligus memberikan fleksibilitas dalam pengaturan parameter dan skenario pengujian.

Dasar teori berisi semua teori yang melatarbelakangi Proyek Akhir. Bagian ini juga berisi pedoman untuk mengerjakan penelitian lebih lanjut. Bentuk dasar teori dapat berupa uraian kualitatif, model atau persamaan matematis.

Semua referensi yang digunakan atau dikutip harus tercantum dalam daftar pustakan. Pengutipan daftar pustakan dilakukan dengan menggunakan gaya IEEE yaitu menggunakan nomor publikasi dalam kurung siku [1]. Penjelasan tentang tata cara pengutipan referensi IEEE dapat dilihat pada link berikut <https://lib.ugm.ac.id/en/ieee-referencing-style/>.

Semua gambar dan tabel harus jelas/tidak kabur/buram/tulisan didalam gambar atau tabel harus dapat terbaca oleh mata normal. Gambar dan tabel diletakkan pada tengah halaman (*center alignment*). Penunjukkan gambar pada paragraph harus berupa angka gambar sesuai dengan gambar atau tabel yang dituju. Contoh gambar dapat dilihat pada Gambar II.4, sedangkan contoh tabel dapat dilihat pada Tabel II.2. Pemanggilan gambar dan tabel dilakukan secara otomatis dengan menggunakan *Cross reference* pada MS. Word.



Gambar II.4 Model aerodinamik *Fixed Wing*

Tabel II.2 Parameter sistem *fixed wing*

<i>Property</i>	Simbol	Nilai
<i>Wing span</i>	B	1.27 m
<i>Wing surface area</i>	S	0.3097 m ²
<i>Maincord</i>	C	0.25 m
Massa	M	1.959 kg
	J _x	0.07151 kg.m ²
	J _y	0.08636 kg.m ²
Inertia	J _v	0.15364 kg.m ²
	J _{xz}	0.014 kg.m ²

2.2.1. Linearisasi

Berdasarkan sifat linearitasnya, sistem dapat terbagi menjadi dua bagian yaitu sistem linear dan sistem nonlinear. Sistem nonlinear dapat ditulis sebagai $\dot{x} = f(x, u)$ dengan x merupakan variable keadaan dan u adalah input kendali. Model linear memiliki bentuk seperti pada Persamaan (II.5) :

$$\begin{aligned}\dot{\mathbf{x}} &= \mathbf{Ax} + \mathbf{Bu} \\ y &= \mathbf{Cx} + \mathbf{Du}\end{aligned}\tag{II.5}$$

2.2.2. Kalman Filter

Kalman filter merupakan salah satu filter yang berfungsi untuk memperbaiki nilai ketidakpastian dalam pengukuran

BAB III

METODE PENELITIAN

3.1. Perangkat Penelitian

Penelitian ini menggunakan perangkat keras dan perangkat lunak untuk mendukung proses pemodelan, implementasi, serta pengujian sistem optimasi pada lingkungan *edge computing*. Perangkat yang digunakan terdiri dari komputer lokal sebagai media pengembangan dan simulasi, serta *Virtual Private Server (VPS)* sebagai lingkungan implementasi sistem yang merepresentasikan *edge nodes* secara terdistribusi.

3.1.1. Perangkat Keras

Perangkat keras yang digunakan dalam penelitian ini berupa laptop yang digunakan untuk proses pengembangan program, simulasi, dan analisis hasil. Spesifikasi perangkat keras yang digunakan ditunjukkan pada Tabel 3.1:

Tabel 3.1 Spesifikasi Laptop

CPU	Intel Core i3-1115G4
RAM	8 GB
Penyimpanan	SSD 1 TB

Perangkat keras ini digunakan untuk menjalankan simulasi program, proses optimasi, serta pengolahan data hasil pengujian.

3.1.2. Perangkat Lunak

Perangkat lunak yang digunakan dalam penelitian ini meliputi sistem operasi, bahasa pemrograman, *library*, serta platform komputasi yang mendukung implementasi sistem *edge computing*.

a. Lingkungan Pengembangan

Lingkungan pengembangan digunakan untuk pengembangan kode program, eksperimen simulasi, serta analisis hasil optimasi dengan spesifikasi sebagai berikut:

Sistem Operasi	Windows 11
Editor	Visual Studio Code 1.117.0
Bahasa Pemrograman	Python

b. Library Simulasi Program

Library yang digunakan pada tahap simulasi program berbasis *notebook* antara lain:

- *NumPy* digunakan untuk komputasi numerik, operasi array, pembentukan assignment task, serta pengaturan random seed agar eksperimen dapat direproduksi.
- *Pandas* digunakan untuk membaca, membersihkan, dan memanipulasi dataset mesin serta dataset task.
- *random* digunakan untuk mendukung proses acak pada sampling dan eksperimen optimasi.
- *os* dan *pathlib* digunakan untuk manajemen direktori, path dataset, dan path output eksperimen.
- *sys* digunakan untuk memastikan environment Python dan dependency project dapat diakses dengan benar pada notebook.
- *time* digunakan untuk mengukur runtime algoritma dan menghitung time-to-target.
- *Matplotlib* digunakan untuk visualisasi hasil eksperimen, seperti grafik konvergensi objective dan cumulative distribution function (CDF).
- *IPython.display* digunakan untuk menampilkan tabel hasil eksperimen secara langsung di notebook.

Library tersebut digunakan untuk melakukan simulasi dan evaluasi performa algoritma optimasi dalam lingkungan terkontrol.

c. Lingkungan Implementasi VPS

Penelitian ini menggunakan *Virtual Private Server* (VPS) sebagai representasi lingkungan *edge computing* yang bersifat terdistribusi. Lingkungan VPS digunakan untuk membangun interaksi antar node dalam sistem, dengan satu *central node* sebagai koordinator dan sembilan *edge*

nodes sebagai unit eksekusi *task*. Perbedaan spesifikasi antar *edge nodes* digunakan untuk merepresentasikan kondisi *heterogeneous resources* pada lingkungan *edge computing*.

Spesifikasi perangkat VPS yang digunakan dalam penelitian ditunjukkan pada Tabel 3.2.

Tabel 3.III.1. Spesifikasi perangkat VPS

Komponen	Jumlah	CPU	RAM	Storage	OS
<i>Central Node</i> Koordinator sistem dan proses optimasi	1 VPS	8 vCPU	16 GB	50 GB	Ubuntu 24.04
<i>Low tier edge node</i> Eksekusi <i>task</i> kapasitas rendah	3 VPS	2 vCPU	2 GB	30 GB	Ubuntu 24.04
<i>Mid tier edge node</i> Eksekusi <i>task</i> kapasitas menengah	3 VPS	4 vCPU	4 GB	30 GB	Ubuntu 24.04
<i>High tier edge node</i> Eksekusi <i>task</i> kapasitas tinggi	3 VPS	8 vCPU	8 GB	30 GB	Ubuntu 24.04

Berdasarkan Tabel 3.2, lingkungan VPS pada penelitian ini terdiri dari tiga tingkatan *edge nodes*, yaitu *low tier*, *mid tier*, dan *high tier*. Pembagian tingkatan tersebut didasarkan pada perbedaan kapasitas CPU dan RAM yang dimiliki oleh setiap node. *Central node* tidak digunakan sebagai node utama untuk mengeksekusi *task*, melainkan sebagai pusat koordinasi sistem, pengelolaan informasi node, dan proses optimasi alokasi *task*. Sementara itu, *edge nodes* berfungsi sebagai unit pemrosesan yang menjalankan *task* sesuai hasil alokasi dari metode optimasi.

d. Library Implementasi Sistem (VPS)

Library yang digunakan dalam implementasi sistem pada VPS meliputi:

- *FastAPI* (v0.135.3) digunakan untuk membangun REST API pada *central node* dan *edge nodes*, seperti *endpoint* pengiriman *task*,

Commented [AJ1]: Tambahkan alasan pemilihan

pengecekan status *task*, *health check*, dan penerimaan hasil eksekusi. FastAPI dipilih karena mendukung pengembangan API secara cepat, memiliki dukungan *asynchronous request handling*, serta terintegrasi langsung dengan Pydantic untuk validasi data. Dibandingkan *framework* web Python lain yang lebih umum, FastAPI lebih sesuai untuk kebutuhan sistem ini karena ringan dan berfokus pada pengembangan API.

- *Uvicorn* (v0.44.0) digunakan sebagai *ASGI server* untuk menjalankan aplikasi FastAPI pada setiap *node*. *Uvicorn* dipilih karena FastAPI berjalan pada standar *ASGI*, sehingga membutuhkan *server* yang mampu menangani proses *asynchronous*. Selain itu, *Uvicorn* ringan dan mudah dijalankan pada VPS dengan konfigurasi sederhana, sehingga sesuai untuk implementasi pada beberapa *edge nodes*.
- *HTTPX* (v0.28.1) digunakan untuk komunikasi HTTP secara *asynchronous*, terutama pada proses komunikasi antar *node* dan pengiriman *heartbeat* atau status dari *edge node*. *HTTPX* dipilih karena mendukung pola komunikasi *async*, sehingga lebih sesuai ketika sistem perlu menangani beberapa permintaan HTTP tanpa menunggu proses secara berurutan. Hal ini membantu meningkatkan efisiensi komunikasi pada sistem terdistribusi.
- *Requests* (v2.33.1) digunakan untuk komunikasi HTTP secara sinkron pada proses eksperimen, seperti pengiriman *task* dari *central node* ke *edge node* dan pengambilan hasil eksekusi *task*. *Requests* dipilih karena memiliki sintaks yang sederhana, stabil, dan mudah digunakan untuk proses eksperimen yang berjalan secara terkontrol. Pada bagian sistem yang tidak membutuhkan komunikasi *async*, *Requests* lebih praktis digunakan karena implementasinya lebih sederhana dan mudah dibaca.
- *Psutil* (v7.2.2) digunakan untuk memantau penggunaan CPU dan memori pada *edge node*, sehingga sistem dapat mencatat kondisi *resource node* selama proses eksekusi. *Psutil* dipilih karena menyediakan akses langsung ke informasi sumber daya sistem melalui Python tanpa perlu memanggil perintah sistem operasi secara manual. *Library* ini juga

mendukung pemantauan lintas platform, sehingga memudahkan pencatatan metrik CPU dan memori pada lingkungan VPS.

- *Pydantic* (v2.12.5) digunakan untuk validasi dan pembentukan struktur data, seperti model *task*, hasil *task*, status node, dan metrik sistem. *Pydantic* dipilih karena mampu memastikan data yang dikirim antar node memiliki format dan tipe yang sesuai. Hal ini penting pada sistem terdistribusi, karena kesalahan struktur data pada komunikasi antar node dapat menyebabkan proses eksekusi *task* atau pengumpulan hasil menjadi tidak konsisten.
- *Starlette* (v1.0.0) digunakan sebagai dependensi utama FastAPI yang mendukung mekanisme *routing*, *middleware*, dan pemrosesan *request-response*. *Starlette* dipilih karena menjadi fondasi dari FastAPI dan mendukung pengembangan aplikasi ASGI yang ringan. Dengan adanya *Starlette*, FastAPI dapat menangani proses komunikasi HTTP secara efisien pada setiap node.
- *NumPy* digunakan pada sisi *central node* untuk proses komputasi numerik dalam pembangkitan *task*, perhitungan sumber daya, assignment *task*, dan algoritma optimasi. *NumPy* dipilih karena menyediakan operasi numerik berbasis *array* yang efisien dan umum digunakan dalam pengolahan data serta komputasi ilmiah. Dalam penelitian ini, *NumPy* membantu mempercepat perhitungan vektor kebutuhan CPU, memori, utilisasi node, dan evaluasi solusi optimasi.

Pemilihan *library* tersebut disesuaikan dengan kebutuhan implementasi sistem *edge computing* berbasis VPS. FastAPI, Uvicorn, HTTPX, dan Requests digunakan untuk mendukung komunikasi antar node. *Pydantic* dan *Starlette* digunakan untuk membangun struktur API yang valid dan konsisten. *Psutil* digunakan untuk pemantauan sumber daya, sedangkan *NumPy* digunakan untuk komputasi numerik pada proses pembentukan *task*, perhitungan model, dan optimasi. Dengan kombinasi tersebut, sistem dapat menjalankan proses pengiriman *task*, eksekusi pada *edge nodes*,

pengumpulan hasil, serta evaluasi performa secara terstruktur pada lingkungan VPS.

e. Alat Pengukuran Performa Sistem

Selain library Python, penelitian ini juga menggunakan alat pengukuran performa pada tingkat sistem operasi untuk memperoleh metrik eksekusi task secara lebih nyata, meliputi:

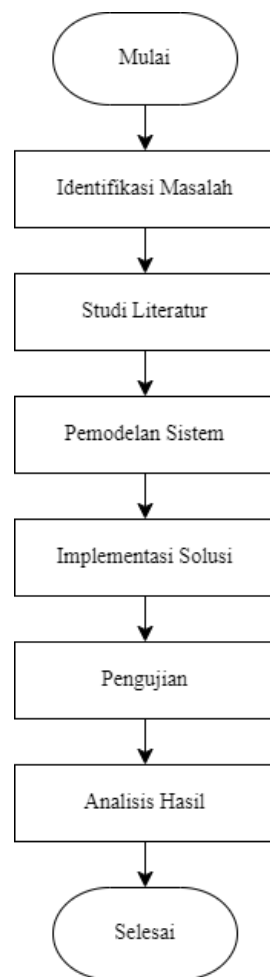
- *perf* digunakan untuk mengukur metrik eksekusi workload pada edge node, khususnya task-clock dan cpu-clock. Task-clock digunakan untuk mengetahui durasi aktif task selama proses eksekusi, sedangkan cpu-clock digunakan untuk melihat waktu penggunaan CPU oleh task. *perf* dipilih karena merupakan alat pengukuran performa bawaan ekosistem Linux yang mampu membaca performance counter pada tingkat sistem operasi. Dibandingkan hanya menggunakan waktu eksekusi dari program, *perf* memberikan metrik yang lebih rinci terkait aktivitas CPU selama task dijalankan.
- *Linux process monitoring* digunakan untuk mendukung pengamatan penggunaan sumber daya node, seperti CPU dan memori, selama proses eksekusi task berlangsung. Pemantauan proses ini dipilih karena dapat memberikan informasi kondisi aktual sistem saat task berjalan pada VPS. Dengan pemantauan proses, sistem tidak hanya mencatat hasil akhir eksekusi, tetapi juga dapat mengamati penggunaan sumber daya yang terjadi selama proses berlangsung.

Penggunaan alat pengukuran ini bertujuan agar evaluasi pada lingkungan VPS tidak hanya bergantung pada model simulasi, tetapi juga mempertimbangkan hasil eksekusi nyata dari task yang dijalankan pada edge node.

3.2. Tahapan proyek akhir

Tahapan penelitian merupakan rangkaian proses yang dilakukan secara sistematis untuk mencapai tujuan penelitian. Proses ini dimulai dari identifikasi masalah hingga analisis hasil pengujian. Setiap tahapan dirancang untuk

memastikan bahwa metode yang diusulkan dapat dikembangkan, diimplementasikan, dan dievaluasi secara terstruktur. Alur tahapan penelitian dalam studi ini ditunjukkan pada **Gambar 3. 1**.



Gambar 3. 1 Diagram Alir Tahapan Penelitian

Tahapan penelitian merupakan rangkaian proses yang dilakukan secara sistematis untuk mencapai:

- a. Identifikasi Masalah

Tahapan ini bertujuan untuk memahami permasalahan utama dalam sistem *edge computing*, khususnya terkait peningkatan *latency* dan konsumsi energi akibat keterbatasan sumber daya dan heterogenitas *edge nodes*. Analisis dilakukan terhadap karakteristik sistem yang melibatkan distribusi beban komputasi dan kebutuhan pemrosesan *task* secara *real-time*. Permasalahan utama yang diidentifikasi adalah bagaimana mengalokasikan tugas secara optimal agar performa sistem tetap efisien. Selain itu, hubungan antara peningkatan beban komputasi dan konsumsi energi juga menjadi fokus dalam tahap ini. Hasil dari tahapan ini menjadi dasar dalam perumusan solusi yang diusulkan.

b. Studi Literatur

Tahapan studi literatur dilakukan dengan mengkaji berbagai penelitian terdahulu yang berkaitan dengan *edge computing*, *task scheduling*, dan metode optimasi. Literatur yang digunakan meliputi jurnal ilmiah, prosiding konferensi, serta referensi akademik lainnya yang relevan. Analisis difokuskan pada metode optimasi yang digunakan, seperti *multi-objective optimization*, serta pendekatan *hybrid optimization*. Selain itu, dilakukan identifikasi kelebihan dan keterbatasan dari setiap metode yang telah dikembangkan. Hasil studi literatur digunakan untuk menentukan *research gap* dan merancang pendekatan optimasi yang diusulkan dalam penelitian ini.

c. Pemodelan Sistem

Tahapan pemodelan sistem bertujuan untuk merepresentasikan lingkungan *edge computing* dalam bentuk model yang dapat dianalisis dan diimplementasikan. Sistem dimodelkan sebagai arsitektur terdistribusi yang terdiri dari satu *central node* dan beberapa *edge nodes* dengan spesifikasi sumber daya yang berbeda. Model ini mencerminkan kondisi *heterogeneous edge computing* di mana setiap node memiliki kapasitas CPU dan memori. Selain itu, dilakukan pemodelan distribusi *task* dari sumber ke node pemrosesan serta interaksi antar node dalam sistem. Pemodelan ini menjadi dasar dalam pengembangan algoritma optimasi dan simulasi sistem.

d. Implementasi Solusi

Tahapan ini meliputi perancangan dan penerapan metode optimasi yang diusulkan dalam penelitian. Proses dimulai dengan perancangan *objective function* yang menggabungkan *latency* dan konsumsi energi sebagai parameter utama yang akan diminimalkan. Selanjutnya, dilakukan definisi *search space* yang merepresentasikan kemungkinan alokasi tugas ke setiap *edge node*. Metode optimasi yang digunakan terdiri dari *Tabu Search* sebagai mekanisme optimasi global dan *diffusion-inspired local balancing* sebagai mekanisme penyeimbangan beban lokal. Implementasi dilakukan dalam bentuk simulasi program dan sistem terdistribusi pada lingkungan VPS. Tahapan ini menghasilkan solusi alokasi tugas yang akan dievaluasi pada tahap pengujian.

e. Pengujian

Tahapan pengujian dilakukan untuk mengevaluasi performa metode optimasi yang diusulkan. Pengujian dilakukan dalam dua pendekatan, yaitu simulasi program dan implementasi pada lingkungan VPS. Pada simulasi program, pengujian dilakukan menggunakan dataset dan skenario yang telah ditentukan untuk membandingkan performa algoritma berdasarkan nilai *objective function*, *latency*, dan konsumsi energi. Selain itu, digunakan analisis seperti *cumulative distribution function (CDF)* untuk mengevaluasi distribusi performa sistem. Pada implementasi VPS, pengujian dilakukan dengan menjalankan sistem secara nyata menggunakan *task generator* dan memonitor performa setiap node. Hasil dari kedua pendekatan ini digunakan untuk menilai efektivitas metode optimasi.

f. Analisis Hasil

Tahapan analisis hasil bertujuan untuk menginterpretasikan hasil pengujian yang telah dilakukan. Analisis dilakukan dengan membandingkan performa metode yang diusulkan terhadap metode lain berdasarkan metrik yang telah ditentukan. Selain itu, dilakukan evaluasi terhadap pola distribusi beban, efisiensi energi, serta stabilitas sistem. Hasil analisis digunakan untuk menentukan keunggulan dan keterbatasan metode yang dikembangkan.

3.3. Pemodelan Sistem

3.3.1. Diagram Arsitektur Sistem

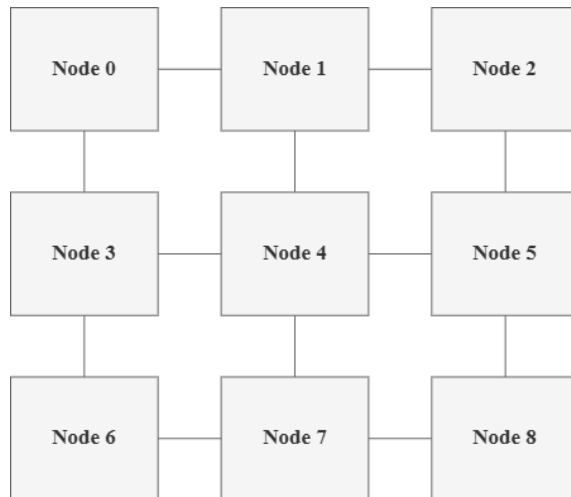
Menjelaskan arsitektur umum: 1 central node dan 9 edge nodes. Diagram ini berbasis implementasi VPS/VM, yaitu central sebagai koordinator dan edge nodes sebagai eksekutor task.

Pemodelan sistem pada penelitian ini menggunakan arsitektur *edge computing* yang terdiri dari sembilan *edge nodes* heterogen. Setiap node merepresentasikan unit komputasi yang dapat menerima dan memproses *task* berdasarkan kapasitas sumber daya yang dimiliki. Heterogenitas pada model ini ditunjukkan melalui perbedaan kapasitas CPU, memori, estimasi daya, dan karakteristik *delay* antar node. Dengan adanya perbedaan karakteristik tersebut, proses *task scheduling* tidak dapat dilakukan hanya dengan membagi *task* secara merata, tetapi perlu mempertimbangkan kondisi masing-masing node agar alokasi *task* lebih efisien.

Topologi dasar yang digunakan pada penelitian ini ditunjukkan pada Gambar 3. 2. Pada gambar tersebut, sembilan *edge nodes* disusun dalam bentuk grid 3×3. Bentuk grid digunakan untuk menggambarkan hubungan antar node dalam sistem *edge computing* yang terdistribusi. Setiap node merupakan tempat untuk eksekusi *task*, sedangkan hubungan antar node digunakan sebagai dasar untuk merepresentasikan keterhubungan dalam proses penjadwalan dan penyesuaian beban lokal.

Commented [AJ2]: Diambil persamaan anatr simulasi program dan VPS. Sekilas ditunjukkna perbedda, dari spesifikasi

Commented [AJ3R2]: Topologi



Gambar 3. 2 Topologi dasar sembilan edge nodes heterogen berbentuk grid 3×3

Model arsitektur pada Gambar 3. 2 digunakan sebagai acuan umum dalam dua lingkungan pengujian, yaitu simulasi program dan lingkungan VPS. Pada simulasi program, node pada grid direpresentasikan sebagai model parameter, seperti kapasitas CPU, kapasitas memori, estimasi daya, dan *network delay*. *Task* tidak dieksekusi secara nyata, tetapi dialokasikan ke node dan dievaluasi menggunakan model perhitungan energi serta *latency*. Dengan pendekatan ini, simulasi program digunakan untuk membandingkan beberapa kombinasi metode optimasi dalam kondisi yang terkontrol.

Pada lingkungan VPS, arsitektur yang sama direalisasikan dalam bentuk sistem terdistribusi berbasis server virtual. Sistem VPS terdiri dari satu *central node* dan sembilan *edge nodes*. *Central node* berperan sebagai koordinator yang membentuk *task batch*, menjalankan proses optimasi, mengirim *task* ke node tujuan, serta mengumpulkan hasil eksekusi. Sementara itu, *edge nodes* berperan sebagai unit eksekusi yang menerima *task*, menjalankan *workload*, dan mengembalikan hasil eksekusi kepada *central node*. Dengan demikian, lingkungan VPS digunakan untuk memvalidasi kandidat metode optimasi pada kondisi eksekusi yang lebih mendekati sistem nyata.

Perbedaan utama antara simulasi program dan VPS terletak pada cara *node* dan *task* direpresentasikan. Pada simulasi program, *node* dan *task* dimodelkan secara komputasional berdasarkan dataset dan parameter sistem. Sementara itu, pada VPS, *task* benar-benar dikirim melalui komunikasi antar *node* dan dieksekusi pada *edge nodes*. Oleh karena itu, metrik pada simulasi program berfokus pada nilai objektif, energi model, *latency* model, dan waktu komputasi algoritma, sedangkan metrik pada VPS mencakup *real latency*, *execution time*, *task-clock*, penggunaan memori terobservasi, dan estimasi energi.

Selain itu, kedua lingkungan pengujian menggunakan baseline yang sesuai dengan tujuan masing-masing fase. Pada simulasi program, baseline digunakan sebagai pembanding awal untuk melihat kualitas hasil optimasi pada model komputasi. Pada lingkungan VPS, baseline dibentuk melalui pendekatan *random baseline average*, yaitu beberapa kali alokasi acak yang metriknya dirata-ratakan untuk memperoleh pembanding yang lebih stabil. Dengan pemisahan ini, simulasi program berfungsi sebagai tahap perbandingan metode, sedangkan VPS berfungsi sebagai tahap validasi kandidat metode terpilih.

Secara keseluruhan, arsitektur pada penelitian ini dirancang untuk mendukung evaluasi *task scheduling* pada sistem *edge computing* heterogen. Simulasi program digunakan untuk menguji banyak kombinasi metode optimasi secara terkontrol, sedangkan lingkungan VPS digunakan untuk melihat perilaku metode ketika *task* dijalankan pada *node* virtual secara nyata. Pembagian dua lingkungan ini membantu penelitian dalam membedakan hasil berbasis model dan hasil berbasis eksekusi nyata.

3.3.2. Pemodelan Sistem pada Simulasi Program

Menjelaskan model yang digunakan pada [Comparision.ipynb](#), termasuk [grid 3x3](#), [heterogeneous node](#), [dataset](#), [parameter CPU/memori](#), [baseline Round-Robin](#), [objective energy-latency](#), dan [interaksi algoritma optimasi](#).

Pemodelan sistem pada simulasi program dilakukan untuk merepresentasikan proses *task scheduling* pada lingkungan *heterogeneous edge computing* secara terkontrol. Pada tahap ini, sistem dimodelkan menggunakan program simulasi berbasis *Python notebook* untuk mengevaluasi kinerja beberapa kombinasi metode optimasi sebelum diterapkan pada lingkungan VPS. Simulasi ini digunakan sebagai fase awal penelitian untuk membandingkan performa algoritma berdasarkan nilai fungsi objektif, waktu pencapaian target, dan waktu eksekusi algoritma.

Berbeda dengan implementasi pada VPS, pemodelan pada simulasi program tidak menjalankan *task* secara langsung pada server fisik atau virtual. Simulasi program merepresentasikan sistem melalui parameter *task*, kapasitas node, topologi jaringan, serta model perhitungan *latency* dan energi. Dengan pendekatan ini, setiap metode optimasi dapat diuji pada kondisi yang sama sehingga hasil perbandingan antar metode menjadi lebih terukur dan konsisten.

Fase simulasi ini berperan sebagai *simulation-based comparison*, yaitu tahap untuk menentukan kandidat metode optimasi yang memiliki performa terbaik atau paling kompetitif. Hasil dari tahap ini kemudian digunakan sebagai dasar untuk memilih metode yang akan divalidasi lebih lanjut pada lingkungan VPS. Dengan demikian, simulasi program tidak hanya berfungsi sebagai model awal sistem, tetapi juga sebagai mekanisme seleksi metode sebelum pengujian pada sistem yang lebih mendekati kondisi nyata.

a. Topologi dan Karakteristik Node Simulasi

Pada simulasi program, sistem *edge computing* dimodelkan dalam bentuk topologi grid berukuran 3×3 yang terdiri dari sembilan *edge nodes*. Setiap node merepresentasikan unit komputasi yang dapat menerima dan memproses *task* berdasarkan kapasitas sumber daya yang dimiliki. Topologi grid digunakan untuk merepresentasikan hubungan antar node dalam sistem terdistribusi, terutama pada mekanisme *local balancing* yang mempertimbangkan keterhubungan antar node dalam proses redistribusi

beban. Pada model ini, hubungan antar node dibentuk menggunakan pola ketetanggaan empat arah, yaitu atas, bawah, kiri, dan kanan.

Setiap *edge node* pada simulasi memiliki karakteristik sumber daya yang berbeda sehingga sistem yang dimodelkan bersifat heterogen. Heterogenitas tersebut direpresentasikan melalui variasi kapasitas CPU dan memori pada setiap node. Perbedaan kapasitas ini menyebabkan kemampuan setiap node dalam menangani *task* menjadi tidak sama. Oleh karena itu, proses *task scheduling* harus mempertimbangkan kapasitas sumber daya setiap node agar distribusi beban tidak menumpuk pada node tertentu.

Dalam simulasi, sembilan *edge nodes* dikelompokkan ke dalam tiga tingkatan sumber daya, yaitu *low tier*, *mid tier*, dan *high tier*. *Low tier* merepresentasikan node dengan kapasitas sumber daya rendah, *mid tier* merepresentasikan node dengan kapasitas menengah, sedangkan *high tier* merepresentasikan node dengan kapasitas lebih tinggi. Pembagian ini digunakan untuk meniru kondisi *heterogeneous edge computing*, yaitu kondisi ketika node dalam sistem memiliki kemampuan komputasi dan karakteristik sumber daya yang tidak seragam.

Selain kapasitas CPU dan memori, model simulasi juga mempertimbangkan estimasi daya dan *network delay* pada setiap tingkatan node. Parameter daya digunakan untuk merepresentasikan perbedaan konsumsi energi antar node, sedangkan *network delay* digunakan untuk merepresentasikan perbedaan waktu komunikasi antar tingkatan node. Dengan demikian, model simulasi tidak hanya menggambarkan perbedaan kemampuan komputasi, tetapi juga mempertimbangkan aspek energi dan *latency* yang menjadi tujuan utama optimasi.

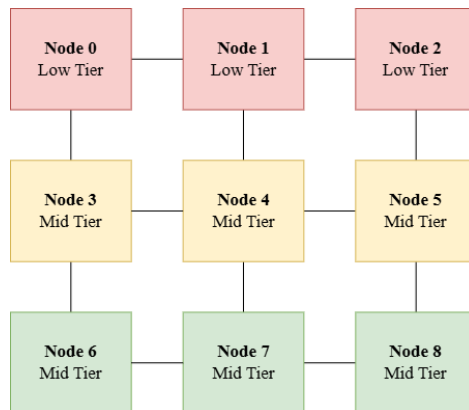
Pembagian tingkatan node pada simulasi program ditunjukkan pada Tabel 3.2.

Tabel 3.III.2. Pembagian tingkatan node pada simulasi program

Tingkatan Node	Node Simulasi	Karakteristik Umum
<i>Low tier</i>	Node 0, Node 1, Node 2	Kapasitas CPU dan memori rendah, konsumsi daya lebih kecil,

		dan network delay relatif lebih tinggi
<i>Mid tier</i>	Node 3, Node 4, Node 5	Kapasitas CPU dan memori menengah, konsumsi daya sedang, dan network delay menengah
<i>High tier</i>	Node 6, Node 7, Node 8	Kapasitas CPU dan memori tinggi, konsumsi daya lebih besar, dan network delay relatif lebih rendah

Topologi grid 3×3 pada simulasi dapat divisualisasikan sebagaimana ditunjukkan pada Gambar 3.X. Gambar ini menunjukkan bahwa setiap node terhubung dengan node tetangga terdekatnya, sehingga mekanisme *local balancing* dapat melakukan evaluasi perpindahan *task* berdasarkan hubungan antar node.



Gambar 3.III.1. Topologi edge nodes berbentuk grid 3x3

Model topologi dan karakteristik node ini menjadi dasar dalam proses evaluasi *task scheduling* pada simulasi program. Setiap solusi alokasi *task* akan dievaluasi berdasarkan kondisi node yang dipilih, termasuk kapasitas CPU, kapasitas memori, estimasi konsumsi energi, dan *latency*. Dengan model ini, algoritma optimasi dapat dibandingkan pada lingkungan yang

sama dan terkontrol sebelum kandidat terbaik divalidasi pada lingkungan VPS.

b. Dataset dan Pembentukan *Task* Simulasi

Dataset pada simulasi program digunakan sebagai dasar untuk membentuk karakteristik *edge nodes* dan karakteristik *task* pada lingkungan simulasi. Penelitian ini menggunakan dataset berbasis *Google Cluster Trace* sebagai sumber *benchmark workload* dan *resource trace*. Dataset tersebut menyediakan informasi mengenai mesin, *job*, *task*, kebutuhan sumber daya, serta penggunaan sumber daya dalam lingkungan komputasi berskala besar. Dengan demikian, dataset ini dapat digunakan untuk membentuk variasi kapasitas node dan kebutuhan *task* yang relevan untuk pengujian metode *task scheduling*.

Penggunaan dataset *Google Cluster Trace* pada penelitian ini tidak dimaksudkan sebagai representasi langsung dari sistem *edge computing* nyata. Dataset tersebut berasal dari lingkungan klaster komputasi, sehingga karakteristiknya berbeda dengan sistem *edge computing* yang memiliki node tersebar lebih dekat dengan pengguna. Namun, dataset ini tetap relevan untuk simulasi karena menyediakan variasi kebutuhan CPU dan memori pada *task*, serta variasi kapasitas mesin yang dapat digunakan untuk membentuk lingkungan *heterogeneous edge computing* secara terkontrol.

Pada sisi node, data mesin digunakan untuk memperoleh informasi kapasitas CPU dan memori. Kapasitas tersebut digunakan sebagai dasar pembentukan sembilan *edge nodes* pada topologi grid 3×3 . Nilai kapasitas CPU dan memori pada dataset bersifat ternormalisasi, sehingga tidak direpresentasikan dalam satuan fisik seperti jumlah *core*, GHz, atau GB. Karena kapasitas node dan kebutuhan *task* berada dalam skema normalisasi yang sama, nilai tersebut dapat digunakan untuk menghitung utilisasi sumber daya pada simulasi.

Untuk membentuk tingkat heterogenitas node, kapasitas CPU dan memori digabungkan menggunakan skor kapasitas. Skor ini digunakan untuk mengurutkan dan mengelompokkan node ke dalam tiga tingkatan, yaitu *low*

tier, *mid tier*, dan *high tier*. Secara umum, skor kapasitas node dihitung menggunakan persamaan 3.1. berikut:

$$capacity_score = \alpha_{cpu} \cdot cpu_capacity + \beta_{mem} \cdot memory_capacity. \quad (3.III.1)$$

Commented [AJ4]: Ganti dengan simbol

dengan α_{cpu} sebagai bobot kapasitas CPU dan β_{mem} sebagai bobot kapasitas memori. Pada simulasi ini, nilai α_{cpu} ditetapkan sebesar 0,4 dan β_{mem} sebesar 0,6. Nilai tersebut ditetapkan berdasarkan pertimbangan sebaran kapasitas pada dataset mesin. Ringkasan statistik dataset menunjukkan bahwa *memory capacity* memiliki variasi yang lebih lebar dibandingkan *CPU capacity*, sehingga memori diberi bobot sedikit lebih besar dalam pembentukan skor kapasitas node. Bobot ini digunakan sebagai asumsi model simulasi dan diterapkan secara konsisten pada seluruh metode optimasi yang dibandingkan.

Pada sisi *task*, dataset pekerjaan digunakan untuk membentuk sekumpulan *task* yang akan dijadwalkan ke *edge nodes*. Setiap *task* direpresentasikan terutama melalui kebutuhan CPU dan kebutuhan memori yang berasal dari *resource request* pada dataset. Kedua parameter tersebut digunakan untuk menghitung utilisasi sumber daya ketika suatu *task* dialokasikan ke node tertentu. Atribut lain seperti *job ID*, *task index*, dan *priority* digunakan sebagai informasi pendukung dalam pembentukan data *task*, tetapi tidak menjadi komponen utama dalam perhitungan fungsi objektif simulasi.

Jumlah *task* yang digunakan pada simulasi dibatasi agar proses pengujian tetap terkontrol dan waktu komputasi algoritma optimasi tetap dapat dikelola. Pada penelitian ini, jumlah *task* yang digunakan dalam satu skenario simulasi adalah 300 *task*. Pembatasan jumlah *task* dilakukan untuk menjaga keseimbangan antara kompleksitas ruang solusi dan efisiensi proses eksperimen.

Parameter utama yang digunakan dalam pembentukan *task* simulasi ditunjukkan pada Tabel 3.3.

Tabel 3.III.3. Parameter *task* simulasi

Parameter	Keterangan
-----------	------------

<i>CPU demand</i>	Kebutuhan CPU dari setiap task
<i>Memory demand</i>	Kebutuhan memori dari setiap task
<i>Priority</i>	Tingkat prioritas task apabila digunakan dalam pembentukan atau pemilihan sampel task
<i>Task index</i>	Identitas task untuk membedakan setiap unit pekerjaan dalam simulasi
<i>Job ID</i>	Identitas pekerjaan asal task yang digunakan untuk menjaga keterkaitan data dengan sumber dataset

Setelah parameter *task* dan node terbentuk, setiap metode optimasi akan menghasilkan solusi berupa pemetaan *task* ke *edge nodes*. Solusi tersebut kemudian dievaluasi berdasarkan model *latency* dan konsumsi energi yang telah ditentukan dalam simulasi. Dengan demikian, dataset berperan sebagai dasar pembentukan beban kerja simulasi sekaligus sebagai input utama dalam proses evaluasi metode *task scheduling* pada lingkungan *heterogeneous edge computing*.

3.3.3. Pemodelan Sistem pada Implementasi VPS

Menjelaskan model implementasi nyata berbasis VPS, termasuk task generator data-driven, central node, edge node, REST API, pengiriman task, eksekusi workload, pengumpulan hasil, baseline random, dan metrik real.

a. Arsitektur Node pada Lingkungan VPS

Arsitektur node pada lingkungan VPS mengikuti rancangan umum sistem yang telah ditunjukkan pada Gambar 3.2. Sistem terdiri dari satu *central node* dan sembilan *edge nodes* yang saling terhubung melalui komunikasi berbasis REST API. Pada fase ini, VPS digunakan sebagai lingkungan validasi untuk menjalankan *task* secara nyata pada node komputasi virtual, sehingga hasil pengujian tidak hanya bergantung pada model simulasi program.

Spesifikasi VPS yang digunakan dalam penelitian telah dirangkum pada Tabel 3.2. Berdasarkan konfigurasi tersebut, *central node* berperan sebagai koordinator sistem yang mengelola pembentukan *task*, pengumpulan

informasi node, proses optimasi, serta pengiriman keputusan alokasi *task* ke *edge nodes*. *Central node* tidak difokuskan sebagai unit eksekusi utama, melainkan sebagai pengendali proses penjadwalan dan pengelola komunikasi antar node.

Sembilan *edge nodes* pada lingkungan VPS berfungsi sebagai unit eksekusi *task*. Node-node tersebut dibagi ke dalam tiga tingkatan sumber daya, yaitu *low tier*, *mid tier*, dan *high tier*. Pembagian tingkatan ini digunakan untuk merepresentasikan kondisi *heterogeneous edge computing*, yaitu kondisi ketika setiap node memiliki kapasitas CPU dan memori yang berbeda. Dengan adanya variasi kapasitas tersebut, metode optimasi dapat diuji dalam kondisi yang menuntut pemilihan node secara lebih selektif.

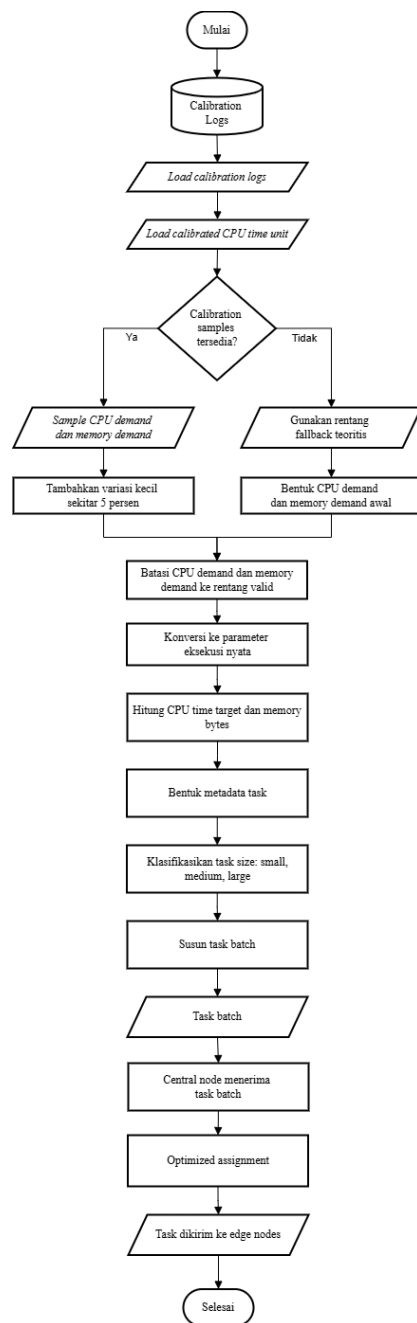
Pada arsitektur VPS, proses pengambilan keputusan dilakukan secara terpusat pada *central node*, sedangkan eksekusi *task* dilakukan secara terdistribusi oleh *edge nodes*. Setelah *task* dibentuk, *central node* menentukan alokasi *task* berdasarkan metode optimasi yang digunakan. Selanjutnya, *task* dikirim ke *edge node* tujuan untuk dieksekusi, kemudian hasil eksekusi dikembalikan ke *central node* untuk dicatat dan dievaluasi.

Pemodelan arsitektur VPS ini digunakan untuk memvalidasi kandidat metode optimasi yang telah dipilih dari fase simulasi program. Berbeda dengan simulasi program yang mengevaluasi solusi berdasarkan model perhitungan, fase VPS mengevaluasi metode berdasarkan hasil eksekusi *task* pada lingkungan server virtual. Dengan demikian, arsitektur VPS berperan sebagai penghubung antara model simulasi dan pengujian sistem yang lebih mendekati kondisi implementasi nyata.

b. **Mekanisme Pembentukan Task pada VPS**

Pada fase validasi VPS, pembentukan *task* dilakukan menggunakan pendekatan *data-driven task generation*. Pendekatan ini digunakan agar *task* yang dieksekusi pada *edge nodes* tidak dibentuk secara acak tanpa acuan, tetapi mengikuti pola kebutuhan sumber daya dari data kalibrasi. Alur umum pembentukan *task* pada lingkungan VPS ditunjukkan pada Gambar 3.X.

Commented [AJ5]: Tambahkan penjelasan simulasi task execution



Berdasarkan Gambar 3.X, proses pembentukan *task* pada VPS diawali dengan pembacaan *calibration logs* dan satuan waktu CPU terkalibrasi. Setelah itu, sistem memeriksa apakah *calibration samples* tersedia. Apabila *calibration samples* tersedia, sistem mengambil sampel *CPU demand* dan *memory demand* dari data kalibrasi, kemudian menambahkan variasi kecil sekitar 5% agar *task* yang dihasilkan tidak identik dengan sampel awal. Apabila *calibration samples* tidak tersedia, sistem menggunakan rentang *fallback* teoritis untuk membentuk nilai awal *CPU demand* dan *memory demand*.

Nilai *CPU demand* dan *memory demand* yang diperoleh dari salah satu jalur tersebut kemudian dibatasi ke dalam rentang valid agar sesuai dengan kapasitas lingkungan VPS. Setelah itu, nilai kebutuhan sumber daya dikonversi menjadi parameter eksekusi nyata, yaitu target waktu komputasi dan kebutuhan memori dalam satuan *bytes*. Parameter tersebut kemudian digunakan untuk membentuk metadata *task*, mengklasifikasikan ukuran *task* menjadi *small*, *medium*, atau *large*, serta menyusun seluruh *task* ke dalam satu *task batch*. *Task batch* tersebut kemudian diterima oleh *central node* untuk diproses dalam tahap optimasi, menghasilkan *optimized assignment*, dan akhirnya *task* dikirim ke *edge nodes* untuk dieksekusi.

Secara formal, mekanisme pembentukan *task* tersebut dapat dinyatakan dengan mendefinisikan sekumpulan data kalibrasi sebagai berikut.

$$\mathcal{C} = \{(c_i, m_i)\}_{i=1}^{N_c}$$

dengan c_i merupakan *CPU demand* dari sampel kalibrasi ke- i , m_i merupakan *memory demand* dari sampel kalibrasi ke- i , dan N_c merupakan jumlah sampel kalibrasi yang tersedia. Untuk membentuk *task* baru, sistem mengambil salah satu sampel dari data kalibrasi sebagai dasar pembentukan kebutuhan sumber daya.

Agar *task* yang dibentuk tidak identik dengan sampel kalibrasi, nilai *CPU demand* dan *memory demand* diberi variasi acak kecil. Variasi ini dapat dinyatakan sebagai:

$$\tilde{c}_i = c_i \cdot \epsilon_c$$

$$\tilde{m}_i = m_i \cdot \epsilon_m$$

dengan ϵ_c dan ϵ_m merupakan faktor variasi acak yang mengikuti distribusi normal di sekitar nilai 1. Pada penelitian ini, variasi yang digunakan sekitar 5%, sehingga secara konseptual dapat dituliskan sebagai:

$$\epsilon_c, \epsilon_m \sim \mathcal{N}(1, 0.05)$$

Nilai hasil variasi kemudian dibatasi pada rentang valid agar *task* yang dihasilkan tetap sesuai dengan kapasitas lingkungan VPS. Pembatasan nilai dilakukan menggunakan fungsi *clip* sebagai berikut:

$$c'_i = \min(\max(\tilde{c}_i, 0.8), 3.6)$$

$$m'_i = \min(\max(\tilde{m}_i, 0.125), 0.75)$$

dengan c'_i merupakan *CPU demand* akhir dan m'_i merupakan *memory demand* akhir yang digunakan oleh *task*. Rentang tersebut digunakan untuk mencegah terbentuknya *task* yang terlalu kecil maupun terlalu besar secara tidak terkendali.

Setelah *CPU demand* dan *memory demand* diperoleh, kedua nilai tersebut dikonversi menjadi parameter eksekusi nyata pada VPS. *CPU demand* dikonversi menjadi target waktu komputasi menggunakan satuan waktu CPU hasil kalibrasi, sedangkan *memory demand* dikonversi menjadi kebutuhan memori aktual dalam satuan *bytes*. Formulasi konversi tersebut dapat dituliskan sebagai berikut:

$$T_i^{cpu} = c'_i \cdot U_{cpu}$$

$$M_i^{bytes} = m'_i \cdot U_{mem}$$

dengan T_i^{cpu} merupakan target waktu komputasi *task* ke-*i*, U_{cpu} merupakan satuan waktu CPU hasil kalibrasi, M_i^{bytes} merupakan kebutuhan memori aktual *task* ke-*i*, dan U_{mem} merupakan satuan memori yang digunakan. Pada penelitian ini, U_{mem} setara dengan 1 GB, sehingga nilai *memory demand* dikonversi menjadi ukuran memori aktual yang akan digunakan pada saat eksekusi.

Setiap *task* yang dibentuk kemudian dilengkapi dengan atribut eksekusi, seperti identitas *task*, *CPU demand*, *memory demand*, target waktu

komputasi, kebutuhan memori aktual, jenis *task*, dan parameter *payload*. Jenis *task* yang digunakan pada validasi VPS adalah *cpu_mem_burn*, yaitu beban kerja yang memberikan tekanan pada CPU dan memori sesuai parameter yang telah dibentuk.

Selain itu, setiap *task* diklasifikasikan ke dalam kategori ukuran *small*, *medium*, atau *large* berdasarkan target waktu komputasi dan kebutuhan memori. Klasifikasi ini dapat dinyatakan sebagai:

$$size_i = \begin{cases} small, & T_i^{cpu} < 325 \text{ ms} \wedge M_i^{bytes} < 0.25 \text{ GB} \\ medium, & T_i^{cpu} < 675 \text{ ms} \wedge M_i^{bytes} < 0.75 \text{ GB} \\ large, & \text{lainnya} \end{cases}$$

Klasifikasi tersebut digunakan untuk membedakan tingkat beban *task* berdasarkan kebutuhan komputasi dan memori. Dengan adanya klasifikasi ini, sistem dapat menganalisis distribusi *task* berdasarkan ukuran beban kerja yang dieksekusi pada *edge nodes*.

Apabila data kalibrasi tidak tersedia, sistem menggunakan mekanisme *fallback* berbasis rentang teoritis. Pada kondisi tersebut, target waktu komputasi dibentuk dari rentang 200 hingga 900 ms, sedangkan kebutuhan memori dibentuk dari rentang 128 hingga 768 MB. Mekanisme ini digunakan agar sistem tetap dapat membentuk *task* meskipun data kalibrasi belum tersedia. Namun, pada penelitian ini, pembentukan *task* berbasis data kalibrasi menjadi mekanisme utama karena lebih merepresentasikan karakteristik beban kerja pada lingkungan VPS.

Secara umum, proses pembentukan *task batch* pada VPS dapat dirumuskan sebagai:

$$\mathcal{T} = \{t_1, t_2, \dots, t_n\}$$

dengan setiap *task* t_i direpresentasikan sebagai:

$$t_i = (id_i, c'_i, m'_i, T_i^{cpu}, M_i^{bytes}, type_i, size_i)$$

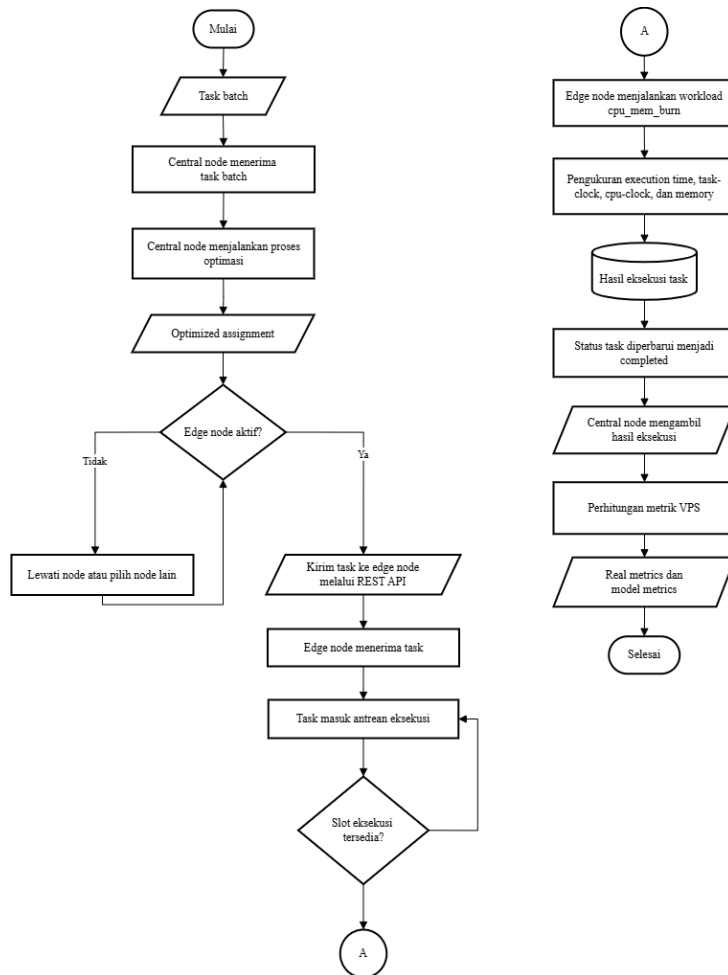
di mana n merupakan jumlah *task* dalam satu skenario pengujian. *Task batch* tersebut kemudian digunakan sebagai input untuk proses optimasi pada *central node*. Setelah proses optimasi selesai, setiap *task* akan dipetakan ke

salah satu *edge node* untuk dieksekusi berdasarkan hasil *optimized assignment*.

Dengan formulasi tersebut, mekanisme pembentukan *task* pada VPS tidak hanya dijelaskan secara konseptual, tetapi juga memiliki bentuk matematis yang menunjukkan bagaimana data kalibrasi dikonversi menjadi beban kerja nyata. Hal ini memperkuat posisi fase VPS sebagai proses validasi berbasis eksekusi nyata, bukan sekadar pembangkitan *task* acak.

c. Komunikasi dan Eksekusi Task antar Node

Setelah *task batch* terbentuk, tahap berikutnya adalah proses komunikasi antara *central node* dan *edge nodes* untuk menjalankan *task* berdasarkan hasil alokasi. Pada lingkungan VPS, *central node* berperan sebagai pengendali utama yang mengatur proses penjadwalan, pengiriman *task*, dan pengumpulan hasil eksekusi. Sementara itu, *edge nodes* berperan sebagai unit komputasi yang menerima *task*, menjalankan beban kerja, dan menyimpan hasil eksekusi. Alur komunikasi dan eksekusi *task* antar node pada lingkungan VPS ditunjukkan pada Gambar 3.X.



Gambar III.2. Alur komunikasi dan eksekusi *task* antar node pada lingkungan VPS

Berdasarkan Gambar 3.X, proses dimulai ketika *central node* menerima *task batch* yang telah dibentuk melalui mekanisme *data-driven task generation*. *Task batch* tersebut kemudian digunakan sebagai masukan dalam proses optimasi untuk menghasilkan *optimized assignment*. *Optimized assignment* merepresentasikan keputusan pemetaan setiap *task* ke *edge node*

tertentu. Keputusan ini menjadi dasar bagi *central node* untuk menentukan node tujuan sebelum *task* dikirimkan ke lingkungan eksekusi.

Sebelum proses pengiriman dilakukan, *central node* memeriksa ketersediaan *edge nodes* untuk memastikan bahwa node tujuan berada dalam kondisi aktif. Pemeriksaan ini diperlukan karena pada lingkungan terdistribusi, setiap node dapat memiliki kondisi ketersediaan yang berbeda. Apabila node tujuan tidak aktif atau tidak dapat menerima *task*, sistem dapat melewati node tersebut dan memilih node lain yang tersedia. Dengan mekanisme ini, proses distribusi *task* dapat tetap berjalan meskipun terdapat node yang tidak siap menerima beban kerja.

Komunikasi antara *central node* dan *edge nodes* dilakukan menggunakan mekanisme HTTP berbasis REST API. Setelah node tujuan dinyatakan aktif, *central node* mengirimkan *task* ke *edge node* sesuai hasil alokasi. Setiap *task* yang dikirim membawa beberapa parameter eksekusi, seperti identitas *task*, kebutuhan CPU, kebutuhan memori, target waktu komputasi, kebutuhan memori dalam satuan *bytes*, serta *payload* pendukung. Parameter tersebut digunakan oleh *edge node* untuk menjalankan beban kerja sesuai karakteristik *task* yang diterima.

Pada sisi *edge node*, *task* yang diterima akan dicatat dan dimasukkan ke dalam antrian eksekusi. Status awal *task* ditandai sebagai *queued*, yang menunjukkan bahwa *task* telah diterima tetapi belum dieksekusi. Mekanisme antrian diperlukan karena setiap *edge node* memiliki batas jumlah *task* yang dapat dijalankan secara bersamaan. Dengan adanya antrian, proses eksekusi dapat dikendalikan agar tidak melebihi kapasitas node dan menghindari pembebanan berlebih pada satu node.

Sebelum *task* dijalankan, *edge node* memeriksa ketersediaan slot eksekusi. Apabila slot eksekusi belum tersedia, *task* tetap berada dalam antrian hingga node siap menjalankan *task* berikutnya. Apabila slot tersedia, *task* dikeluarkan dari antrian dan diproses oleh *worker* pada *edge node*. Mekanisme ini membantu menjaga agar proses eksekusi tetap terkontrol,

terutama ketika jumlah *task* yang dikirim lebih besar dibandingkan kapasitas eksekusi simultan pada node.

Eksekusi *task* pada *edge node* dilakukan menggunakan beban kerja *cpu_mem_burn*. Beban kerja ini dirancang untuk memberikan tekanan pada CPU dan memori berdasarkan parameter yang dibawa oleh setiap *task*. Target waktu komputasi digunakan untuk mengatur durasi aktivitas CPU, sedangkan kebutuhan memori dalam satuan *bytes* digunakan untuk menentukan ukuran memori yang dialokasikan dan disentuh selama proses eksekusi. Dengan demikian, *task* pada fase VPS tidak hanya direpresentasikan sebagai nilai model, tetapi benar-benar dijalankan sebagai beban kerja pada node virtual.

Selama proses eksekusi, sistem mengumpulkan metrik performa yang digunakan untuk mengevaluasi hasil validasi VPS. Metrik yang dikumpulkan meliputi *execution time*, *task-clock*, *cpu-clock*, penggunaan memori terobservasi, dan *latency*. *Execution time* menunjukkan durasi eksekusi *task* dari awal hingga selesai. *Task-clock* dan *cpu-clock* digunakan untuk menggambarkan aktivitas komputasi selama proses eksekusi. Sementara itu, penggunaan memori terobservasi digunakan untuk mengetahui besarnya memori yang benar-benar digunakan oleh *task* pada saat dijalankan.

Setelah proses eksekusi selesai, *edge node* memperbarui status *task* menjadi *completed* dan menyimpan hasil eksekusi. Hasil tersebut kemudian diambil oleh *central node* untuk dihitung sebagai bagian dari metrik evaluasi. Proses pengambilan hasil ini memungkinkan *central node* mengumpulkan informasi dari seluruh *edge nodes* dan menyusun ringkasan performa sistem. Dengan cara ini, evaluasi pada VPS dilakukan berdasarkan hasil eksekusi nyata, bukan hanya berdasarkan estimasi model.

Hasil eksekusi yang dikumpulkan dari seluruh *task* kemudian digunakan untuk membentuk dua kelompok metrik, yaitu *real metrics* dan *model metrics*. *Real metrics* berasal dari hasil eksekusi langsung pada *edge nodes*, seperti *real latency*, waktu eksekusi, *task-clock*, *cpu-clock*, memori terobservasi, dan estimasi energi nyata. Sementara itu, *model metrics* berasal dari perhitungan model estimasi, seperti *model latency*, *model energy*, dan

distribusi *task* antar node. Pemisahan kedua jenis metrik ini penting karena perilaku eksekusi nyata pada VPS dapat berbeda dari estimasi model.

Dengan mekanisme komunikasi dan eksekusi tersebut, fase VPS berfungsi sebagai tahap validasi kandidat metode optimasi yang telah dipilih dari fase simulasi. Sistem tidak hanya mengevaluasi hasil alokasi berdasarkan nilai fungsi objektif, tetapi juga mengamati dampak keputusan alokasi terhadap performa eksekusi nyata. Oleh karena itu, hasil dari fase VPS digunakan untuk melihat sejauh mana metode optimasi mampu meningkatkan performa sistem pada lingkungan *edge computing* heterogen berbasis VPS.

d. Baseline dan Evaluasi pada VPS

Evaluasi pada lingkungan VPS dilakukan untuk memvalidasi kandidat metode optimasi yang telah dipilih dari fase simulasi program. Berbeda dengan fase simulasi yang menggunakan baseline *Round-Robin*, fase VPS menggunakan *random baseline average* sebagai pembanding. Baseline ini dibentuk dengan menjalankan beberapa kali alokasi *task* secara acak pada *task batch* yang sama, kemudian hasil metrik dari seluruh percobaan tersebut dirata-ratakan. Pendekatan ini digunakan agar baseline tidak bergantung pada satu hasil acak tertentu, sehingga perbandingan terhadap metode optimasi menjadi lebih stabil.

Penggunaan *random baseline average* pada VPS dipilih karena lingkungan eksekusi nyata memiliki variasi performa yang lebih besar dibandingkan simulasi program. Faktor seperti kondisi node, antrean eksekusi, komunikasi antar VPS, dan waktu respons sistem dapat memengaruhi hasil eksekusi *task*. Oleh karena itu, penggunaan satu kali baseline acak saja dapat menghasilkan pembanding yang kurang representatif. Dengan merata-ratakan beberapa percobaan baseline acak, nilai pembanding yang diperoleh menjadi lebih adil untuk mengevaluasi performa kandidat metode optimasi.

Setelah baseline acak terbentuk, nilai energi dan *latency* dari baseline tersebut digunakan sebagai referensi dalam evaluasi metode optimasi pada VPS. Nilai referensi ini digunakan untuk membantu proses normalisasi fungsi

objektif pada metode optimasi yang dijalankan. Secara konseptual, referensi tersebut dapat dinyatakan sebagai:

$$E_{ref} = E_{random}$$

$$L_{ref} = L_{random}$$

dengan E_{ref} merupakan nilai energi referensi dari rata-rata baseline acak, sedangkan L_{ref} merupakan nilai *latency* referensi dari rata-rata baseline acak. Nilai referensi ini digunakan agar proses optimasi dapat mengevaluasi solusi alokasi *task* terhadap kondisi pembanding yang sama pada lingkungan VPS.

Pada fase VPS, kandidat metode optimasi dijalankan pada *task batch* yang sama dengan baseline acak. Hal ini dilakukan agar perbandingan antara baseline dan metode optimasi tidak dipengaruhi oleh perbedaan karakteristik *task*. Metode optimasi menghasilkan *assignment* yang menentukan node tujuan untuk setiap *task*, kemudian *task* dikirim dan dieksekusi pada *edge nodes* sesuai hasil alokasi tersebut. Hasil eksekusi dari baseline acak dan metode optimasi kemudian dibandingkan menggunakan metrik evaluasi yang sama.

Evaluasi pada VPS menggunakan dua kelompok metrik, yaitu *real metrics* dan *model metrics*. *Real metrics* merupakan metrik yang diperoleh dari hasil eksekusi langsung pada *edge nodes*. Metrik ini mencakup *real latency*, *execution time*, *task-clock*, *cpu-clock*, penggunaan memori terobservasi, dan estimasi energi nyata. Metrik tersebut digunakan untuk melihat dampak alokasi *task* terhadap performa eksekusi sistem secara aktual pada lingkungan VPS.

Sementara itu, *model metrics* merupakan metrik yang diperoleh dari perhitungan model estimasi. Metrik ini mencakup *model latency*, *model energy*, dan distribusi *task* antar node. *Model metrics* digunakan untuk melihat bagaimana model evaluasi memperkirakan performa solusi alokasi yang dihasilkan oleh metode optimasi. Pemisahan antara *real metrics* dan *model metrics* diperlukan karena hasil eksekusi nyata pada VPS dapat berbeda dari estimasi model, terutama pada aspek energi dan waktu eksekusi.

Metrik evaluasi utama pada fase VPS ditunjukkan pada Tabel 3.X.

Tabel III.4. Metrik evaluasi utama pada fase VPS

Kelompok Metrik	Metrik	Keterangan
<i>Real metrics</i>	<i>Real average latency</i>	Rata-rata <i>latency</i> hasil eksekusi nyata seluruh <i>task</i>
<i>Real metrics</i>	<i>Real total latency</i>	Total <i>latency</i> seluruh <i>task</i> pada satu skenario pengujian
<i>Real metrics</i>	<i>Average execution time</i>	Rata-rata waktu eksekusi <i>task</i> pada <i>edge nodes</i>
<i>Real metrics</i>	<i>Task-clock</i>	Waktu aktivitas komputasi yang terukur selama eksekusi <i>task</i>
<i>Real metrics</i>	<i>CPU-clock</i>	Aktivitas CPU yang tercatat selama eksekusi <i>task</i>
<i>Real metrics</i>	<i>Observed memory bytes</i>	Besar memori yang terobservasi selama eksekusi <i>task</i>
<i>Real metrics</i>	<i>Estimated real energy</i>	Estimasi energi nyata berdasarkan metrik eksekusi dan model daya
<i>Model metrics</i>	<i>Model average latency</i>	Estimasi rata-rata <i>latency</i> berdasarkan model perhitungan
<i>Model metrics</i>	<i>Model total energy</i>	Estimasi total energi berdasarkan model sistem
<i>Model metrics</i>	<i>Task distribution</i>	Distribusi jumlah <i>task</i> pada setiap <i>edge node</i>

Estimasi energi nyata pada VPS tidak diperoleh melalui pengukuran langsung menggunakan alat ukur daya eksternal, seperti *wattmeter*. Energi dihitung menggunakan pendekatan estimasi berdasarkan karakteristik daya node dan metrik eksekusi yang teramati, seperti *task-clock* dan *cpu-clock*. Oleh karena itu, nilai energi pada fase VPS perlu dipahami sebagai estimasi energi, bukan konsumsi listrik aktual yang diukur langsung dari perangkat keras.

Hasil evaluasi VPS digunakan untuk membandingkan performa metode optimasi terhadap *random baseline average*. Perbandingan dilakukan dengan

melihat apakah metode optimasi mampu menurunkan *latency*, memperbaiki waktu eksekusi, dan mengurangi estimasi energi dibandingkan baseline. Namun, karena metrik nyata dapat dipengaruhi oleh kondisi eksekusi VPS, hasil *real metrics* dan *model metrics* dianalisis secara terpisah. Pemisahan ini penting agar pembahasan hasil tidak menyamakan penurunan nilai model dengan peningkatan performa nyata secara langsung.

Dengan mekanisme evaluasi tersebut, fase VPS berperan sebagai tahap validasi kandidat metode optimasi pada lingkungan eksekusi yang lebih mendekati sistem nyata. Evaluasi ini tidak dimaksudkan untuk membandingkan seluruh kombinasi metode seperti pada fase simulasi, melainkan untuk menguji perilaku kandidat metode terpilih ketika diterapkan pada sistem *edge computing* berbasis VPS. Oleh karena itu, hasil dari fase VPS digunakan untuk melengkapi hasil simulasi program dan memberikan gambaran mengenai performa metode optimasi pada eksekusi *task* nyata.

3.4. Implementasi Solusi

fokusnya adalah menjelaskan bagaimana solusi optimasi dirancang dan diterapkan. Bagian ini tidak lagi menjelaskan arsitektur sistem, tetapi masuk ke cara metode bekerja pada representasi *task scheduling*. Subbabnya:

3.4.1. Perancangan Objective Function

Di sini kita jelaskan fungsi objektif yang digunakan untuk mengevaluasi kualitas alokasi *task*. Harus dibedakan antara fungsi objektif simulasi dan adaptasi fungsi objektif pada VPS. Simulasi memakai objektif gabungan energi dan *latency* yang dinormalisasi terhadap *Round-Robin baseline*, sedangkan VPS memakai referensi dari *random baseline average* dan memiliki adaptasi tambahan agar sesuai dengan kondisi node nyata.

Objective function digunakan untuk mengevaluasi kualitas solusi alokasi *task* pada sistem *edge computing*. Dalam penelitian ini, satu solusi merepresentasikan pemetaan sejumlah *task* ke beberapa *edge nodes*. Setiap solusi dievaluasi berdasarkan dua parameter utama, yaitu konsumsi energi dan *latency*.

Commented [AJ6]: Dijelaskan implementasi objective function di program dan VPS

Commented [AJ7R6]: Sinkronkan kan penyebutan objective function --> Multi Objective function

Commented [AJ8R6]: Penjelasan konversi Sistem model Energi Latensi

Kedua parameter tersebut dipilih karena sesuai dengan tujuan penelitian, yaitu menurunkan konsumsi energi dan *latency* pada lingkungan *heterogeneous edge computing*.

Pada fase simulasi program, *objective function* dirancang sebagai fungsi gabungan berbasis normalisasi terhadap baseline. Baseline yang digunakan pada fase simulasi adalah *Round-Robin*, yaitu metode alokasi *task* secara bergantian ke setiap node tanpa mempertimbangkan kondisi sumber daya. Nilai energi dan *latency* dari baseline tersebut digunakan sebagai nilai referensi untuk menormalisasi hasil setiap solusi optimasi. Dengan demikian, kualitas solusi dari setiap metode dapat dibandingkan dalam skala yang sama.

Secara matematis, *objective function* pada fase simulasi dinyatakan sebagai berikut:

$$F(x) = w_E \cdot \frac{E(x)}{E_{ref}} + w_L \cdot \frac{L(x)}{L_{ref}}$$

dengan $F(x)$ merupakan nilai objektif dari solusi alokasi x , $E(x)$ merupakan konsumsi energi dari solusi x , dan $L(x)$ merupakan nilai *latency* dari solusi x . Sementara itu, E_{ref} dan L_{ref} masing-masing merupakan nilai energi dan *latency* dari baseline *Round-Robin*. Nilai w_E dan w_L merupakan bobot untuk komponen energi dan *latency*.

Pada penelitian ini, nilai bobot yang digunakan pada fase simulasi adalah:

$$w_E = 0,5$$

$$w_L = 0,5$$

Pembobotan tersebut digunakan untuk memberikan kontribusi yang seimbang antara konsumsi energi dan *latency* dalam proses evaluasi solusi. Normalisasi terhadap baseline dilakukan agar kedua komponen dapat dibandingkan meskipun memiliki satuan dan rentang nilai yang berbeda. Nilai objektif yang lebih kecil menunjukkan bahwa solusi memiliki performa yang lebih baik dibandingkan baseline.

Konsumsi energi pada model simulasi dihitung menggunakan pendekatan *power-aware*. Pendekatan ini mempertimbangkan estimasi daya node berdasarkan karakteristik *idle power*, *maximum power*, utilisasi CPU, utilisasi memori, serta penalti ketika terjadi kelebihan beban. Dengan model ini, penggunaan node berdaya tinggi secara berlebihan dapat meningkatkan nilai energi, sehingga algoritma optimasi didorong untuk memilih alokasi *task* yang tidak hanya cepat, tetapi juga efisien secara energi.

Nilai *latency* pada model simulasi dihitung menggunakan pendekatan *network-aware*. Model ini mempertimbangkan waktu layanan, kondisi antrean akibat utilisasi node, penalti kelebihan memori, serta estimasi *network delay* pada setiap node. Dengan demikian, solusi yang menyebabkan beban terlalu tinggi pada node tertentu akan menghasilkan nilai *latency* yang lebih besar. Pendekatan ini digunakan agar proses optimasi tidak hanya mempertimbangkan kapasitas komputasi node, tetapi juga kondisi beban dan karakteristik komunikasi sistem.

Pada fase VPS, *objective function* digunakan untuk mengevaluasi kandidat metode optimasi pada lingkungan eksekusi yang lebih mendekati kondisi nyata. Berbeda dengan fase simulasi, baseline referensi pada VPS diperoleh dari *random baseline average*. Baseline ini dibentuk dari beberapa percobaan alokasi *task* secara acak pada *task batch* yang sama, kemudian metriknya dirata-ratakan. Nilai energi dan *latency* dari baseline acak tersebut digunakan sebagai referensi normalisasi pada proses optimasi VPS.

Secara konseptual, *objective function* pada VPS tetap mempertimbangkan energi dan *latency*, tetapi dilengkapi dengan komponen penalti tambahan agar lebih sesuai dengan kondisi node nyata. Bentuk umum fungsi objektif pada VPS dapat dituliskan sebagai berikut:

$$F_{VPS}(x) = F_{base}(x) + \lambda \cdot P_{VPS}(x)$$

dengan $F_{base}(x)$ merupakan komponen objektif dasar berbasis energi dan *latency*, $P_{VPS}(x)$ merupakan penalti yang berkaitan dengan kondisi lingkungan VPS, dan λ merupakan faktor skala penalti. Komponen penalti pada VPS

digunakan untuk menghindari alokasi yang terlalu agresif ke node tertentu, terutama node berdaya tinggi, ketika node dengan kapasitas lebih rendah masih mampu menangani *task*.

Komponen penalti pada VPS mencakup beberapa aspek, seperti penalti penggunaan *high-tier node*, penalti tekanan sumber daya, penalti *concurrency*, penalti *energy overshoot*, dan penalti ketidaksesuaian ukuran *task* dengan tier node. Penalti ini digunakan agar proses optimasi tidak hanya mengejar *latency* rendah dengan memindahkan terlalu banyak *task* ke node berkapasitas tinggi, tetapi juga mempertimbangkan efisiensi energi dan keseimbangan penggunaan sumber daya antar node.

Perbedaan utama antara *objective function* simulasi dan VPS terletak pada tujuan penggunaannya. Pada fase simulasi, fungsi objektif digunakan untuk membandingkan berbagai kombinasi metode optimasi secara terkontrol. Pada fase VPS, fungsi objektif digunakan untuk menyesuaikan keputusan alokasi terhadap karakteristik eksekusi nyata pada node VPS. Oleh karena itu, *objective function* VPS merupakan adaptasi dari fungsi objektif simulasi, bukan salinan identik dari model simulasi.

Dengan rancangan *objective function* tersebut, proses optimasi diarahkan untuk menghasilkan alokasi *task* yang mampu menurunkan nilai energi dan *latency* secara simultan. Solusi dengan nilai objektif lebih rendah dianggap memiliki kualitas yang lebih baik karena menunjukkan performa yang lebih efisien dibandingkan baseline. Rancangan ini menjadi dasar bagi proses pencarian solusi pada *search space* yang dijelaskan pada subbab berikutnya.

3.4.2. Perancangan *Search Space* Optimasi

Di sini kita jelaskan representasi solusi. Satu solusi adalah vektor alokasi *task* ke node. Misalnya, untuk *ntask* dan *mnode*, solusi dapat ditulis:

$$X = [x_1, x_2, \dots, x_n]$$

Commented [AJ9]: Perkata huruf besar

Commented [AJ10R9]: Rincian yang menggunakan program dan VPS. Yang diprogram butuh parameter apa untuk menghitung. Di VPS butuh parameter apa, jikpun tidak ada perlu da konfersi.

dengan $x_i \in \{0, 1, \dots, m - 1\}$, yang menunjukkan node tujuan untuk *task* ke- i . Bagian ini juga cocok menjelaskan representasi pada PSO, BFO, dan Tabu secara umum, misalnya posisi partikel atau kandidat solusi tetap diterjemahkan menjadi indeks node untuk setiap *task*.

a. **Representasi Solusi Task Scheduling**

Commented [AJ11]: Search Space untuk global dan lokal

Search space pada penelitian ini merepresentasikan seluruh kemungkinan alokasi *task* ke *edge nodes*. Dalam sistem *edge computing*, setiap *task* harus dipetakan ke salah satu node yang tersedia untuk dieksekusi. Oleh karena itu, satu solusi dalam penelitian ini dapat dipahami sebagai konfigurasi lengkap yang menunjukkan node tujuan dari setiap *task*. Representasi ini digunakan baik pada fase simulasi program maupun pada fase validasi VPS, meskipun tujuan evaluasi pada kedua fase tersebut berbeda.

Misalkan terdapat n *task* dan *medge nodes*. Satu solusi alokasi dapat direpresentasikan sebagai vektor:

$$x = [x_1, x_2, \dots, x_n]$$

dengan:

$$x_i \in \{0, 1, 2, \dots, m - 1\}$$

di mana x_i menunjukkan indeks *edge node* yang digunakan untuk mengeksekusi *task* ke- i . Apabila $x_i = 0$, maka *task* ke- i dialokasikan ke node 0. Apabila $x_i = 5$, maka *task* ke- i dialokasikan ke node 5. Dengan representasi ini, setiap elemen vektor menunjukkan keputusan alokasi untuk satu *task*, sedangkan keseluruhan vektor menunjukkan distribusi semua *task* pada sistem.

Representasi solusi berbasis vektor dipilih karena sesuai dengan karakteristik permasalahan *task scheduling* pada sistem *edge computing*. Setiap posisi dalam vektor berkaitan langsung dengan satu *task*, sehingga perubahan nilai pada posisi tersebut menunjukkan perpindahan *task* ke node lain. Bentuk representasi ini juga memudahkan proses evaluasi

menggunakan *objective function*, karena dari satu vektor solusi dapat dihitung distribusi beban, utilisasi CPU, utilisasi memori, estimasi energi, dan *latency* sistem.

Pada penelitian ini, jumlah node yang digunakan pada model simulasi adalah sembilan *edge nodes*, sehingga nilai setiap elemen solusi berada pada rentang 0 sampai 8. Sementara itu, jumlah *task* pada simulasi program dibatasi sebanyak 300 *task*. Dengan demikian, satu solusi pada simulasi dapat direpresentasikan sebagai vektor sepanjang 300 elemen, di mana setiap elemen berisi indeks node tujuan untuk masing-masing *task*. Representasi yang sama juga digunakan secara konseptual pada fase VPS, dengan perbedaan bahwa hasil alokasi pada VPS kemudian dikirim dan dieksekusi secara nyata oleh *edge nodes*.

b. Ukuran Ruang Pencarian

Ruang pencarian pada permasalahan *task scheduling* bersifat kombinatorial karena setiap *task* memiliki beberapa kemungkinan node tujuan. Apabila terdapat n *task* dan m *edge nodes*, maka setiap *task* dapat dialokasikan ke salah satu dari m node yang tersedia. Dengan demikian, jumlah kemungkinan solusi dalam ruang pencarian dapat dinyatakan sebagai:

$$|S| = m^n$$

dengan $|S|$ merupakan ukuran ruang pencarian, m merupakan jumlah *edge nodes*, dan n merupakan jumlah *task*. Persamaan tersebut menunjukkan bahwa jumlah kombinasi solusi akan meningkat secara eksponensial seiring bertambahnya jumlah *task*. Kondisi ini menyebabkan pencarian menyeluruh terhadap seluruh kemungkinan solusi menjadi tidak efisien untuk dilakukan.

Pada penelitian ini, model simulasi menggunakan sembilan *edge nodes* dan 300 *task*. Dengan demikian, ukuran ruang pencarian secara teoritis dapat dituliskan sebagai:

$$|S| = 9^{300}$$

Commented [AJ12]: Dihubungkan dengan kondisi real. Agar terlihat gambarannya. Misal satu node bisa mengeksekusi berapa data dari IoT. Intervalnya berapa

Nilai tersebut menunjukkan bahwa jumlah kemungkinan alokasi *task* sangat besar. Oleh karena itu, pencarian solusi secara *exhaustive search* tidak realistis untuk diterapkan karena membutuhkan waktu komputasi yang sangat tinggi. Permasalahan ini menjadi dasar penggunaan metode optimasi berbasis *metaheuristic*, seperti *Tabu Search*, PSO, dan BFO, untuk mencari solusi yang mendekati optimal dalam waktu komputasi yang lebih efisien.

Besarnya ruang pencarian juga menunjukkan pentingnya mekanisme eksplorasi solusi yang efektif. Algoritma optimasi tidak hanya perlu menemukan solusi dengan nilai *objective function* rendah, tetapi juga harus mampu menghindari pencarian yang berulang pada wilayah solusi yang sama. Oleh karena itu, setiap metode optimasi menggunakan strategi eksplorasi yang berbeda dalam ruang pencarian yang sama. *Tabu Search* menggunakan mekanisme *neighborhood search* dan *tabu list*, PSO menggunakan pergerakan partikel, sedangkan BFO menggunakan mekanisme pencarian berbasis perilaku bakteri.

Dengan memahami ukuran ruang pencarian tersebut, proses optimasi pada penelitian ini diarahkan untuk mencari konfigurasi alokasi *task* yang mampu menurunkan konsumsi energi dan *latency* tanpa harus mengevaluasi seluruh kemungkinan solusi. Representasi ruang pencarian ini menjadi dasar bagi penerapan metode optimasi yang dijelaskan pada subbagian berikutnya.

c. Representasi *Search Space* pada *Tabu Search*

Pada *Tabu Search*, satu solusi dalam *search space* direpresentasikan sebagai vektor alokasi *task* ke *edge nodes*. Setiap elemen vektor menunjukkan node tujuan dari satu *task*. Proses pencarian dilakukan dengan membentuk solusi tetangga (*neighborhood solution*) melalui perubahan kecil terhadap solusi saat ini. Perubahan tersebut disebut sebagai *move*, yaitu operasi pemindahan satu *task* dari node asal ke node lain.

Misalkan solusi saat ini dinyatakan sebagai:

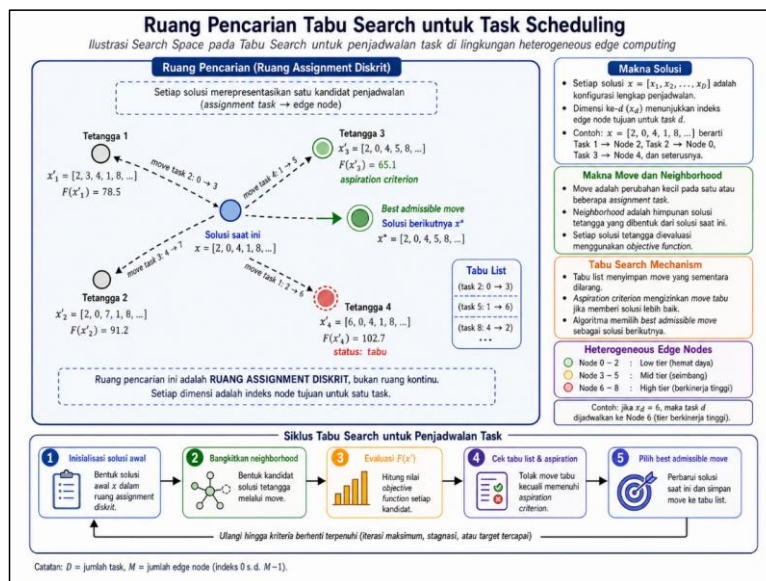
$$x = [x_1, x_2, \dots, x_i, \dots, x_n]$$

Sebuah *move* dapat dinyatakan sebagai perubahan nilai x_i dari node *a* ke node *b*, dengan $a \neq b$. Secara sederhana, operasi tersebut dapat dituliskan sebagai:

$$\text{move}(i, a, b): x_i = a \rightarrow x_i = b$$

dengan *i* merupakan indeks *task*, *a* merupakan node asal, dan *b* merupakan node tujuan baru. Hasil dari operasi ini adalah solusi baru x' yang memiliki konfigurasi alokasi berbeda dari solusi sebelumnya.

Ilustrasi mekanisme eksplorasi *search space* pada *Tabu Search* ditunjukkan pada Gambar 3.X. Pada gambar tersebut, solusi saat ini x direpresentasikan sebagai vektor alokasi *task* ke *edge nodes*. Dari solusi tersebut, algoritma membentuk beberapa solusi tetangga x'_1, x'_2, x'_3 , dan x'_4 melalui operasi *move*. Setiap *move* merepresentasikan perubahan alokasi satu *task* dari node asal ke node tujuan baru.



Gambar III.3. Ilustrasi search space pada Tabu Search

Himpunan solusi tetangga pada *Tabu Search* dibentuk dari beberapa kandidat *move* yang mungkin dilakukan terhadap solusi saat ini. Setiap

solusi tetangga kemudian dievaluasi menggunakan *objective function* untuk menentukan kualitasnya. Solusi dengan nilai objektif paling rendah dipilih sebagai kandidat solusi berikutnya, selama *move* tersebut tidak termasuk dalam *tabu list* atau memenuhi *aspiration criterion*.

Setiap solusi tetangga yang terbentuk kemudian dievaluasi menggunakan *objective function* $F(x')$. Nilai objektif tersebut digunakan untuk menentukan kandidat solusi yang paling baik pada iterasi berjalan. Namun, pemilihan solusi berikutnya tidak hanya didasarkan pada nilai objektif terkecil, karena *Tabu Search* juga mempertimbangkan status *tabu list*. Solusi atau *move* yang termasuk dalam *tabu list* dapat ditolak sementara untuk mencegah algoritma kembali ke solusi yang baru saja dikunjungi.

Gambar 3.X juga menunjukkan bahwa kandidat yang berada dalam *tabu list* masih dapat diterima apabila memenuhi *aspiration criterion*. Mekanisme ini memungkinkan algoritma menerima solusi tabu apabila solusi tersebut memberikan perbaikan yang signifikan terhadap solusi terbaik yang telah ditemukan. Setelah seluruh kandidat dievaluasi, *Tabu Search* memilih *best admissible move* sebagai solusi berikutnya x^* . Dengan demikian, proses pencarian tidak hanya mengeksplorasi solusi tetangga, tetapi juga menghindari siklus pencarian dan meningkatkan peluang keluar dari *local optimum*.

Tabu list digunakan untuk menyimpan informasi *move* yang untuk sementara tidak boleh digunakan kembali. Mekanisme ini mencegah algoritma kembali ke solusi yang baru saja dikunjungi dan mengurangi risiko terjadinya siklus pencarian. Apabila suatu *move* termasuk dalam *tabu list*, maka *move* tersebut tidak dipilih kecuali menghasilkan solusi yang lebih baik dibandingkan solusi terbaik yang telah ditemukan sebelumnya. Kondisi pengecualian ini disebut *aspiration criterion*.

Dalam penelitian ini, *Tabu Search* digunakan sebagai mekanisme *global optimization* karena mampu mengeksplorasi ruang solusi alokasi *task* secara menyeluruh. Setiap kandidat solusi yang dihasilkan dari

proses *move* dievaluasi berdasarkan fungsi objektif energi dan *latency*. Dengan demikian, *Tabu Search* diarahkan untuk menemukan konfigurasi alokasi *task* yang memiliki nilai objektif lebih rendah dibanding baseline.

Parameter utama yang digunakan pada *Tabu Search* dalam simulasi ditunjukkan pada Tabel 3.X.

Tabel III.5 Parameter Tabu Search

Parameter	Nilai	Keterangan
<i>Maximum iteration</i>	300	Jumlah maksimum iterasi pencarian
<i>Tabu tenure</i>	30	Lama suatu <i>move</i> disimpan dalam <i>tabu list</i>
<i>Candidate moves</i>	70	Jumlah kandidat <i>move</i> yang dievaluasi pada setiap iterasi

Dengan mekanisme tersebut, *Tabu Search* dapat melakukan eksplorasi solusi secara bertahap melalui perubahan assignment *task*. Pendekatan ini sesuai dengan karakteristik permasalahan *task scheduling* yang memiliki ruang solusi besar dan bersifat kombinatorial. Solusi terbaik yang diperoleh dari proses pencarian global selanjutnya dapat digunakan sebagai dasar untuk proses *local refinement* pada mekanisme *diffusion-inspired local balancing*.

d. Representasi *Search Space* pada PSO dan BFO

. Selain *Tabu Search*, fase simulasi program juga menggunakan *Particle Swarm Optimization* (PSO) dan *Bacterial Foraging Optimization* (BFO) sebagai metode pembandingan. Kedua metode tersebut menggunakan ruang pencarian yang sama, yaitu ruang assignment diskrit untuk pemetaan *task* ke *edge nodes*. Perbedaan utamanya terletak pada cara setiap metode mengeksplorasi kandidat solusi. Pada PSO, pencarian dilakukan oleh sekumpulan partikel, sedangkan pada BFO pencarian dilakukan oleh sekumpulan bakteri. Dalam konteks penelitian ini, partikel dan bakteri bukan entitas fisik, melainkan representasi kandidat solusi penjadwalan.

Pada PSO, setiap partikel merepresentasikan satu konfigurasi lengkap alokasi *task* ke *edge nodes*. Posisi partikel dinyatakan sebagai vektor assignment:

$$p^{(k)} = [p_1^{(k)}, p_2^{(k)}, \dots, p_n^{(k)}]$$

dengan $p^{(k)}$ merupakan posisi partikel ke- k , n merupakan jumlah *task*, dan $p_i^{(k)}$ menunjukkan indeks *edge node* yang dipilih untuk mengeksekusi *task* ke- i . Apabila terdapat *medge nodes*, maka setiap elemen posisi partikel harus berada pada rentang indeks node yang valid, yaitu:

$$p_i^{(k)} \in \{0, 1, 2, \dots, m - 1\}$$

Artinya, setiap dimensi pada partikel merepresentasikan satu *task*, sedangkan nilai pada dimensi tersebut menunjukkan node tujuan dari *task* tersebut. Sebagai contoh, apabila suatu partikel memiliki posisi:

$$p^{(k)} = [2, 0, 4, 1, 8]$$

maka *task* pertama dialokasikan ke node 2, *task* kedua ke node 0, *task* ketiga ke node 4, *task* keempat ke node 1, dan *task* kelima ke node 8. Ilustrasi representasi *search space* pada PSO untuk permasalahan *task scheduling* ditunjukkan pada Gambar 3.X.

Gambar 3.X Ilustrasi *search space* pada PSO untuk penjadwalan *task* di lingkungan *heterogeneous edge computing*.

Berdasarkan Gambar 3.X, setiap partikel bergerak dalam ruang assignment diskrit untuk mencari konfigurasi alokasi *task* dengan nilai *objective function* yang lebih rendah. Partikel memiliki informasi *personal best* atau *pBest*, yaitu posisi terbaik yang pernah ditemukan oleh partikel tersebut, serta *global best* atau *gBest*, yaitu posisi terbaik yang ditemukan oleh seluruh populasi partikel. Pergerakan partikel dipengaruhi oleh posisi saat ini, arah ke *pBest*, dan arah ke *gBest*. Setelah posisi partikel diperbarui, posisi tersebut dipetakan kembali ke

indeks node yang valid agar dapat digunakan sebagai solusi assignment *task* ke *edge nodes*.

Karena PSO pada dasarnya dikembangkan untuk ruang pencarian kontinu, sedangkan permasalahan *task scheduling* pada penelitian ini bersifat diskrit, maka posisi partikel perlu dipetakan ke indeks node yang valid. Secara konseptual, proses pemetaan tersebut dapat dinyatakan sebagai:

$$x_i = \text{clip}(\text{round}(p_i), 0, m - 1)$$

dengan x_i merupakan indeks node hasil pemetaan untuk *task* ke- i , p_i merupakan nilai posisi partikel sebelum dipetakan, dan m merupakan jumlah *edge nodes*. Fungsi *round* digunakan untuk mengubah posisi menjadi bilangan bulat, sedangkan fungsi *clip* digunakan untuk memastikan nilai yang dihasilkan tetap berada pada rentang indeks node yang tersedia. Dengan demikian, setiap posisi partikel dapat diterjemahkan menjadi solusi penjadwalan yang valid.

Pada BFO, setiap bakteri juga merepresentasikan satu kandidat solusi penjadwalan *task*. Posisi bakteri ke- k dapat dinyatakan sebagai:

$$b^{(k)} = [b_1^{(k)}, b_2^{(k)}, \dots, b_n^{(k)}]$$

dengan $b_i^{(k)}$ menunjukkan indeks *edge node* tujuan untuk *task* ke- i . Sama seperti pada PSO, setiap dimensi vektor bakteri merepresentasikan satu *task*, sedangkan nilai pada dimensi tersebut menunjukkan node tempat *task* akan dieksekusi. Oleh karena itu, setiap elemen dalam vektor bakteri juga harus berada pada rentang node yang valid:

$$b_i^{(k)} \in \{0, 1, 2, \dots, m - 1\}$$

Dalam proses pencarian BFO, perubahan posisi bakteri merepresentasikan perubahan assignment *task* ke node lain. Perubahan tersebut dilakukan melalui mekanisme *chemotaxis*, yaitu proses eksplorasi lokal untuk membentuk kandidat solusi baru di sekitar posisi bakteri saat ini. Apabila perubahan tersebut menghasilkan nilai *objective function*

yang lebih baik, maka proses pencarian dapat dilanjutkan ke arah tersebut melalui mekanisme *swim*. Ilustrasi representasi *search space* pada BFO ditunjukkan pada Gambar 3.Y.

Gambar 3.Y Ilustrasi *search space* pada BFO untuk penjadwalan task di lingkungan *heterogeneous edge computing*.

Berdasarkan Gambar 3.Y, bakteri bergerak dalam ruang assignment diskrit dengan melakukan perubahan lokal terhadap vektor assignment. Sebagai contoh, perubahan dari:

$$b = [2,0,4,1,8]$$

menjadi:

$$b' = [2,0,4,5,8]$$

menunjukkan bahwa salah satu *task* dipindahkan dari node 1 ke node 5. Kandidat solusi baru tersebut kemudian dievaluasi menggunakan *objective function*. Bakteri dengan kualitas solusi yang lebih baik dipertahankan melalui mekanisme *reproduction*, sedangkan sebagian bakteri dapat dipindahkan secara acak melalui mekanisme *elimination-dispersal* untuk meningkatkan eksplorasi ruang pencarian.

Dengan representasi tersebut, PSO dan BFO menggunakan bentuk solusi yang sama dengan *Tabu Search*, yaitu vektor assignment *task* ke *edge nodes*. Perbedaannya terletak pada strategi eksplorasi ruang solusi. PSO mengeksplorasi solusi melalui pergerakan partikel yang dipengaruhi oleh *pBest* dan *gBest*, sedangkan BFO mengeksplorasi solusi melalui mekanisme *chemotaxis*, *swim*, *reproduction*, dan *elimination-dispersal*. Karena seluruh metode dievaluasi menggunakan representasi solusi dan *objective function* yang sama, hasil perbandingan pada fase simulasi dapat dilakukan secara lebih adil.

Dalam penelitian ini, PSO dan BFO digunakan sebagai metode pembandingan untuk mengevaluasi performa *Tabu Search* dalam menyelesaikan permasalahan *task scheduling*. Hasil perbandingan dari ketiga metode global optimizer tersebut digunakan untuk menentukan kandidat metode yang paling kompetitif sebelum dilanjutkan ke tahap

validasi pada lingkungan VPS. Dengan demikian, representasi *search space* yang seragam menjadi dasar penting agar perbedaan hasil yang diperoleh benar-benar mencerminkan perbedaan strategi optimasi, bukan perbedaan bentuk solusi yang dievaluasi.

e. Peran *Search Space* dalam *Hybrid Tabu-Diffusion*

Pada penelitian ini, representasi *search space* berbasis vektor assignment menjadi dasar bagi penerapan metode *Hybrid Tabu-Diffusion*. Dalam pendekatan ini, *Tabu Search* digunakan sebagai mekanisme *global optimization* untuk mengeksplorasi berbagai kemungkinan alokasi *task* ke *edge nodes*. Hasil pencarian global tersebut berupa solusi assignment yang menunjukkan node tujuan untuk setiap *task*.

Solusi yang dihasilkan oleh *Tabu Search* kemudian dapat diperbaiki melalui mekanisme *diffusion-inspired local balancing*. Mekanisme ini bekerja pada ruang solusi yang sama, tetapi fokus pada penyesuaian lokal terhadap distribusi beban antar node. Jika *Tabu Search* berperan untuk mencari konfigurasi assignment secara global, maka *Diffusion* berperan untuk mengevaluasi kemungkinan perpindahan *task* secara lokal agar distribusi beban menjadi lebih seimbang.

Secara konseptual, solusi hasil pencarian global dapat dituliskan sebagai:

$$x^{global} = [x_1, x_2, \dots, x_n]$$

Kemudian mekanisme lokal melakukan evaluasi terhadap kemungkinan perubahan assignment pada sebagian *task*, sehingga menghasilkan solusi hasil perbaikan:

$$x^{hybrid} = refine(x^{global})$$

dengan *refine(.)* merepresentasikan proses *local refinement* yang dilakukan oleh mekanisme *diffusion-inspired local balancing*. Proses ini tidak mengubah bentuk representasi solusi, tetapi hanya memperbaiki nilai assignment pada elemen tertentu apabila perpindahan tersebut dapat menurunkan nilai *objective function* atau memperbaiki keseimbangan beban.

Pada mekanisme *Diffusion*, perpindahan *task* dilakukan dengan mempertimbangkan kondisi node tujuan, seperti utilisasi CPU, utilisasi memori, estimasi daya, dan karakteristik *latency*. Sebuah *task* dapat dipindahkan dari satu node ke node lain apabila perpindahan tersebut menghasilkan kualitas solusi yang lebih baik berdasarkan evaluasi lokal. Dengan demikian, proses *Diffusion* tidak membentuk ruang solusi baru, melainkan melakukan penyesuaian terhadap solusi yang telah ditemukan oleh *global optimizer*.

Peran *search space* dalam *Hybrid Tabu-Diffusion* dapat dipahami sebagai ruang bersama yang digunakan oleh kedua tahap optimasi. *Tabu Search* mengeksplorasi ruang tersebut secara global untuk menemukan solusi awal yang kompetitif, sedangkan *Diffusion* melakukan penyesuaian lokal pada solusi tersebut. Kombinasi ini diharapkan mampu menghasilkan *assignment task* yang tidak hanya memiliki nilai *objective function* rendah, tetapi juga memiliki distribusi beban yang lebih seimbang pada lingkungan *heterogeneous edge computing*.

Dengan demikian, rancangan *search space* yang seragam memungkinkan metode utama dan metode pembanding dievaluasi pada dasar yang sama. Pada fase simulasi, representasi ini digunakan untuk membandingkan *Tabu Search*, PSO, dan BFO secara adil. Pada fase VPS, representasi yang sama digunakan secara konseptual untuk menghasilkan *assignment task* yang kemudian dikirim dan dieksekusi pada *edge nodes*.

3.4.3. Penerapan Algoritma Optimasi

Di sini kita jelaskan penerapan *Tabu Search*, BFO, PSO, serta *local method* seperti *None*, CPM, dan *Diffusion*. Bagian ini bisa memuat tabel parameter algoritma, seperti jumlah iterasi, *tabu tenure*, jumlah kandidat *move*, parameter *diffusion*, dan sebagainya. Detail kode cukup dijelaskan konseptual, sedangkan potongan kode besar bisa masuk lampiran.

a. Alur Umum Optimasi

Commented [AJ13]: Lebih ke kombinasi antar algoritma, nilai paramet dll

Commented [AJ14R13]: Dijelaskan kombinasi global dan lokal

. Penerapan algoritma optimasi pada penelitian ini dilakukan untuk menghasilkan solusi alokasi *task* ke *edge nodes* dengan nilai *objective function* serendah mungkin. Input utama dalam proses optimasi terdiri dari data *task*, kapasitas sumber daya node, parameter energi, parameter *latency*, serta konfigurasi metode optimasi yang digunakan. Setiap solusi yang dibentuk merepresentasikan vektor assignment, yaitu pemetaan setiap *task* ke salah satu *edge node* yang tersedia.

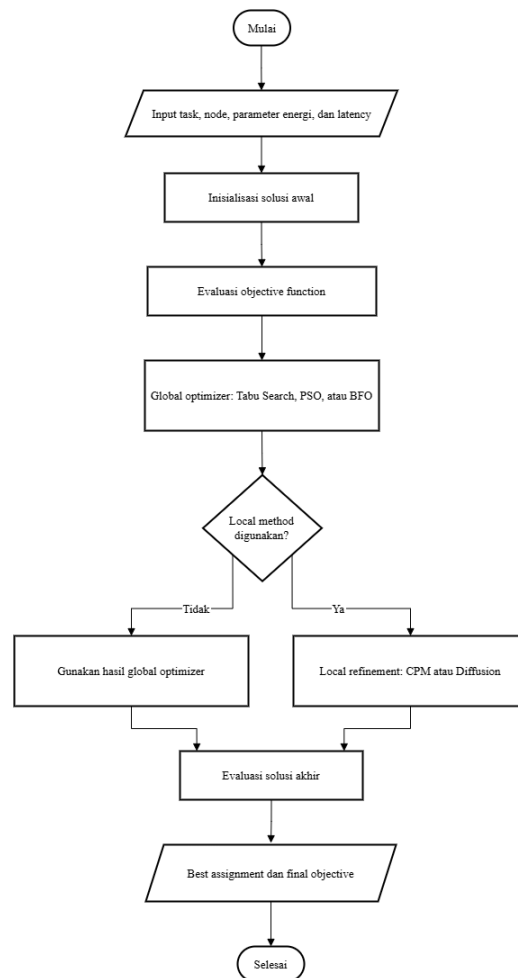
Secara umum, proses optimasi dimulai dengan pembentukan solusi awal. Solusi awal dapat dibentuk menggunakan strategi sederhana, seperti alokasi awal secara acak atau strategi inisialisasi tertentu sesuai metode yang digunakan. Setelah solusi awal terbentuk, solusi tersebut dievaluasi menggunakan *objective function* yang mempertimbangkan konsumsi energi dan *latency*. Nilai objektif yang diperoleh digunakan sebagai dasar untuk menentukan kualitas solusi dan membandingkan solusi tersebut dengan kandidat solusi lain.

Tahap berikutnya adalah proses pencarian solusi oleh *global optimizer*. Pada fase simulasi, *global optimizer* yang dibandingkan terdiri dari *Tabu Search*, PSO, dan BFO. Masing-masing metode mengeksplorasi ruang pencarian dengan strategi yang berbeda, tetapi seluruh metode menggunakan representasi solusi yang sama, yaitu vektor assignment *task* ke node. Setiap kandidat solusi yang dihasilkan oleh *global optimizer* dievaluasi menggunakan *objective function* yang sama agar hasil perbandingan antar metode tetap adil.

Setelah proses pencarian global dilakukan, solusi dapat dilanjutkan ke tahap *local method*. Tahap ini digunakan untuk melihat pengaruh mekanisme perbaikan lokal terhadap kualitas solusi yang dihasilkan oleh *global optimizer*. Pada penelitian ini, *local method* yang digunakan terdiri dari *None*, CPM, dan *Diffusion*. Kondisi *None* menunjukkan bahwa hasil dari *global optimizer* digunakan langsung tanpa perbaikan lokal tambahan. Sementara itu, CPM dan *Diffusion* digunakan sebagai

mekanisme *local refinement* untuk menyesuaikan distribusi *task* pada ruang solusi yang sama.

Alur umum penerapan optimasi pada penelitian ini ditunjukkan pada Gambar 3.X. Berdasarkan alur tersebut, setiap metode optimasi menghasilkan keluaran berupa *best assignment* dan nilai objektif akhir. *Best assignment* menunjukkan konfigurasi alokasi *task* terbaik yang ditemukan oleh metode optimasi, sedangkan nilai objektif akhir digunakan sebagai dasar perbandingan performa pada fase simulasi program. Pada fase VPS, assignment kandidat terpilih kemudian digunakan untuk mengirim dan mengeksekusi *task* pada *edge nodes*.



Commented [AJ15]: Ini lebih ke pengujian

Berdasarkan Gambar 3.X, proses optimasi memiliki dua tahap utama, yaitu pencarian global dan perbaikan lokal. Pencarian global bertujuan mengeksplorasi ruang solusi secara menyeluruh untuk menemukan assignment yang memiliki nilai objektif rendah. Perbaikan lokal digunakan sebagai tahap tambahan untuk menyesuaikan hasil pencarian global apabila mekanisme lokal diaktifkan. Dengan alur ini, penelitian dapat mengevaluasi pengaruh masing-masing *global optimizer* dan *local method* terhadap kualitas solusi *task scheduling*.

b. Penerapan Global Optimizer

Global optimizer pada penelitian ini digunakan untuk mengeksplorasi ruang solusi alokasi *task* secara menyeluruh. Setiap *global optimizer* bekerja pada representasi solusi yang sama, yaitu vektor assignment *task* ke *edge nodes*. Dengan representasi tersebut, setiap kandidat solusi menunjukkan konfigurasi lengkap node tujuan untuk seluruh *task*. Kualitas setiap kandidat solusi kemudian dievaluasi menggunakan *objective function* berbasis energi dan *latency*.

Pada fase simulasi program, penelitian ini membandingkan tiga *global optimizer*, yaitu *Tabu Search*, *Particle Swarm Optimization* (PSO), dan *Bacterial Foraging Optimization* (BFO). Ketiga metode tersebut digunakan untuk melihat pengaruh strategi pencarian yang berbeda terhadap kualitas hasil *task scheduling*. *Tabu Search* digunakan sebagai metode utama karena sesuai dengan karakteristik permasalahan kombinatorial, sedangkan PSO dan BFO digunakan sebagai metode pembanding berbasis populasi.

Tabu Search diterapkan sebagai pencarian global berbasis *neighborhood search*. Proses pencarian dimulai dari solusi awal, kemudian algoritma membentuk sejumlah solusi tetangga melalui operasi *move*. Operasi *move* merepresentasikan perpindahan satu *task* dari node asal ke node tujuan lain. Setiap kandidat solusi hasil *move* dievaluasi menggunakan *objective function*, kemudian kandidat terbaik dipilih sebagai solusi berikutnya dengan mempertimbangkan *tabu list* dan *aspiration criterion*. Parameter yang digunakan pada *Tabu Search* ditunjukkan pada Tabel 3.X.

Tabel 3.X Parameter *Tabu Search*

Parameter	Nilai	Keterangan
TABU_MAX_ITER	300	Jumlah maksimum iterasi pencarian
TABU_TENURE	30	Lama <i>move</i> disimpan dalam <i>tabu list</i>

TABU_CANDIDATE_MOVES	70	Jumlah kandidat <i>move</i> yang dievaluasi pada setiap iterasi
TABU_STAGNATION_TRIGGER	12	Batas iterasi tanpa perbaikan sebelum memicu mekanisme tambahan

Berdasarkan Tabel 3.X, *Tabu Search* dijalankan dengan jumlah iterasi maksimum 300 untuk memberikan ruang eksplorasi yang cukup besar terhadap kombinasi assignment *task*. Nilai *tabu tenure* sebesar 30 digunakan untuk mencegah algoritma kembali ke *move* yang baru saja digunakan dalam periode tertentu. Sementara itu, jumlah kandidat *move* sebanyak 70 digunakan agar setiap iterasi tidak mengevaluasi seluruh kemungkinan perpindahan *task*, tetapi tetap memiliki pilihan kandidat yang cukup untuk mencari solusi yang lebih baik. Parameter TABU_STAGNATION_TRIGGER digunakan sebagai penanda ketika proses pencarian tidak mengalami perbaikan dalam sejumlah iterasi tertentu.

PSO diterapkan sebagai metode pembandingan berbasis populasi partikel. Dalam penelitian ini, setiap partikel merepresentasikan satu kandidat solusi berupa vektor assignment *task* ke node. Posisi partikel diperbarui berdasarkan kombinasi antara posisi sebelumnya, pengalaman terbaik partikel tersebut atau *pBest*, dan pengalaman terbaik seluruh populasi atau *gBest*. Karena representasi assignment bersifat diskrit, posisi partikel yang diperbarui perlu dipetakan kembali ke indeks node yang valid sebelum dievaluasi menggunakan *objective function*. Parameter yang digunakan pada PSO ditunjukkan pada Tabel 3.X.

Tabel 3.X Parameter PSO

Parameter	Nilai	Keterangan
PSO_SWARM_SIZE	40	Jumlah partikel dalam populasi PSO
PSO_MAX_ITER	150	Jumlah maksimum iterasi PSO
PSO_W	0.73	Bobot inersia dalam pembaruan kecepatan partikel

PSO_C1	1.6	Koefisien pengaruh <i>personal best</i>
PSO_C2	1.6	Koefisien pengaruh <i>global best</i>

Berdasarkan Tabel 3.X, PSO menggunakan 40 partikel untuk mengeksplorasi beberapa kandidat solusi secara paralel pada setiap iterasi. Nilai iterasi maksimum sebesar 150 digunakan untuk membatasi waktu komputasi selama simulasi. Parameter w , c_1 , dan c_2 digunakan dalam pembaruan kecepatan dan posisi partikel. Bobot inersia w berperan menjaga pengaruh pergerakan sebelumnya, sedangkan c_1 dan c_2 mengatur pengaruh *personal best* dan *global best* terhadap arah pencarian.

BFO diterapkan sebagai metode pembandingan berbasis populasi bakteri. Setiap bakteri merepresentasikan satu kandidat solusi assignment *task* ke *edge nodes*. Proses pencarian dilakukan melalui mekanisme *chemotaxis*, yaitu perubahan posisi bakteri untuk mengeksplorasi solusi baru di sekitar posisi saat ini. Jika perubahan tersebut menghasilkan nilai *objective function* yang lebih baik, bakteri dapat melanjutkan pergerakan ke arah tersebut melalui mekanisme *swim*. Selain itu, BFO juga memiliki mekanisme *reproduction* untuk mempertahankan bakteri dengan kualitas solusi lebih baik dan mekanisme *elimination-dispersal* untuk memindahkan sebagian bakteri secara acak agar eksplorasi ruang solusi lebih luas. Parameter yang digunakan pada BFO ditunjukkan pada Tabel 3.X.

Tabel 3.X Parameter BFO

Parameter	Nilai	Keterangan
N_bacteria	40	Jumlah bakteri sebagai agen pencari solusi
Nc	12	Jumlah langkah <i>chemotaxis</i>
Nre	6	Jumlah tahap <i>reproduction</i>
Ned	6	Jumlah kejadian <i>elimination-dispersal</i>
Ped	0.2	Probabilitas bakteri mengalami <i>elimination-dispersal</i>

Berdasarkan Tabel 3.X, BFO menggunakan 40 bakteri sebagai populasi kandidat solusi. Parameter Nc menentukan jumlah langkah *chemotaxis*

yang digunakan untuk mengeksplorasi solusi baru di sekitar posisi bakteri. Parameter *Nre* mengatur jumlah tahap *reproduction* untuk mempertahankan bakteri dengan kualitas solusi lebih baik. Sementara itu, *Ned* dan *Ped* digunakan untuk mengatur proses *elimination-dispersal*, yaitu mekanisme pemindahan sebagian bakteri secara acak agar algoritma tidak terlalu terfokus pada satu wilayah ruang solusi.

Ketiga *global optimizer* tersebut dijalankan pada skenario simulasi yang sama, yaitu jumlah *task*, jumlah *edge nodes*, dataset, dan *objective function* yang sama. Dengan demikian, perbedaan hasil yang diperoleh dapat dikaitkan dengan perbedaan strategi eksplorasi masing-masing metode. *Tabu Search* mengeksplorasi solusi melalui *move* dan memori pencarian, *PSO* mengeksplorasi solusi melalui pergerakan partikel menuju *pBest* dan *gBest*, sedangkan *BFO* mengeksplorasi solusi melalui mekanisme *chemotaxis*, *reproduction*, dan *elimination-dispersal*. Hasil dari proses pencarian global kemudian dapat digunakan langsung sebagai solusi akhir atau dilanjutkan ke tahap *local method* untuk dilakukan perbaikan lokal

c. Penerapan Local Method

d. Kombinasi dan Parameter Optimasi

3.5. Pengujian

3.5.1. Pengujian menggunakan simulasi program

Di sini dijelaskan cara pengujian pada fase *simulation-based comparison*. Isinya mencakup skenario 9 kombinasi metode, baseline *Round-Robin*, metrik *final objective*, *time-to-target*, *runtime*, serta CDF untuk kandidat terbaik. Bagian ini menjadi dasar rekomendasi metode yang akan divalidasi di VPS. Selain alur juga tujuannya mendapatkan metrik apa saja.

Konvergensi setiap metode, CDF

3.5.2. Pengujian menggunakan lingkungan VPS

Di sini dijelaskan bedanya dengan simulasi program: VPS menjalankan *task* nyata, memakai *random baseline average*, mengukur *real metrics*, dan membandingkan kandidat optimasi terhadap baseline. Kalau ada dashboard monitoring, bagian ini juga menjelaskan bagaimana monitoring dilakukan, misalnya memantau status node, status *task*, dan metrik hasil eksekusi.

BAB IV

HASIL DAN PEMBAHASAN

Bagian ini memuat semua temuan ilmiah yang diperoleh sebagai data hasil Proyek Akhir, atau hasil unjuk kerja purwarupa yang dibuat. Bagian ini diharapkan dapat memberikan penjelasan ilmiah, yang secara logis dapat menerangkan alasan diperolehnya hasil-hasil tersebut. Pada bagian ini, penulis menyusun secara sistematis disertai argumentasi yang rasional tentang hasil unjuk kerja purwarupa maupun informasi ilmiah yang diperoleh dalam Proyek Akhir, terutama informasi yang relevan dengan masalah Proyek Akhir.

Pembahasan terhadap hasil Proyek Akhir yang diperoleh dapat disajikan dalam bentuk uraian logis, kualitatif, kuantitatif maupun statistik. Untuk memperjelas penyajian, hasil Proyek Akhir disajikan secara lengkap dan jelas, seperti: satuan, kondisi eksperimen, dan lain-lain. Pembahasan juga dapat diadakan perbandingan antara hasil yang diperoleh dengan hasil penelitian sebelumnya, atau unjuk kerja purwarupa dengan produk yang ada maupun prototipe sebelumnya. Sebaiknya penulis tidak menuliskan kembali prosedur Proyek Akhir pada bab ini.

BAB V

PENUTUP

5.1 Kesimpulan

Kesimpulan merupakan pernyataan singkat dan jelas tentang hasil Proyek Akhir yang diperoleh sesuai dengan tujuan Proyek Akhir.

5.2 Saran

Saran digunakan untuk menyampaikan perbaikan atau cara lain untuk mengatasi masalah atau kelemahan yang baru diketahui setelah Proyek Akhir selesai dilaksanakan atau untuk menunjukkan arah penelitian lebih lanjut.

DAFTAR PUSTAKA

- [1] Satyajit Sinha, "State of IoT 2025: Number of connected IoT devices growing 14% to 21.1 billion globally," IoT Analytics. Diakses: 3 April 2026. [Daring]. Tersedia pada: <https://iot-analytics.com/number-connected-iot-devices/>
- [2] F. Golpayegani *dkk.*, "Adaptation in Edge Computing: A Review on Design Principles and Research Challenges," *ACM Trans. Auton. Adapt. Syst.*, vol. 19, no. 3, hlm. 1–43, Sep 2024, doi: 10.1145/3664200.
- [3] M. Ergen *dkk.*, "Edge Computing in Future Wireless Networks: A Comprehensive Evaluation and Vision For 6G and Beyond," *ICT Express*, vol. 10, no. 5, hlm. 1151–1173, Okt 2024, doi: 10.1016/j.ict.2024.08.007.
- [4] I. Rojek, P. Prokopowicz, M. Piechowiak, P. Kotlarz, N. Náprstková, dan D. Mikołajewski, "The Impact of Data Analytics Based on Internet of Things, Edge Computing, and Artificial Intelligence on Energy Efficiency in Smart Environment," *Applied Sciences*, vol. 16, no. 1, hlm. 225, Des 2025, doi: 10.3390/app16010225.
- [5] Drshya Manohar, "An Introduction to Edge Computing in the Era of Connected Devices," Cavli Wireless. [Daring]. Tersedia pada: <https://www.cavliwireless.com/blog/nerdiest-of-things/edge-computing-for-iot-real-time-data-and-low-latency-processing>
- [6] N. Yamsani dan C. R. P, "SLA aware deep reinforcement learning for adaptive EdgeCloud task scheduling," *Sci Rep*, vol. 16, no. 1, hlm. 10037, Feb 2026, doi: 10.1038/s41598-026-40237-8.
- [7] Y. Liu, H. Qu, S. Chen, dan X. Feng, "Energy efficient task scheduling for heterogeneous multicore processors in edge computing," *Sci Rep*, vol. 15, no. 1, hlm. 11819, Apr 2025, doi: 10.1038/s41598-025-92604-6.
- [8] Ye, J, Zhao, Y, dan Yu, Y, "Artificial intelligence for low-carbon energy and information networks," *Nat Rev Electr Eng*, doi: 10.1038/s44287-026-00271-0.
- [9] K. Ma dan L. Xu, "Energy-conscious scheduling in edge environments: hybridization of traditional control and DE algorithm," *Front. Robot. AI*, vol. 12, hlm. 1656516, Des 2025, doi: 10.3389/frobt.2025.1656516.
- [10] D. Sahu *dkk.*, "Optimizing Energy and Latency in Edge Computing through a Boltzmann-Driven Bayesian Framework for Adaptive Resource Scheduling," *Sci Rep*, vol. 15, no. 1, hlm. 30452, Agu 2025, doi: 10.1038/s41598-025-16317-6.
- [11] P. Arroba, R. Buyya, R. Cárdenas, J. L. Risco-Martín, dan J. M. Moya, "Sustainable edge computing: Challenges and future directions," *Softw Pract Exp*, vol. 54, no. 11, hlm. 2272–2296, Nov 2024, doi: 10.1002/spe.3340.
- [12] X. Zhang dan S. Debroy, "Resource Management in Mobile Edge Computing: A Comprehensive Survey," *ACM Comput. Surv.*, vol. 55, no. 13s, hlm. 1–37, Des 2023, doi: 10.1145/3589639.
- [13] H. Asghar dan E.-S. Jung, "A Survey on Scheduling Techniques in the Edge Cloud: Issues, Challenges and Future Directions," 2022, *arXiv*. doi: 10.48550/ARXIV.2202.07799.

- [14] X. Kong, Y. Wu, H. Wang, dan F. Xia, "Edge Computing for Internet of Everything: A Survey," *IEEE Internet Things J.*, vol. 9, no. 23, hlm. 23472–23485, Des 2022, doi: 10.1109/JIOT.2022.3200431.
- [15] M. Mahbub dan R. M. Shubair, "Contemporary Advances in Multi-Access Edge Computing: A survey of fundamentals, architecture, technologies, deployment cases, security, challenges, and directions," *Journal of Network and Computer Applications*, vol. 219, hlm. 103726, Okt 2023, doi: 10.1016/j.jnca.2023.103726.
- [16] H. Nouhas, A. Belangour, dan M. Nassar, "Heuristic-Based Workload Scheduling Approaches in Edge-Cloud Environments: A Review," *IJETT*, vol. 73, no. 11, hlm. 39–50, Nov 2025, doi: 10.14445/22315381/IJETT-V73I11P104.
- [17] S. Dong *dkk.*, "Task Offloading Strategies for Mobile Edge Computing: A survey," *Computer Networks*, vol. 254, hlm. 110791, Des 2024, doi: 10.1016/j.comnet.2024.110791.
- [18] X. Deng, J. Li, E. Liu, dan H. Zhang, "Task Allocation Algorithm and Optimization Model on Edge Collaboration," *Journal of Systems Architecture*, vol. 110, hlm. 101778, Nov 2020, doi: 10.1016/j.sysarc.2020.101778.
- [19] V. Patsias, P. Amanatidis, D. Karampatzakis, T. Lagkas, K. Michalakopoulou, dan A. Nikitas, "Task Allocation Methods and Optimization Techniques in Edge Computing: A Systematic Review of the Literature," *Future Internet*, vol. 15, no. 8, hlm. 254, Jul 2023, doi: 10.3390/fi15080254.
- [20] D. Sahu *dkk.*, "Beyond Boundaries: A Hybrid Cellular Potts and Particle Swarm Optimization Model for Energy and Latency Optimization in Edge Computing," *Sci Rep*, vol. 15, no. 1, hlm. 6266, Feb 2025, doi: 10.1038/s41598-025-90348-x.
- [21] J. Zhai, J. Bi, H. Yuan, J. Zhang, dan R. Buyya, "Energy-Efficient and Latency-Aware Task Offloading for Industrial Cloud-Edge Systems With Heterogeneous CPUs and GPUs," *IEEE Internet Things J.*, vol. 12, no. 13, hlm. 25757–25772, Jul 2025, doi: 10.1109/JIOT.2025.3559690.
- [22] W. Liu *dkk.*, "Resource Scheduling Algorithm for Edge Computing Networks Based on Multi-Objective Optimization," *Applied Sciences*, vol. 15, no. 19, hlm. 10837, Okt 2025, doi: 10.3390/app151910837.
- [23] N. M. Dankolo *dkk.*, "Optimizing Resource Allocation for IoT Applications in the Edge Cloud Continuum Using Hybrid Metaheuristic Algorithms," *Sci Rep*, vol. 15, no. 1, hlm. 14409, Apr 2025, doi: 10.1038/s41598-025-97648-2.
- [24] G. Li dan J. Li, "Optimizing low-power task scheduling for multiple users and servers in mobile edge computing by the MUMS framework," *Heliyon*, vol. 10, no. 11, hlm. e31622, Jun 2024, doi: 10.1016/j.heliyon.2024.e31622.
- [25] X. Jin, S. Zhang, Y. Ding, dan Z. Wang, "Task offloading for multi-server edge computing in industrial Internet with joint load balance and fuzzy security," *Sci Rep*, vol. 14, no. 1, hlm. 27813, Nov 2024, doi: 10.1038/s41598-024-79464-2.

- [26] W. Li, X. Sun, B. Wan, H. Liu, J. Fang, dan Z. Wen, "A hybrid GA-PSO strategy for computing task offloading towards MES scenarios," *PeerJ Computer Science*, vol. 9, hlm. e1273, Apr 2023, doi: 10.7717/peerj-cs.1273.
- [27] J. Pérez, J. Díaz, J. Berrocal, R. López-Viana, dan Á. González-Prieto, "Edge Computing: A grounded theory study," *Computing*, vol. 104, no. 12, hlm. 2711–2747, Des 2022, doi: 10.1007/s00607-022-01104-2.
- [28] W. Z. Khan, E. Ahmed, S. Hakak, I. Yaqoob, dan A. Ahmed, "Edge Computing: A survey," *Future Generation Computer Systems*, vol. 97, hlm. 219–235, Agu 2019, doi: 10.1016/j.future.2019.02.050.
- [29] L. Cao dkk., "Cost Optimization in Edge Computing: a survey," *Artif Intell Rev*, vol. 57, no. 11, hlm. 312, Okt 2024, doi: 10.1007/s10462-024-10947-4.
- [30] G. Carvalho, B. Cabral, V. Pereira, dan J. Bernardino, "Edge Computing: Current Trends, Research Challenges and Future Directions," *Computing*, vol. 103, no. 5, hlm. 993–1023, Mei 2021, doi: 10.1007/s00607-020-00896-5.
- [31] H. Topcuoglu, S. Hariri, dan Min-You Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, hlm. 260–274, Mar 2002, doi: 10.1109/71.993206.
- [32] Y. Mao, C. You, J. Zhang, K. Huang, dan K. B. Letaief, "A Survey on Mobile Edge Computing: The Communication Perspective," 2017, *arXiv*. doi: 10.48550/ARXIV.1701.01090.
- [33] E. M. Ali, J. Abawajy, F. Lemma, dan S. A. Baho, "Analysis of Deep Reinforcement Learning Algorithms for Task Offloading and Resource Allocation in Fog Computing Environments," *Sensors*, vol. 25, no. 17, hlm. 5286, Agu 2025, doi: 10.3390/s25175286.
- [34] M. O. Okello, "Time Governed Multi-Objective Optimization," *EPSTEM*, vol. 16, hlm. 167–181, Des 2021, doi: 10.55549/epstem.1068585.
- [35] L. Yang dkk., "Task Offloading Strategy of Multi-Objective Optimization Algorithm Based on Particle Swarm Optimization in Edge Computing," *Applied Sciences*, vol. 15, no. 17, hlm. 9784, Sep 2025, doi: 10.3390/app15179784.
- [36] K. Deb, *Multi-Objective Optimization using Evolutionary Algorithms*. dalam Wiley Interscience Series in Systems and Optimization. Wiley, 2001. [Daring]. Tersedia pada: <https://books.google.co.id/books?id=OSTn4GSy2uQC>
- [37] S. S. Rao, *Engineering Optimization: Theory and Practice*, 1 ed. Wiley, 2009. doi: 10.1002/9780470549124.
- [38] J. Crank, *The mathematics of diffusion*, 2d ed. Oxford, [Eng]: Clarendon Press, 1975.
- [39] G. Cybenko, "Dynamic load balancing for distributed memory multiprocessors," *Journal of Parallel and Distributed Computing*, vol. 7, no. 2, hlm. 279–301, Okt 1989, doi: 10.1016/0743-7315(89)90021-X.
- [40] F. Glover, "Tabu Search—Part I," *ORSA Journal on Computing*, vol. 1, no. 3, hlm. 190–206, Agu 1989, doi: 10.1287/ijoc.1.3.190.

- [41] H. Rhim dan M. Simic, "Tabu Search." [Daring]. Tersedia pada: <https://www.baeldung.com/cs/tabu-search>
- [42] F. Glover, "Tabu Search—Part II," *ORSA Journal on Computing*, vol. 2, no. 1, hlm. 4–32, Feb 1990, doi: 10.1287/ijoc.2.1.4.
- [43] M. Kaleia, "Tabu Search — gentle introduction." [Daring]. Tersedia pada: <https://medium.com/%40m.kaleia/tabu-search-gentle-introduction-46c479eb6525>
- [44] X. Fan, W.-D. Weber, dan L. A. Barroso, "Power provisioning for a warehouse-sized computer," *SIGARCH Comput. Archit. News*, vol. 35, no. 2, hlm. 13–23, Jun 2007, doi: 10.1145/1273440.1250665.
- [45] S. Deng, H. Zhao, W. Fang, J. Yin, S. Dustdar, dan A. Y. Zomaya, "Edge Intelligence: The Confluence of Edge Computing and Artificial Intelligence," *IEEE Internet Things J.*, vol. 7, no. 8, hlm. 7457–7469, Agu 2020, doi: 10.1109/JIOT.2020.2984887.
- [46] C. Gea, K. J. D. Lase, dan M. Syamsudin, "Implementasi Virtual Private Server untuk Mini Hosting," *JURNAL INFACIT*, vol. 7, no. 01, hlm. 5–9, Jan 2023, doi: 10.61179/jurnalinfact.v7i01.402.
- [47] R. Kaur dan S. Chopra, "Virtualization In Cloud Computing : A Review," *IJSRCSEIT*, hlm. 01–05, Jul 2020, doi: 10.32628/CSEIT20641.

LAMPIRAN

Lampiran memuat keterangan tambahan untuk melengkapi Proyek Akhir. Lampiran data digunakan untuk menyajikan prosedur, program komputer, hasil simulasi, buku, atau keterangan lain yang tidak mungkin disingkat sehingga terlalu panjang untuk dimuat di bagian hasil dan pembahasan Proyek Akhir. Lampiran juga dapat digunakan untuk menampilkan data primer yang diperoleh dalam Proyek Akhir yang tidak dapat diinterpretasikan secara langsung.